

E1. Fundamentos y ciclo de vida de apps

Sitio:

Agencia de Habilidades para el Futuro

Curso:

Desarrollo de Aplicaciones para Dispositivos 2° A

Libro:

E1. Fundamentos y ciclo de vida de apps

Imprimido por:

Johana Micaela Navarro

Día:

miércoles, 1 de abril de 2026, 10:36

Descripción

Tabla de contenidos

- 1. Introducción al libro**
- 2. TEMA I. Conceptos generales de aplicaciones móviles**
 - 2.1. ¿Qué es una aplicación móvil?
 - 2.2. ¿Cuáles son las etapas del desarrollo de una app?
 - 2.3. Elementos del desarrollo
 - 2.4. Cómo elegir tipos de app
- 3. TEMA II. Ciclo de vida de una aplicación móvil**
 - 3.1. Inicio/selección del proyecto
 - 3.2. Ciclo de vida de un desarrollo móvil
 - 3.3. Diseño
 - 3.4. Desarrollo
 - 3.5. Pruebas y calidad
 - 3.6. Implementación / puesta en producción
- 4. Consideraciones sobre el desarrollo móvil**
 - 4.1. Comunes
 - 4.2. de iOS
 - 4.3. de Android
- 5. Documentar una app**
 - 5.1. Plantilla de planificación
 - 5.2. Historias de usuario
- 6. EXTRA: Glosario**
 - 6.1. Glosario



Introducción

Este primer libro es la puerta de entrada al universo de las aplicaciones móviles. Contiene las bases conceptuales de este tipo de desarrollo, presentadas en torno a dos temas:

- 1 **Conceptos generales de aplicaciones móviles**
- 2 **Ciclo de vida de una aplicación móvil**

- Además... ¡tenemos un extra!

Un glosario de términos técnicos que te permitirá familiarizarte con el vocabulario del área.

¡Comencemos!



Conceptos generales de aplicaciones móviles

¡Comenzamos nuestro recorrido sobre [aplicaciones móviles](#) ! Nuestra vida cotidiana y profesional está atravesada por el uso de este tipo de desarrollos... desde pedir comida por el celular o utilizar el GPS para llegar a un lugar, estamos usando aplicaciones móviles.

Definición | Una **aplicación móvil** es el *software* en un dispositivo portátil (similar a los programas en una computadora de escritorio). Desarrollar una aplicación de este tipo supone un largo camino, que consta de varias etapas de desarrollo, desde su conceptualización hasta su planificación.

A lo largo de este tema, estudiaremos en detalle cada etapa.

Vamos a empezar por ahondar en la definición de desarrollo móvil, los elementos que se usan para el desarrollo, como la plataforma -de *software* y de *hardware*- y las aplicaciones móviles, clasificándolas y presentando sus ventajas y desventajas. También, aprenderemos cómo elegir los tipos de app que varía según nuestro público, el tiempo, el rendimiento y acceso al dispositivo y, muy importante, según el presupuesto.

Entramos en un nuevo universo de conocimiento, y [es necesario empezar a familiarizarnos con los términos técnicos](#) de los dispositivos móviles: para eso, tendrás un glosario con los conceptos más importantes.

Algunas preguntas clave que trataremos de responder son:

- ¿Qué tipos de aplicaciones móviles hay?
- ¿En qué se parecen y en qué se diferencian?
- ¿Qué es necesario considerar para elegir una aplicación para nuestro proyecto?
- ¿Qué se realiza en cada etapa de su desarrollo?

¡Adelante!



¿Qué es una aplicación móvil?

El **teléfono celular** revolucionó los formatos de comunicación y a medida que la tecnología avanza cambia la perspectiva en cuanto a su uso.

Podemos decir que **el software transita una nueva era**, ya que las distintas **actividades** que se realizaban por medio de una computadora; ahora **pueden ser ejecutadas en pequeños dispositivos portátiles**. Es así como surgen las populares aplicaciones móviles.

Evolución de las aplicaciones móviles

Aunque parecen un fenómeno reciente, la realidad es que las aplicaciones hace tiempo que están entre nosotros. Últimamente, no sólo se han vuelto más populares y atractivas para los/las usuarios/as, sino también para los/las diseñadores/as y desarrolladores/as que están sacando provecho de las posibilidades que ofrecen las nuevas pantallas de teléfono de mayor calidad; por ello deben trabajar codo a codo para lograr aplicaciones que cuiden todos y cada uno de los detalles.

En esencia, una aplicación no deja de ser un *software*. Para entender un poco mejor el concepto, podemos decir que **las aplicaciones son para los móviles lo que los programas son para las computadoras de escritorio**.

Las aplicaciones comparten la pantalla del teléfono con las webs móviles, pero mientras las primeras tienen que ser descargadas e instaladas antes de usar, a una web puede accederse simplemente usando internet y un navegador; sin embargo, no todas pueden verse correctamente desde una pantalla generalmente más pequeña que la de una computadora de escritorio.

Pero, ¿cuáles son las etapas del desarrollo de una app? **¡Te lo contamos!**



¿Cuáles son las etapas del desarrollo de una app?

Cuando estamos frente al desarrollo de una app puede ocurrir que ya exista una web como antecedente o que el diseño comienza desde cero, que es cuando todavía no hay ni web ni aplicación y hay que decidir por cuál de ellas empezar. Si estamos en este último caso se puede considerar un nuevo concepto conocido como **mobile first**.

Mobile first sugiere plantear el proceso de diseño teniendo en cuenta el móvil en primer lugar.

- La **ventaja** de esta forma de trabajar es que el pensar en el móvil como punto de partida, obliga a concentrarse en lo esencial de un producto y a hacer foco sólo en lo que tiene sentido para este dispositivo.
- Una vez que la aplicación está diseñada, puede preguntarse **cuál es la mejor forma de llevar lo hecho para el teléfono a una pantalla de computadora o a otros dispositivos**, extendiendo y escalando el contenido y repensando la diagramación.
- Todos los dispositivos tienen usos diferentes, y en el momento de adaptar el diseño, **hay que tener en cuenta las características particulares de cada uno de ellos**.

El **proceso de diseño y desarrollo de una aplicación**, abarca desde la concepción de la idea hasta el análisis posterior a su publicación en las tiendas. Durante las diferentes etapas, diseñadores/as y desarrolladores/as trabajan la mayor parte del tiempo de manera simultánea y coordinada.

Las etapas son las siguientes:

Hacé clic en el ícono expandir pantalla del visor y podrás ampliar la imagen

Veamos cómo trabajan solapados los/las diseñadores/as y desarrolladores/as:

Hacé clic en el ícono expandir pantalla del visor y podrás ampliar la imagen

¡Estudiaremos los elementos de desarrollo!



Elementos del desarrollo

Muchos términos son nuevos en tu lenguaje, por eso en el “[glosario de términos técnicos](#)” se explica el significado para que la lectura del material pueda ser comprendida.

¡No dudes en consultarlo haciendo click [aquí](#)!

Para introducirnos en el mundo de las aplicaciones móviles debemos conocer los elementos que se usan para el desarrollo.

Analicemos cuáles son:

Plataforma: Una plataforma es una combinación de *hardware* y *software* utilizado para ejecutar aplicaciones de software. Una **plataforma** puede ser descrita simplemente como un sistema operativo, la arquitectura de una computadora o la combinación de ambos (un ejemplo de una plataforma común es Microsoft Windows que se ejecuta en la arquitectura x86).

Teniendo en cuenta la definición anterior, a continuación te presentamos los distintos tipos de plataformas:

- 1 Una **plataforma de hardware** se refiere a la arquitectura de la computadora o del procesador. Las más comunes son x86 o x64 (para computadoras) o arquitectura ARM para dispositivos móviles.
- 2 Una **plataforma de software** se refiere al sistema operativo, entorno de programación o la combinación de ambos. Una notable excepción a esto es Java, que utiliza un sistema operativo independiente de la máquina virtual para cada código compilado, conocido en el mundo de Java como *bytecode* (es por esto que se dice que Java es multiplataforma).

Ejemplos de plataformas de *software* incluyen:

- Android (sistema operativo móvil)
- iOS (sistema operativo móvil)
- Java
- Linux (x86, x86-64, PowerPC, y otras arquitecturas)
- Mac OS X (x86, x86-64)
- Microsoft Windows (x86, x86-64)

Aplicaciones móviles: Deben correr en un sistema operativo y en la actualidad existen 2 grandes sistemas operativos móviles: Android y iOS. Esto quiere decir que tenemos que sortear de alguna manera el problema de la incompatibilidad entre ambas plataformas.

Tenemos hoy en día 3 posibles soluciones a esto, resultando en 3 tipos de aplicaciones diferentes:

- ❖ **Aplicaciones móviles nativas (Native App):** se desarrollan específicamente para una plataforma. Las aplicaciones nativas sólo pueden ser ejecutadas en la plataforma para la que se desarrollaron, y si se desea que se ejecuten en otra plataforma debe hacerse una aplicación distinta para la misma. Para una aplicación en iOS programamos en el lenguaje propietario de Apple, Swift (también podemos usar Objective-C), y para una aplicación nativa de Android programamos en Java/Kotlin usando Android Studio. En el mundo de los videojuegos encontramos muchísimas aplicaciones nativas que solo existen para la plataforma Windows, si se tiene una computadora con Mac OS o con Linux no se podrá jugar.

- ❖ **Aplicaciones móviles híbridas o multiplataforma:** son aquellas aplicaciones que desarrollamos para múltiples plataformas a la vez, haciendo sólo una aplicación. Para esto podemos usar lenguajes como Flutter o React Native.
- ❖ **Aplicaciones móviles web:** se realiza un único desarrollo para todas las plataformas, pero se acceden desde el navegador web. Se les llama también “aplicaciones web *responsive*” ya que deben programarse teniendo en cuenta que se utilizarán en dispositivos móviles. Cualquier lenguaje web sirve para este tipo de desarrollo, como por ejemplo React.

Te mostramos un cuadro con las ventajas y desventajas de cada tipo de aplicación para que al momento de analizar el desarrollo puedas elegir cual utilizar.

	Ventajas	Desventajas
Nativas	• Tienen el mejor rendimiento posible de todas las aplicaciones móviles. • Tenemos acceso completo a todas las funciones nativas del dispositivo (como cámara, GPS, almacenamiento, sensores, etc.) • Cada aplicación puede ser subida a su respectiva App Store.	• Debemos aprender más de un lenguaje de programación (si es que queremos cubrir todos los dispositivos). • Debemos desarrollar varias aplicaciones (al menos 2), lo cual aumenta considerablemente el costo.
Híbridas	• Al ser multiplataforma se puede subir a la App Store. • Solo se necesita desarrollar una sola aplicación, disminuyendo el tiempo y los costos.	• Tienen menos rendimiento que las aplicaciones nativas. El rendimiento que requiere una aplicación depende del tipo de experiencia que se quiera generar, por ejemplo, hay juegos que requieren una interfaz visual muy optimizada para brindar una mejor experiencia, por lo que el rendimiento es un punto clave. • El acceso al dispositivo es menos permisivo que en las apps desarrolladas en nativo.
Web	• El tiempo y costo de desarrollo son más bajos ya que es más simple hacer una aplicación web <i>responsive</i> que una aplicación móvil. • Es más simple encontrar desarrolladores/as con el conocimiento necesario para hacer este tipo de aplicación.	• No puede subirse a Apps Stores. • Requiere acceso a internet para funcionar. • El acceso a los permisos del dispositivo es significativamente menor al de las aplicaciones híbridas.



Cómo elegir tipos de app

El tipo de aplicación a elegir dependerá de la situación de partida, los recursos y el público al que nos queramos dirigir.

- ❖ **Según nuestro público:** tenemos que tener en cuenta el sistema operativo que utiliza mayoritariamente nuestro público objetivo.
- ❖ **Según el tiempo:** si necesitás contar con una aplicación en el menor tiempo posible, las aplicaciones web y las híbridas serán una gran opción.
- ❖ **Según el rendimiento y acceso al dispositivo:** si lo que necesitás es un gran rendimiento y acceso a funciones específicas del dispositivo, requerís una app nativa.
- ❖ **Según el presupuesto:** si tu presupuesto es limitado, las aplicaciones web e híbridas son las opciones más económicas. Luego, podés elegir en función de si deseás tener presencia en las tiendas de apps o no, entre otros factores.

- En resumen, antes de desarrollar una app, debés sentarte a pensar qué funcionalidades querés que tenga, a quién te dirigirás con ella y ser realista en cuanto a presupuesto.
- Solo a partir de ahí podrás elegir el tipo de aplicación más conveniente para tu negocio.
- Sin embargo, esto es solo el principio si realmente querés triunfar con tu app.



Ciclo de vida de una aplicación móvil

Con este tema nos adentraremos en el estudio del ciclo de vida de un desarrollo de *software* móvil, desde la selección del proyecto, pasando por su diseño y desarrollo de *software*, luego por sus pruebas y calidad, hasta la implementación y puesta en producción.

Definición | Entendemos al **ciclo de vida** como el tiempo de duración de una aplicación de software, desde que se concibe la idea hasta que se retira del uso.

Analizaremos las consideraciones sobre el desarrollo móvil, que comprenden: las consideraciones comunes, las consideraciones de iOS y las consideraciones de Android. También, aprenderemos cómo documentar una app, siguiendo un orden de planificación, con su determinada plantilla, y la historia de usuario.

Hay preguntas claves que vamos a respondernos:

- ¿Cómo seleccionamos un proyecto?
- ¿De qué manera lo diseñamos y luego lo desarrollamos?
- ¿Qué método de prueba llevamos a cabo y cómo evaluamos su calidad?
- ¿Cuándo es preciso su implementación y cuándo su puesta en producción?

La respuesta a estas preguntas será el punto de partida de nuestra búsqueda de conocimiento. ¡Hacia allí vamos!



Inicio/selección del proyecto

La fase de inicio consiste en definir y perfeccionar la idea de una aplicación. Para crear una aplicación correctamente, es importante hacerse algunas preguntas fundamentales.

Antes de publicar una aplicación en una de las App Store públicas debemos observar los siguientes puntos:

- ❖ **Ventaja competitiva:** ¿ya existen aplicaciones similares? Si es así, ¿cómo se diferencia esta aplicación de las otras?
- ❖ **Integración de la infraestructura:** ¿con qué infraestructura existente se integrará o cuál extenderá?
- ❖ **Valor:** ¿qué valor aporta esta aplicación a los usuarios? ¿Cómo la usarán?
- ❖ **Forma/movilidad:** ¿cómo funcionará esta aplicación en un factor de forma móvil? ¿Cómo puedo agregar valor mediante tecnologías móviles, como el reconocimiento de ubicación, la cámara, etc.?



Ciclo de vida de un desarrollo móvil

Una buena gestión del ciclo de vida de las aplicaciones de cualquier tipo, es fundamental para el éxito de cualquier desarrollador/ra de *software* en el mercado actual.

El ciclo de vida de una aplicación es la **vida útil** completa de una aplicación de *software*, desde el concepto hasta el final de su vida útil. Es decir, el término se refiere a todo el proceso desde el momento en que se concibe la idea de la aplicación hasta el momento en que se retira del uso.

Si bien el ciclo de vida de la app la podemos resumir en los pasos de

Hacé clic en el ícono expandir pantalla del visor y podrás ampliar la imagen

A estas etapas las podemos descomponer en otras más detalladas y específicas al desarrollar una aplicación móvil, es decir que, cuando nos referimos a este ciclo de vida, agregamos 5 partes importantes en el proceso:

- 1) **Inicio/selección del proyecto**: todas las aplicaciones se inician con una idea. Normalmente, la idea se perfecciona en una base sólida para una aplicación.
- 2) **Diseño**: la fase de diseño consiste en definir la experiencia del usuario (UX) de la aplicación (como cuál es el diseño general, cómo funciona, etc.), así como convertir esa experiencia del usuario en un diseño de interfaz de usuario (UI) adecuado, normalmente con la ayuda de un diseñador gráfico.
- 3) **Desarrollo**: normalmente, es la fase con un uso más intensivo de recursos, esta es la creación real de la aplicación. Principalmente, esto implica dos partes:
 - a. **Desarrollo *front-end***: se trata del desarrollo de la parte que verá el/la cliente, la capa con la que interactuará el/la usuario/a.
 - b. **Desarrollo *back-end***: es la parte de desarrollo en conexión con un servidor o base de datos, que conecta el *front-end* de la aplicación móvil con los datos.

4) **Pruebas y calidad:** cuando el desarrollo ha avanzado lo suficiente, normalmente el control de calidad empieza a probar la aplicación y se corrigen los errores. A veces, una aplicación pasará a una fase *beta* limitada en la que una audiencia de usuarios más amplia tiene la oportunidad de usarla, enviar comentarios y notificar cambios.

5) **Implementación/puesta en producción:** a menudo, muchas de estas partes se superponen, por ejemplo, es común que el desarrollo siga mientras se finaliza la interfaz de usuario e incluso puede afectar al diseño de la interfaz de usuario. Además, una aplicación puede estar en una fase de pruebas al mismo tiempo que se agregan nuevas características a una nueva versión.

¿Pero qué implica cada fase?



Diseño

Una vez que se hayan determinado las características y funcionalidades de la aplicación, el siguiente paso es intentar resolver la experiencia del usuario o UX.

El UX está formado por la experiencia del usuario y por la interfaz del usuario, veamos de que se trata:

Diseño de la experiencia del usuario	Diseño de la interfaz de usuario (UI)
◦ La experiencia del usuario se efectúa normalmente a través de prototipos con uno de los muchos conjuntos de herramientas de diseño. ◦ Los prototipos de experiencia del usuario permiten diseñar esta experiencia sin tener que preocuparse por el diseño real de la interfaz de usuario. ◦ Al crear prototipos de experiencia del usuario, es importante tener en cuenta las instrucciones de la interfaz para las diferentes plataformas a las que se dirigirá la aplicación. ◦ La aplicación debería "sentirse cómoda" en todas las plataformas. ◦ Además, el propio <i>hardware</i> también impone decisiones de la experiencia del usuario. ◦ Asimismo, el factor de forma también influye en las decisiones de la experiencia del usuario. ◦ Una tableta tiene mucho más espacio y, por tanto, puede mostrar más información. A menudo, lo que necesita varias pantallas en un teléfono, se comprime en una para una tableta.	◦ Una vez determinada la experiencia del usuario, el siguiente paso es crear el diseño de la interfaz de usuario. ◦ Mientras que la experiencia del usuario suele componerse de prototipos en blanco y negro, la fase de diseño de la interfaz de usuario es donde se introducen y finalizan los colores, gráficos, etc. ◦ Es importante dedicar tiempo a un buen diseño de la interfaz de usuario y, por lo general, las aplicaciones más populares tienen un diseño profesional. ◦ Al igual que con la experiencia del usuario, es importante comprender que cada plataforma tiene su propio lenguaje de diseño, por lo que una aplicación bien diseñada puede tener un aspecto diferente en cada plataforma.



Desarrollo

Normalmente, la fase de desarrollo se inicia muy pronto.

Una vez que la idea ha madurado en la fase conceptual o de inspiración:

- Se desarrolla un prototipo de trabajo que valida la funcionalidad, las suposiciones y ayuda a comprender el ámbito del trabajo.



Pruebas y calidad

Definición

Una **prueba** es el proceso de solucionar los errores de la aplicación. No debe entenderse sólo desde un punto de vista funcional (por ejemplo: "Se bloquea al hacer clic en este botón"), sino también de facilidad de uso y rendimiento. Es mejor empezar la estabilización muy pronto en el proceso de desarrollo para que se puedan realizar correcciones antes de que sean costosas.

Normalmente, las aplicaciones pasan por las fases Prototipo, Alfa, Beta y Versión candidata para lanzamiento. Hay personas que las definen de forma diferente, pero, generalmente, siguen el modelo siguiente:

- a) **Prototipo**: la aplicación aún está en fase de prueba de concepto y sólo funcionan la funcionalidad principal o determinadas partes de la aplicación. Hay errores principales.
- b) **Alfa**: la funcionalidad principal tiene normalmente el código completado (compilado, pero no probado por completo). Aún hay errores principales, puede que no haya funcionalidades aisladas.
- c) **Beta**: la mayoría de la funcionalidad está completa y ha pasado al menos una corrección de errores y una prueba ligera. Puede que aún haya problemas conocidos principales.
- d) **Versión candidata para lanzamiento**: toda la funcionalidad está completa y probada. Salvo errores nuevos, la aplicación es una versión candidata para lanzamiento.

Nunca es demasiado pronto para empezar a probar una aplicación.

Por ejemplo

Si se encuentra un problema importante en la fase de prototipo, aún se puede modificar la experiencia del usuario de la aplicación para darle cabida. Si se encuentra un problema de rendimiento en la fase *alfa*, es lo suficientemente pronto para modificar la arquitectura antes de que se haya generado mucho código basado en suposiciones falsas.

Normalmente, según una aplicación avanza en el ciclo de vida, se abre a más personas para que la prueben, envíen comentarios, etc.

Por ejemplo

Las aplicaciones prototipo pueden mostrarse sólo a partes interesadas clave o ponerse a disposición de estas partes, mientras que las aplicaciones de versión candidata para lanzamiento pueden distribuirse a los clientes que se hayan registrado para obtener acceso anticipado.

Para realizar primeras pruebas y la implementación en pocos dispositivos, normalmente es suficiente implementar directamente desde un equipo de desarrollo. En cambio, si se va ampliando el público, se puede volver más complicado rápidamente. Por tanto, hay una serie de **opciones de implementación de pruebas** que facilitan este proceso al permitirle invitar a usuarios a un grupo de pruebas, lanzar compilaciones en la web y proporcionar herramientas que permiten que los usuarios envíen comentarios.



Implementación / puesta en producción

Una vez que la aplicación se ha estabilizado o probado, es el momento de **publicarla**. Hay una serie de opciones de distribución diferentes, dependiendo de la plataforma.



Consideraciones sobre el desarrollo móvil

Aunque desarrollar aplicaciones móviles no es fundamentalmente diferente que el desarrollo web o de escritorio tradicional en términos de proceso o arquitectura, existen algunas consideraciones que se deben tener en cuenta, veamos cuales son:

- 1 Consideraciones comunes
- 2 Consideraciones de IOS
- 3 Consideraciones de Android

A continuación profundizarás más en cada una de ellas.



Comunes

Estas consideraciones comunes abordan aspectos fundamentales que deben tenerse en cuenta independientemente del tipo de dispositivo o plataforma de desarrollo.

Consideraciones

Descripción

Multitarea

- Implica la gestión eficiente del rendimiento y la energía cuando varias aplicaciones se ejecutan simultáneamente.
- Por ejemplo, en dispositivos móviles, solo una aplicación puede estar en primer plano a la vez para maximizar el espacio en pantalla y reducir el consumo de batería.
- La multitarea debe abordarse considerando las diferencias en la gestión entre plataformas como iOS y Android.

Factor de forma

- Refiere a adaptar el diseño de la interfaz de usuario según el tamaño y las características del dispositivo, ya sea teléfono o tableta.
- Por ejemplo, el espacio limitado en la pantalla de un teléfono requiere un diseño más compacto y simplificado, mientras que las tabletas permiten una disposición más espaciosa de los elementos de la interfaz.
- La efectividad en el diseño de la interfaz varía según el factor de forma del dispositivo.

- Considera las diferencias en hardware y características entre dispositivos, lo que puede afectar el rendimiento y la compatibilidad de la aplicación.
- Por ejemplo, una aplicación de videollamadas puede no ser compatible con todos los dispositivos si no todos tienen cámaras funcionales.
- Es esencial adaptar el desarrollo, diseño y pruebas para garantizar la compatibilidad con una amplia gama de dispositivos:

Fragmentación del sistema operativo y del dispositivo

Planeación y conceptualización: hay que tener en cuenta que el hardware y las características varían de un dispositivo a otro, puede que una aplicación que se basa en determinadas características no funcione correctamente en algunos dispositivos. Por ejemplo, no todos los dispositivos tienen cámaras, por lo que, si se está creando una aplicación de mensajería de vídeo, puede que algunos dispositivos reproduzcan vídeos, pero no los puedan grabar.

Diseño: al diseñar la experiencia del usuario (UX) de una aplicación, hay que prestar atención a las diferentes relaciones y tamaños de pantalla de los dispositivos. Además, al diseñar la interfaz de usuario (UI) de una aplicación, se deben tener en cuenta diferentes resoluciones de pantalla.

Desarrollo: al usar una característica del código, siempre se debe probar primero la presencia de esa característica. Por ejemplo, antes de usar una característica de dispositivo, como una cámara, confirmar siempre primero que el sistema operativo tenga esa característica. Después, al inicializar el dispositivo o característica, hay que asegurarse de solicitar la compatibilidad actual del sistema operativo sobre el dispositivo y después usar esas opciones de configuración.

Pruebas: es muy importante probar la aplicación al principio y con frecuencia en dispositivos reales. Puede haber incluso dispositivos con las mismas especificaciones de hardware en que varíe mucho su comportamiento.

Recursos limitados

- Reconoce las limitaciones de memoria, procesamiento y rendimiento en dispositivos móviles en comparación con equipos de escritorio.
- Por ejemplo, cargar imágenes de alta calidad puede agotar rápidamente la memoria disponible, y las aplicaciones que requieren mucho procesamiento pueden impactar negativamente en el rendimiento del dispositivo.

Debido a consideraciones como estas, **es importante crear el código de manera inteligente y realizar la implementación pronto y con frecuencia** en dispositivos reales para validar la capacidad de respuesta.

Según el tipo de dispositivo y la plataforma de desarrollo debemos tener en cuenta diferentes consideraciones



De IOS

Consideraciones

Descripción

Multitarea

- La multitarea está muy controlada en iOS y hay una serie de reglas y comportamientos que la aplicación debe cumplir cuando otra aplicación está en primer plano, de lo contrario, iOS cancelará la aplicación.

Recursos
específicos del
dispositivo

- En un factor de forma determinado, el hardware puede variar considerablemente entre diferentes modelos.
- Por ejemplo, algunos dispositivos tienen una cámara trasera, algunos también tienen una cámara frontal y otros no tienen ninguna.
- Algunos dispositivos antiguos (iPhone 3G y anteriores) ni siquiera permiten la multitarea.

Restricciones
específicas del
sistema operativo

- Para asegurarse de que las aplicaciones respondan correctamente y sean seguras, iOS impone una serie de reglas que deben cumplir las aplicaciones.
- Además de las reglas con respecto a la multitarea, hay una serie de métodos de evento de los que la aplicación debe volver en una determinada cantidad de tiempo, de lo contrario, iOS la cancelará.
- También las aplicaciones se ejecutan en lo que se conoce como espacio aislado, un entorno que aplica restricciones de seguridad que restringen a qué puede tener acceso la aplicación.
- Por ejemplo, una aplicación puede leer y escribir en su propio directorio, pero si intenta escribir en otro directorio de aplicaciones, se cancelará.



De Android

Consideraciones

Descripción

Multitarea

- La multitarea en Android tiene dos componentes; el primero es el ciclo de vida de la actividad:

Cada pantalla de una aplicación de Android se representa mediante una actividad y hay un conjunto específico de eventos que se producen cuando una aplicación se coloca en segundo plano o vuelve al primer plano.

Las aplicaciones deben respetar este ciclo de vida para responder y comportarse correctamente.

- El segundo componente de la multitarea de Android es el uso de servicios.

Los servicios son procesos de ejecución prolongada que existen independientes de una aplicación y se usan para ejecutar procesos mientras la aplicación está en segundo plano.

- La multitarea está muy controlada en iOS y hay una serie de reglas y comportamientos que la aplicación debe cumplir cuando otra aplicación está en primer plano, de lo contrario, iOS cancelará la aplicación.

Muchos dispositivos y muchos factores de forma

- Google no impone ningún límite en lo que concierne a los dispositivos que pueden ejecutar el sistema operativo Android.
- Este paradigma abierto da como resultado un entorno de productos lleno de una gran variedad de dispositivos diferentes con hardware, resoluciones y relaciones de pantalla, características del dispositivo y capacidades muy diferentes.
- Debido a la fragmentación extrema de los dispositivos Android, la mayoría de los usuarios elige los 5 o 6 dispositivos más populares para los que diseñar y probar y les da prioridad a estos.

Consideraciones de seguridad

- Todas las aplicaciones en el sistema operativo Android se ejecutan bajo una identidad distinta y aislada con permisos limitados.
- De manera predeterminada, las aplicaciones pueden hacer muy poco.
- Por ejemplo, sin permisos especiales, una aplicación no puede enviar un mensaje de texto, determinar el estado del teléfono, ni siquiera obtener acceso a Internet.
- Para tener acceso a estas características, las aplicaciones deben especificar en su archivo de manifiesto de la aplicación qué permisos quieren y cuándo se instalan.
- El sistema operativo lee los permisos, notifica al usuario que la aplicación pide esos permisos y, después, permite al usuario continuar o cancelar la instalación.
- Este es un paso esencial en el modelo de distribución de Android, debido al modelo de tienda de aplicaciones abierto, ya que las aplicaciones no se mantienen de la misma forma que en iOS.



Documentar una app

Uno de los elementos más importantes en el ciclo de vida del desarrollo de un *software* es el de su **documentación**, pues esta ayuda entender de manera sencilla los programas y procesos, evitando así que el conocimiento se pierda.

Generalmente **se siguen pasos de alguna metodología** para ajustarse a normas y reglas entre los participantes de la tarea de documentación.

Proponemos documentar el desarrollo de la app usando metodologías ágiles, dentro del proceso de desarrollo brinda mayor flexibilidad y rapidez en cuanto a las modificaciones que requiera el proyecto, mediante la inclusión constante del usuario en las actividades requeridas; por otro lado, beneficia la aplicación móvil al asegurar la calidad, usabilidad y demás requerimientos especiales que posee el *software* móvil.

Veamos entonces como aplicar esta metodología.

Pasemos a describir de manera concisa las etapas:

Hacé clic en el ícono expandir pantalla del visor y podrás ampliar la imagen

El modelo propone en primer lugar **realizar una correcta planificación de las principales dimensiones del proyecto**, de esta manera se conoce el entorno y modelo de negocio que posee la empresa y con ello dar paso al planeamiento de soluciones.

Cuando la planificación esté lista se procede **al modelado de la base de datos**, así como la **elaboración de prototipos** que permitirán a todos los interesados tener una noción del entregable o producto final.

La **ejecución o codificación** es la parte más extensa del proyecto, los/las desarrolladores/as se limitan a realizar lo que está planificado para dicho *sprint*. Para que una aplicación pueda ser lanzada a los usuarios finales, esta debe pasar por una serie de **pruebas** que permitan garantizar la calidad de la misma, donde **en caso de identificar errores** o no alcanzar un determinado requerimiento, se debe llevar un **registro de cambios**.

Fases

Cada una de las fases posee actividades organizadas con el objetivo de obtener un producto funcional como entregable al terminar cada *sprint*, donde además se obtendrá los cambios necesarios a realizarse lo cuáles serán discutidos antes de empezar un nuevo *sprint*, esto con la finalidad de ofrecer una aplicación de calidad.

Vamos a documentar la primera etapa.



Plantilla de planificación

Esta planilla nos da la primera aproximación al desarrollo, una vez analizada podemos completar la planilla de constitución del proyecto y pasar a las historias de usuario.

Hacé clic en el ícono expandir pantalla del visor y podrás ampliar la imagen

Actividad

La acción o tarea que se va a realizar dentro del proyecto.

Descripción

Explicación detallada de la actividad, indicando cómo se llevará a cabo y qué se espera lograr.

Entorno de estudio

El contexto donde se desarrolla el proyecto, en este caso, entorno utilizado para programar y modalidad remota.

Objetivo del Proyecto

La meta principal que se busca alcanzar con el proyecto.

Identificar los procesos

Listar los procesos internos que forman parte del proyecto y que deben ser analizados o gestionados.

Análisis de factibilidad

- **Operativa:** Evaluar si el proyecto se puede llevar a cabo con los recursos y tiempo disponibles.
- **Técnica:** Verificar si la tecnología y conocimientos necesarios están disponibles.
- **Económica:** Analizar de manera simbólica o estimada si el proyecto es viable en términos de costos, pudiendo aclarar que se trata de un proyecto académico.

Gestión de Stakeholders

Los stakeholders son todas las personas, grupos o instituciones que **tienen interés en el proyecto, se ven afectados por él o participan de alguna manera**. Gestionarlos significa **identificarlos, entender sus necesidades o expectativas, y planificar cómo se va a interactuar o comunicar con ellos durante el proyecto**.

Por ejemplo, incluye a quienes **reciben el resultado**, quienes **lo implementan**, y quienes **controlan o evalúan el proyecto**. En un proyecto académico, también se consideran los **miembros del equipo y el docente** como stakeholders, aunque sea de manera simulada.

Roles

Definir las responsabilidades de cada miembro del equipo dentro del proyecto.

Ejemplo:

Actividad

Desarrollo de una aplicación móvil para la gestión de reservas y hospedajes en un hotel.

Descripción

La app permitirá que el personal de recepción registre reservas, valide disponibilidad de habitaciones, realice check-in y check-out de huéspedes, y emita comprobantes básicos. Se busca digitalizar el proceso para hacerlo más ágil y organizado.

Entorno de estudio

El proyecto se desarrollará en **Android Studio** con **Kotlin** y **SQLite** como base de datos local. Modalidad de trabajo: **remota y colaborativa** en equipo.

Objetivo del Proyecto

Diseñar y desarrollar una aplicación académica que simule el sistema de gestión de un hotel, enfocándose en el flujo de **reserva – ingreso – egreso** de huéspedes.

Identificar los procesos

- Registro de reservas de hospedaje.
- Validación de disponibilidad de habitaciones.
- Cancelación de reservas vigentes.
- Ingreso de huéspedes (check-in).
- Egreso de huéspedes (check-out y facturación).

Análisis de factibilidad

- **Operativa:** viable dentro del cuatrimestre, con alcance reducido (funcionalidades principales).
- **Técnica:** contamos con Android Studio, Kotlin y conocimientos básicos adquiridos en clase.
- **Económica:** al ser un proyecto académico, no hay costos reales asociados.

Gestión de Stakeholders

- **Recepcionistas (usuarios simulados):** quienes interactúan con la app.
- **Gerencia (simulada):** interesada en la información de reservas e ingresos.
- **Equipo de desarrollo (grupo de estudiantes):** encargados de diseñar y programar la aplicación.
- **Docente:** quien supervisa y evalúa el proyecto.

Roles

- **Líder de proyecto:** coordina avances y entrega de la app.
- **Diseñador de interfaz:** arma pantallas y experiencia de usuario.
- **Programador back-end:** se encarga de la lógica y base de datos.
- **Tester/documentador:** verifica funcionamiento y redacta entregables.

Historias de usuario

Se pueden definir como **cartillas** que poseen los pasos a seguir para efectuar una determinada actividad, estas indicaciones deben ser entendibles para los demás interesados, debido a que deben pasar por una revisión donde se valide y confirme su contenido.

Debido a las características propias de los dispositivos móviles como sus pantallas es necesario **diseñar una aplicación que sea fácil de comprender y navegar para el usuario**, pero a su vez tenga todo el potencial para cumplir con su función, por tal razón este modelo recomienda el **uso de bosquejos digitales o prototipos**, de esta manera se busca complementar las historias de usuario.



Historias de usuario

Por lo general, el encargado del producto es quien redacta las historias de usuario con base en la investigación de los usuarios y las organiza en una lista para el equipo de desarrollo, también conocida como trabajo pendiente del producto (*backlog*). Aunque **técnicamente cualquiera puede redactar historias de usuario, es responsabilidad del gerente de producto asegurarse de tener toda la información** que el equipo de desarrollo necesita para ejecutar sus iniciativas.

Luego, el equipo de desarrollo priorizará y decidirá qué historias de usuario abordar durante su reunión de **planificación de sprint**.

Para la definición de las historias de usuario se tomó un modelo que presenta los siguientes campos:

- 1 **ID:** identificador de la historia de usuario.
- 2 **Rol:** tipo de usuario o función que tiene la persona.
- 3 **Característica/funcionalidad:** descripción de la historia de usuario, acción que el usuario va a realizar. Acción principal que el usuario realiza en el sistema para cumplir con una necesidad
- 4 **Razón/resultado:** describe el resultado obtenido al realizar una acción.
- 5 **Número de escenario:** identifica el número de escenarios asociado con la historia de usuario.
- 6 **Criterio de aceptación:** condiciones o requisitos que deben cumplirse para que la historia de usuario sea considerada completa y funcional.
- 7 **Contexto:** describe la situación en la que se ejecuta el escenario y las condiciones bajo las cuales se debe cumplir el criterio de aceptación..
- 8 **Evento:** acción específica que el usuario lleva a cabo dentro del sistema, activando un proceso o interacción.
- 9 **Resultado/comportamiento esperado:** comportamiento esperado o el resultado que debe ocurrir una vez que el usuario haya realizado la acción.

ID	Rol	Características/ Funcionalidad	Razón / Resultado	Nº de Escenario	Criterio de Aceptación	Contexto	Evento	Resultado /Comportamiento e sperado

Ejemplo de una historia de usuario:

Hacé clic en el ícono expandir pantalla del visor y podrás ampliar la imagen



Glosario

Aquí encontrarás una herramienta muy útil para tu inmersión en el mundo del desarrollo móvil: un glosario de términos técnicos.

En este glosario están las [definiciones de los principales términos clave de esta materia](#). Te recomendamos consultarlo cada vez que lo necesites.

Glosario

Capacitor

Capacitor es una herramienta que permite convertir cualquier aplicación web (desarrollada con HTML, CSS y JS) en una aplicación multiplataforma (para iOS, Android, Escritorio o Web), es decir, es el puente de Ionic hacia lo nativo. Capacitor nos permite acceder a los recursos nativos de cada dispositivo (cámara, GPS, multimedia, almacenamiento, etc.) a través de una comunicación sencilla.

Framework

Conjunto estandarizado de conceptos, prácticas y criterios para enfocar un tipo de problemática particular. Su objetivo principal es ofrecer funcionalidad extra a un lenguaje de forma autocontenida para que la programación sea más simple.

Gestor de paquetes

Software que maneja de forma automática los paquetes que podamos necesitar. Se encargan de instalar, actualizar, configurar y eliminar de forma automática los paquetes que se desee utilizar. Los gestores de paquetes más conocidos son npm y yarn.

JavaScript

Lenguaje web por excelencia, todas las páginas web modernas están hechas con JavaScript, aunque la mayoría utiliza algún tipo de framework.

Lenguaje de programación

Lenguaje formal que da a una persona la capacidad de escribir una serie de instrucciones en forma de algoritmos con el fin de controlar el comportamiento de un sistema informático.

Open-Source Software

Es un software que tiene una licencia abierta, esto quiere decir que permite que los/las usuarios/as lo utilicen de forma libre. El código Open-Source es de uso libre y gratuito para cualquiera que desee usarlo, para cualquier propósito.

Paquete

Archivos de software que traen data y metadata necesaria para programar. Pueden incluir funcionalidades puntuales o componentes totalmente nuevos y pre-programados.

Software Development Kit (SDK)

Colección de herramientas que facilitan el desarrollo de software. Estos SDK suelen ser específicos a un lenguaje y una plataforma en particular. Por ejemplo, si queremos desarrollar aplicaciones móviles para Android necesitamos el JDK (Java Development Kit) y si queremos desarrollar aplicaciones para iOS necesitamos el iOS SDK.

Software multiplataforma

Para que el software pueda ser considerado multiplataforma, debe ser capaz de funcionar en más de una arquitectura de computadora o sistema operativo. Esto puede ser una tarea que consume tiempo, ya que los diferentes sistemas operativos tienen diferentes interfaces de programación de aplicaciones o API. Lo que funciona sin problemas en una plataforma puede no compilar en otra, esto aplica no solo para programas de software sino también para lenguajes.

E2 Prototipar y testear

Sitio:

[Agencia de Habilidades para el Futuro](#)

Curso:

Desarrollo de Aplicaciones para Dispositivos 2° A

Libro:

E2 Prototipar y testear

Imprimido por:

Johana Micaela Navarro

Día:

miércoles, 1 de abril de 2026, 10:38

Descripción

Tabla de contenidos

1. Introducción al libro

2. TEMA I. Sistemas de diseño

3. Introducción a los sistemas de diseño

4. ¿Qué es el diseño de apps o aplicaciones móviles?

5. Diseño UX UI

5.1. Principios UX

5.2. Usabilidad

5.3. Accesibilidad

5.4. Relación entre usabilidad y accesibilidad

6. Diseño UI o Interface de usuario

6.1. Conceptos del diseño UI

7. Prototipado

8. TEMA II. Usabilidad

9. Pruebas de usabilidad

9.1. Cuándo hacer pruebas de usabilidad

9.2. ¿Cómo hacer pruebas de usabilidad para dispositivos móviles?

9.3. Beneficios de los test de usabilidad

10. Métodos para la evaluación de la usabilidad

10.1. Método por inspección

10.2. Método por test de usuarios

11. Pruebas de usabilidad: Paso 1 y 2

11.1. Pasos 3 y 4

11.2. Paso 5

11.3. Pasos 6 y 7

12. Conclusiones sobre usabilidad



Introducción

En este libro encontrarás desarrollados, con ejemplos y explicaciones claras, dos temas fundamentales de nuestro recorrido.

- 1 **Sistemas de diseño**
- 2 **Usabilidad: pruebas y evaluaciones**

Te recomendamos leer cada capítulo con atención, tomando notas y volviendo al contenido cada vez que sea necesario.

¡Comencemos!



Sistemas de diseño

Diseñar una app es todo un desafío. Implica pensar su funcionalidad y darle una primera forma, a través de un prototipo, que permita evaluar ese diseño.

- 1 **El diseño** supone pensar en la usabilidad, la accesibilidad y la experiencia de usuario.
- 2 **El prototipado** es una herramienta esencial que permite visualizar y probar conceptos de diseño de manera interactiva antes de la implementación final.

Estos dos elementos forman un enfoque integral para optimizar la creación de aplicaciones, desde la conceptualización hasta la entrega, mejorando la calidad del producto y la experiencia del usuario.

Algunas preguntas que podremos responder a partir de este contenido, son:

- ¿Qué papel desempeña un sistema de diseño en el desarrollo de aplicaciones y cómo contribuye a la coherencia visual?
- ¿Cuáles son los elementos clave de un sistema de diseño para aplicaciones y cómo se implementan en el proceso de diseño?
- ¿Cómo se relaciona el prototipado con el diseño de aplicaciones y cuáles son los beneficios de utilizar prototipos durante el proceso de desarrollo?
- ¿Cuál es la importancia del prototipado en la iteración y validación de conceptos antes de la implementación final?
- ¿Cómo influye la retroalimentación obtenida a través de prototipos en la toma de decisiones de diseño y desarrollo?

A continuación, exploraremos estos dos conceptos y cómo su integración puede potenciar el diseño y desarrollo de aplicaciones de manera significativa.

¡Te lo contamos!



Introducción a los sistemas de diseño

Las aplicaciones móviles nativas para iOS y Android tienen características específicas para cada sistema operativo.

En primer lugar debemos reconocer que **Android usa Material Design** (diseño de materiales), que establece una normativa de diseño enfocado en la visualización del sistema operativo Android.

iOS, por otra parte, **utiliza Human Interface Guidelines** (Pautas de Interfaz Humana) **de Apple**. Son documentos que ofrecen a los diseñadores/as y desarrolladores/as de aplicaciones un conjunto de recomendaciones destinadas a mejorar la experiencia de usuario.

Aunque hay varios elementos similares entre sistemas operativos, vale la pena interiorizar sobre cómo podemos crear diseños útiles para cada uno.

Ahora, hablemos de diseño UX y UI.



¿Qué es el diseño de apps o aplicaciones móviles?

Definición

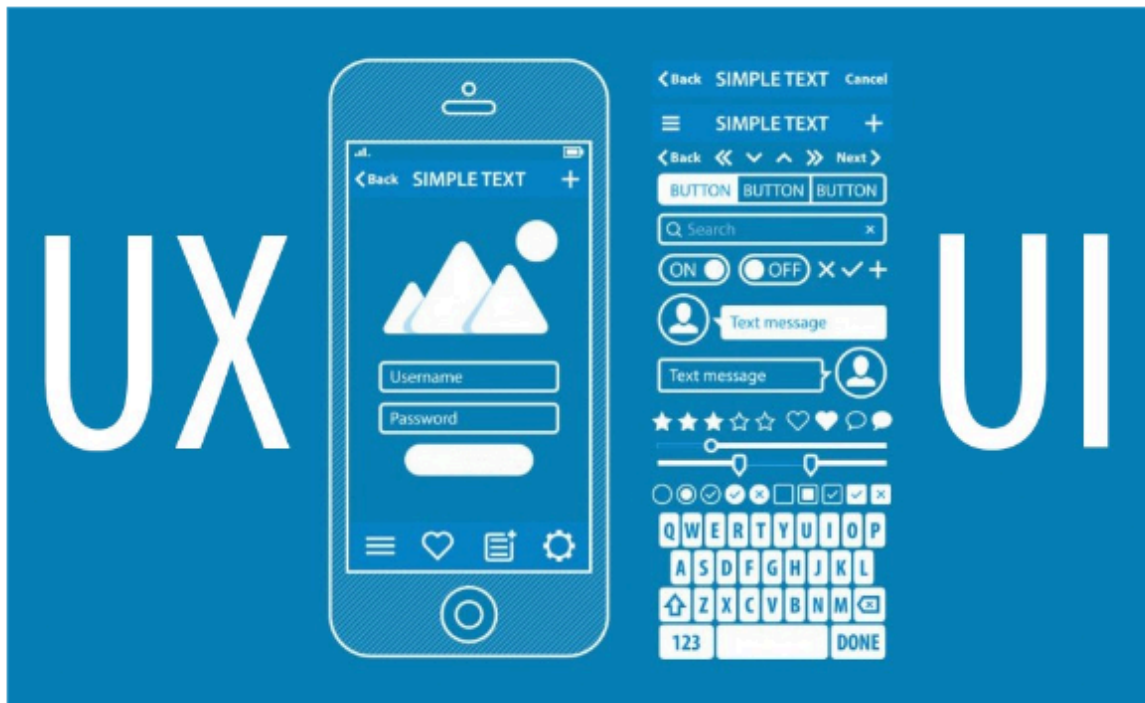
Consiste en **estructurar la navegación de la aplicación** a partir de la lista funcionalidades de la solución y definir las características visuales que se aplicarán a los distintos elementos y pantallas.

Es la manera que tenemos de transmitir al usuario de la aplicación que pensamos en él, para que la app haga atractivo al código que hay detrás. Es un **paso imprescindible antes de un desarrollo adecuado** y debe dar soporte durante toda la creación de la aplicación ya que debe ordenar toda la información de forma jerarquizada comprobando la fluidez entre pantallas para obtener una buena experiencia de usuario.

Te preguntarán: ¿qué sistemas de diseño hay?



Diseño UX UI



¿Qué es *user experience* (UX)?

Es la disciplina que se encarga de **agregar valor a la experiencia del usuario** al utilizar nuestra aplicación. El **objetivo** es que esta experiencia sea significativa, relevante y placentera, y que de un valor para el usuario. Dentro de User experience **hay diversas áreas de especialidad**, una de ellas es el diseño de interfaces (UI).

El **diseño de interfaces** es un proceso iterativo y repetitivo en el que se va mejorando y optimizando

constantemente, teniendo al usuario como centro de referencia y *feedback*. Es clave conocer sus necesidades, temores, objetivos y todo lo que nos pueda brindar información acerca de cómo y para qué usaría nuestra aplicación.

A continuación profundizaremos en los principios de UX.



Principios UX

1- No me hagas pensar

Un diseño debe ser obvio, evidente, claro y fácil de entender. Nuestro diseño debe evitar que el usuario se pregunte: ¿Por dónde empiezo? ¿Puedo hacer clic acá? ¿Qué sucederá si hago clic en este botón?

2- Ley de Fitts

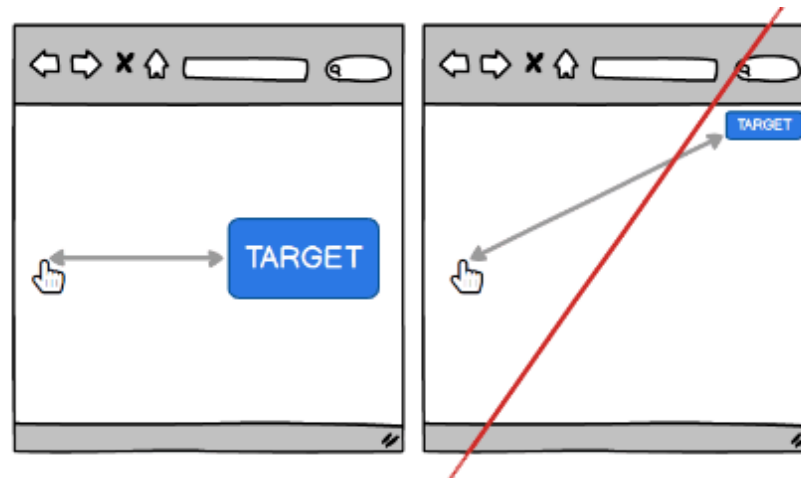
“

"El tiempo necesario para alcanzar un objetivo depende de la distancia y tamaño que hay que recorrer hasta él"

”

La ley de Fitts es un **modelo reconocido** que se aplica en campos como la ergonomía, diseño de interacción y psicomotricidad.

¿Qué significa la Ley de Fitts? Cuando esperamos que el usuario haga clic en un elemento de interacción (un botón o entrada de texto por ejemplo), el tamaño del elemento y su posición con respecto al punto de partida del cursor es sumamente importante.



Podemos observar la distancia entre el cursor y el botón Target en ambas interfaces. En la primera, si bien parece menos estético, el botón es rápidamente reconocible y fácil de acceder para el usuario. En la segunda imagen, el botón sigue siendo notorio debido a que el resto de la pantalla está en blanco; sin embargo, si hubiesen otros componentes como es habitual en una aplicación, probablemente, el tamaño más pequeño y la ubicación arriba a la izquierda harían que el botón no llame la atención del usuario.

Ejemplo

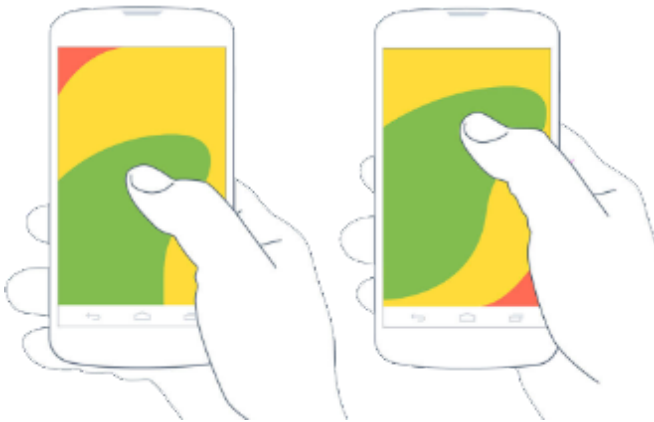
Cuando un usuario está completando un proceso de compra (ya eligió qué artículo quiere, lo agregó al carrito y está terminando de completar sus datos para hacer la compra). En ese instante en el que el cliente está apunto de comprar o arrepentirse, los pequeños detalles en el diseño pueden marcar la diferencia entre una venta exitosa o no, ya que si la interfaz con la que está interactuando no es adecuada y accesible el usuario podría arrepentirse rápidamente o tomarse más tiempo para analizar su decisión.

¿Cómo podemos aplicar la Ley de Fitts a dispositivos móviles? En dispositivos móviles, la Ley de Fitts no puede calcularse según el tiempo que un usuario tarda en llegar hasta el componente/objetivo deseado con su mouse, ya que para interactuar con celulares no usamos un mouse sino nuestros propios dedos.

¿Y con qué dedo utilizamos más estos dispositivos? Con el dedo pulgar. Este dedo tiene la característica de ser más bien corto en comparación con los otros dedos y además tiene menos movilidad. Por supuesto que además depende de la fisonomía de cada persona, por ejemplo un niño tendrá dedos más pequeños que un adulto o adolescente.



Al utilizar un dispositivo móvil podemos observar las diferentes áreas de la pantalla que pueden ser alcanzadas con nuestro dedo pulgar. Un buen diseño móvil debe buscar evitar la tensión muscular lo máximo posible.



3- Ley de Hick

El tiempo que tarda un usuario en tomar una decisión aumenta cuantas más opciones tiene, ya que generalmente intenta contemplar todas las opciones posibles y lo que cada una conlleva, para luego finalmente tomar la que considere mejor opción.

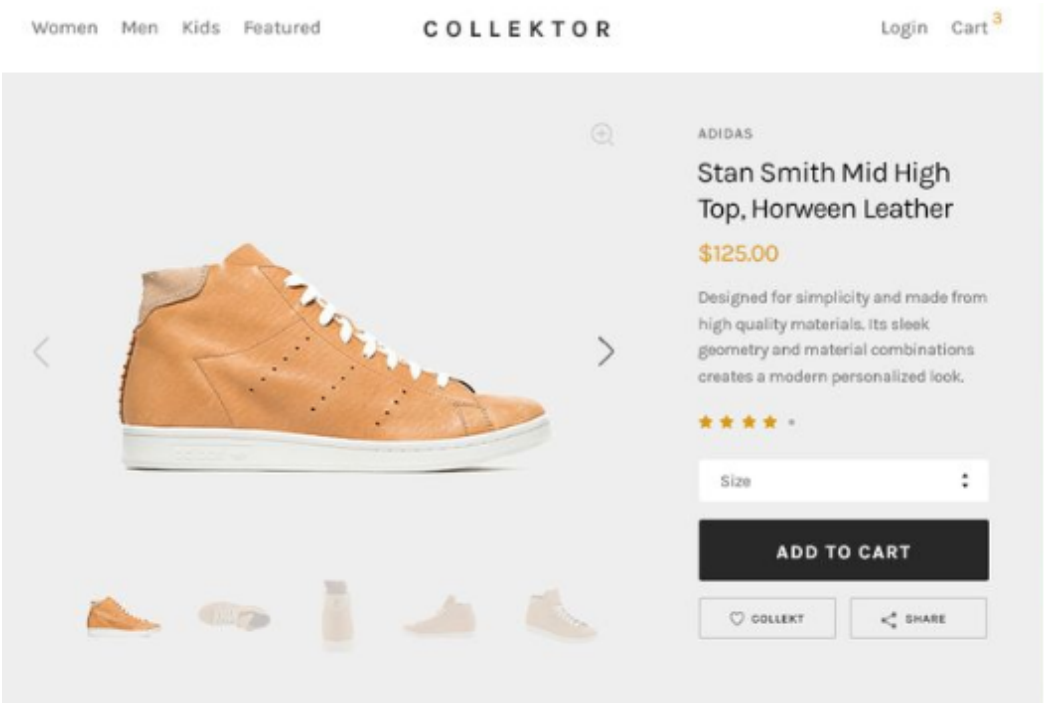
Por ejemplo Si tenemos que armar un formulario de registro muy largo con 20 opciones para elegir se recomienda dividirlo en etapas más simples y obtener un formulario con 4 etapas y 5 opciones por etapa. Airbnb utiliza un sencillo formulario paso a paso para permitir al usuario buscar alojamiento, dividido en 4 etapas: lugar a donde vamos, *checkin*, *checkout* y cantidad de personas.

4- Ley de Jakob

Los usuarios pasan la mayor parte de su tiempo en otras aplicaciones. Esto significa que generalmente los usuarios prefieren que una aplicación funcione de la misma manera que todas las demás que ya conocen. Es por esto que en algunos componentes o interfaces es mejor no abusar de la originalidad o creatividad en nuestros diseños y mantenerlos similares a los de apps más conocidas.

Ejemplo La interfaz típica de *e-commerce* (compra y venta de productos) como Mercado Libre o Amazon

Si bien cada app tiene sus variaciones, es muy común ver interfaces similares a la de la imagen, con la fotografía principal grande y una pequeña galería, el título del producto y precio, descripción detallada del producto más pequeña, valoración de los usuarios (estrellitas) y opciones/variantes para elegir (talle, color, etc.) Podemos notar el típico botón para “agregar al carrito” el cual debe ser fácilmente visible y accesible.



5- Ley de Miller

“

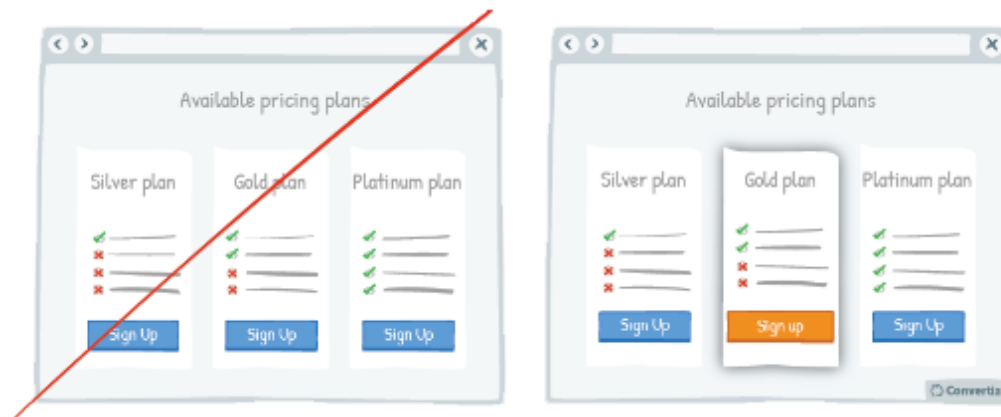
"Las personas pueden recordar hasta 7 elementos distintos en su memoria de trabajo."

”

La memoria de trabajo es la memoria que utilizamos para procesar cosas a corto plazo. Por ejemplo, una buena práctica a nivel diseño consiste en agrupar bien los contenidos de forma que no se creen demasiadas categorías principales en nuestros menús y los usuarios no tengan que recordar tantos elementos.

6- Efecto Von Restorff

Cuando múltiples objetos están presentes, el que difiere del resto probablemente será el más recordado. Un buen ejemplo de este efecto lo podemos ver en las contrataciones de servicios online que nos muestran diversos planes para elegir. Generalmente las marcas suelen diferenciar una por encima de las demás, utilizando el Efecto Von Restorff.



7- Efecto Zeigarnik

La gente recuerda tareas incompletas o interrumpidas más que las tareas completas.

8- Ley de la región común

Los elementos tienden a percibirse en grupo si comparten un área claramente definida. Para aplicar esta ley, los límites pueden generarse simplemente con un buen uso del espacio, sin tener que insertar elementos cromáticos o separadores, creando una sensación de homogeneidad y limpieza.

Uno de los temas importantes del diseño es el concepto de [usabilidad](#).

Te contamos a continuación.



Usabilidad

Definición | La **usabilidad**, significa "facilidad de uso". La definición más extendida, es la ofrecida por la ISO, y que define usabilidad como el "grado de eficacia, eficiencia y satisfacción con la que usuarios específicos pueden lograr objetivos específicos, en contextos de uso específicos".

En la definición podemos observar que la usabilidad se compone de **atributos cuantificables**.

❖ **Eficacia:** Número de errores cometidos por el usuario durante la realización de una tarea.

❖ **Eficiencia:** Tiempo empleado por el usuario para la continuación de una tarea.

La usabilidad de una aplicación debe ser entendida siempre en relación con la **forma y condiciones de uso por parte de sus usuarios**, así como con las características y necesidades propias de estos usuarios.

Un diseño no es en sí mismo usable: "lo es para usuarios específicos en contextos de uso específicos".

Normalmente, toda aplicación se diseña con la intención de satisfacer las necesidades de una audiencia concreta y determinada, por lo que será más usable cuanto más adaptado esté su diseño a esta audiencia específica, y por tanto menos lo esté para el resto de personas.

Definición | El concepto de **usabilidad** puede ser definido, además de como atributo de calidad de una aplicación, como disciplina o enfoque de diseño y evaluación.

Junto con la usabilidad se trabaja el concepto de **accesibilidad**.



Accesibilidad

Un concepto íntimamente ligado al de usabilidad es el de **accesibilidad**. Éste ya no se refiere a la facilidad de uso, sino a la **posibilidad de acceso**.

Hassan Montero, 2003



En concreto el diseño debe, como prerequisite imprescindible para ser usable, posibilitar el acceso a todos sus potenciales usuarios, sin excluir a aquellos con limitaciones individuales, tales como discapacidades, dominio del idioma, o limitaciones derivadas del contexto de acceso tales como software y hardware empleado para acceder, ancho de banda de la conexión empleada, etc.

Mientras que un diseño usable requiere delimitar a su audiencia potencial con el fin de diseñar para lo concreto, un **diseño accesible implica la necesidad de diseñar para la diversidad y heterogeneidad de necesidades de acceso** presentadas por esta audiencia específica.

Cuando la audiencia para la que se diseña es muy amplia y presenta necesidades de acceso muy diferentes, normalmente se hace necesaria la puesta a disposición de varias versiones del diseño o un diseño adaptable, como son las conocidas **"versiones solo texto"** o **versiones en varios idiomas**.

Jakob Nielsen dice que hay 5 **componentes de calidad** que todo producto debería tener cuando se está hablando de usabilidad:

- 1 **Aprendizaje:** qué tan fácil es aprender a usar el producto.
- 2 **Eficiencia:** qué tan eficiente es el producto para realizar las tareas que se le pide.
- 3 **Memoria:** qué tan fácil es recordar cómo utilizar el producto sin tener que pasar por la curva de aprendizaje de nuevo.
- 4 **Errores:** qué tan fácilmente te ayuda a salir el producto de los errores.
- 5 **Satisfacción:** qué tan contentos están los usuarios con el producto.

Ambos conceptos, ¿cómo se relacionan?



Relación entre usabilidad y accesibilidad

Desde el punto de vista conceptual, **ambos términos buscan el mismo objetivo**: que el usuario pueda hacer mejor uso del software.

Para tener en cuenta

La accesibilidad intenta vencer las discapacidades del usuario para acceder a la información; mientras que la usabilidad busca mejorar la experiencia del usuario al usar aplicaciones.



Diseño UI o interface de usuario

Es el área del diseño que se enfoca en la parte visual de un producto digital. Permite crear interfaces intuitivas, usables, interactivas e impactantes.

Características

- **Intuitivo:** El diseño UI busca que los usuarios comprendan fácilmente cómo interactuar con el producto digital, utilizando elementos visuales claros y coherentes.
- **Usable:** Se centra en facilitar la navegación y la realización de tareas dentro del producto, asegurándose de que la experiencia sea fluida y eficiente.
- **Interactivo:** Incorpora elementos interactivos que permiten al usuario realizar acciones y recibir retroalimentación instantánea, mejorando así la experiencia de uso.
- **Impactante:** Busca crear interfaces atractivas y memorables, utilizando una combinación adecuada de colores, formas, tipografía y elementos visuales que generen un impacto visual positivo.
- **Funcionalidad sobre estética:** Aunque el diseño UI se preocupa por la estética, su principal enfoque es en la funcionalidad. Es importante que el diseño satisfaga las necesidades y expectativas de los usuarios para que el producto sea exitoso, más allá de su apariencia visual.



Conceptos del diseño UI

Existen 3 metodologías principales al momento de pensar el diseño de una aplicación:

- 1 **Mejora progresiva (*progressive enhancement*)**: se parte de una base donde tenemos sólo los elementos esenciales del diseño para un móvil, e iremos ampliando la complejidad a medida que vamos subiendo hasta pantallas más grandes como una PC.
- 2 **Degradación agraciada (*graceful degradation*)**: empezamos desde una versión completa y vamos reduciendo la complejidad a medida que bajamos de tamaño de pantalla en el dispositivo.
- 3 **Diseño responsivo**: es una metodología que se utiliza para crear diseños que sean adaptables a diferentes dispositivos. Sus principales características son:
 - **Priorizar el diseño móvil**: al momento de pensar el diseño de una aplicación, siempre debemos empezar por dispositivos móviles. Esto nos garantiza que partimos desde el contenido básico hasta el contenido más complejo.
 - **Separar los componentes de diseño y los de funcionalidad**: utilizar sistemas de grilla y columnas. Las columnas son referencia para dividir el ancho de la pantalla.

Actualmente se alienta más el uso de la mejora progresiva, de manera tal que comencemos el diseño pensando en la versión móvil y luego adaptarlo a pantallas más grandes. Esto se debe a que generalmente es más sencillo comenzar diseñando los componentes básicos para una pantalla móvil pequeña y luego ir adaptándose a pantallas más grandes como tablets o webs. En caso de hacer lo contrario y comenzar diseñando la versión web, podríamos tener dificultades al momento de adaptar los componentes a una pantalla más chica, ya que el espacio es mucho menor y la orientación de la pantalla es vertical.

Todas las decisiones que tomemos deben tener en cuenta y fundamentarse en estos principios:

- ❖ **Accesibilidad**: nuestro producto debería ser usable para cualquier usuario.
- ❖ **Consistencia**: todas las interfaces deben hablarle de la misma forma a los usuarios, ya sea visualmente o por escrito.

Por ejemplo Si en la pantalla principal de nuestra app utilizamos un menú de navegación lateral que se desliza hacia la derecha, se espera que esto se respete en las demás pantallas de la aplicación, salvo algunas excepciones muy específicas. No sería apropiado por ejemplo tener otras pantallas con un menú hamburguesa o que se deslice hacia la izquierda.
- ❖ **Reusabilidad**: se debe priorizar en diseño de componentes reutilizables para evitar el re trabajo y mantener la consistencia.
- ❖ **User control**: debemos generar en el usuario la sensación de “tener el control” de lo que sucede.
- ❖ **Forgiveness**: debemos permitir que el usuario pueda cambiar de opinión o volver a empezar algún proceso
- ❖ **Perceived stability**: debemos transmitir que, aún en una plataforma compleja, tanto los procesos como los elementos son familiares y predecibles.

Cada aplicación podría tener sus propios principios de diseño de acuerdo al público objetivo y requerimientos del producto.

¿Cómo comenzamos a diseñar nuestra app? Podemos usar prototipado antes de realizar el diseño definitivo.

Pero, ¿qué es el prototipado?



Prototipado

La evaluación de la usabilidad de la aplicación se debe realizar desde las primeras etapas de diseño, pero **¿cómo evaluar una aplicación que no está desarrollada?** A través de **prototipos**.

La etapa de prototipado se basa en la elaboración de modelos o prototipos de la interfaz de la aplicación.

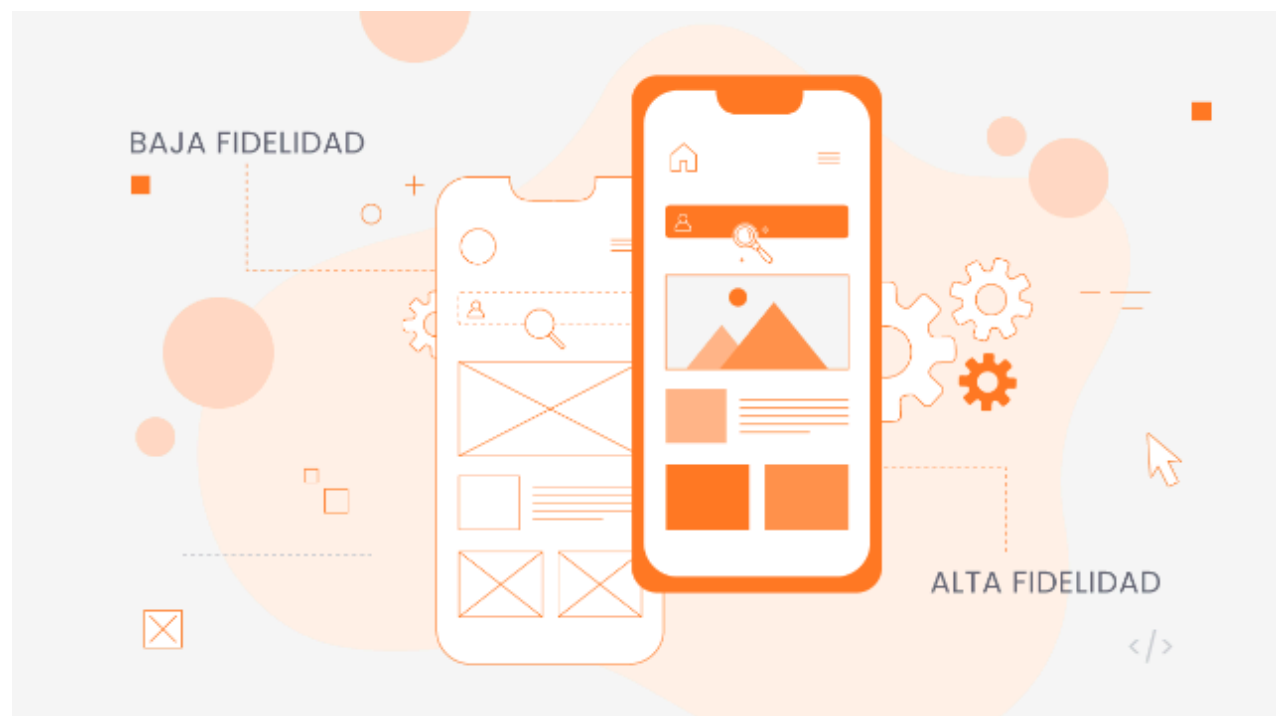
Su aspecto no se corresponde exactamente con el que tendrá la app una vez finalizada, pero pueden servir para evaluar su usabilidad sin necesidad de esperar a su desarrollo.

Podemos clasificar los tipos de prototipado **según el nivel de funcionalidad**:

- 1 **Prototipado horizontal:** se reproduce gran parte del aspecto visual de la app, pero sin que esos modelos de interfaz estén respaldados por la funcionalidad real que tendrá finalmente la app.
- 2 **Prototipado vertical:** se reproduce únicamente el aspecto visual de una parte de la app, pero la parte reproducida poseerá la misma funcionalidad que la app una vez desarrollada.

Según el grado de fidelidad o calidad del prototipo se distingue entre:

- 1 **Prototipado de alta fidelidad:** el prototipo será muy parecido a la app una vez terminada.
- 2 **Prototipado de baja fidelidad:** el aspecto del prototipo distará bastante del que tenga la app final.



En las primeras etapas de desarrollo se puede hacer uso del prototipado en papel o de bajo costo, que consiste en reproducir los aspectos básicos de la interfaz de la app en papel.

El prototipado de apps en papel es una herramienta de creatividad proveniente del diseño de experiencia de usuario que tiene como objetivo definir las pantallas de una aplicación móvil de forma rápida y sencilla. Se utiliza para **refinar y testear tu solución**.

Por ejemplo Podemos reproducir a través de bocetos cómo serán las diferentes pantallas que conformarán la app a desarrollar, cada una en una página de papel diferente. La reproducción suele ser a mano (lápiz y tijeras), por lo que resulta una técnica de prototipado muy económica.

Otra forma de realizar prototipos es mediante la reproducción del aspecto de la app a través de herramientas software. Mediante el procesador de textos podemos esbozar cómo será la interfaz.

Una vez que el prototipo se ha utilizado se tira, no es parte de la aplicación mobile.

La utilidad real del prototipado se fundamenta en que no tendría sentido empezar a implementar una interfaz si no nos hemos asegurado antes de que el diseño es usable. Un prototipeo en papel puede comenzar de manera simplificada y ganar complejidad a lo largo de las iteraciones con el usuario o con el equipo.

¡Es el momento de poner en práctica esta técnica! Te proporcionamos una [plantilla modelo](#) para que puedas practicar.

Podés descargar la plantilla de prototipado de apps en papel haciendo clic [aquí](#) para luego utilizarla en la práctica formativa.



Usabilidad

¡Continuamos nuestro camino de aprendizaje sobre desarrollo de aplicaciones móviles!

En el mundo actual, las aplicaciones móviles se han convertido en un elemento fundamental de nuestra vida cotidiana. Sin embargo, la mera creación de estas apps no es suficiente para garantizar el éxito. La clave está en brindar la mejor experiencia de usuario (UX), lo que implica un proceso de evolución constante y enfoque en las necesidades de los usuarios.

- 1 Introducción y importancia de las pruebas de usabilidad en el desarrollo de aplicaciones móviles:** A lo largo de nuestro recorrido, comprenderemos por qué estas pruebas son esenciales, desde validar ideas y perfeccionar la usabilidad hasta garantizar la eficacia, eficiencia y satisfacción del usuario en un entorno específico de uso.
- 2 Fases de las pruebas de usabilidad y sus beneficios:** Además, presentaremos las diferentes fases de las pruebas de usabilidad, los beneficios que aportan, y detallaremos los métodos y técnicas para su evaluación.
- 3 Métodos y técnicas para la evaluación de la usabilidad:** Se explicarán y discutirán métodos para definir objetivos, tipos de pruebas y métricas a considerar. Además, se analizarán los documentos necesarios, perfiles de participantes y cómo realizar y analizar tareas, todo con un enfoque práctico.

Preguntas clave de este tema:

- ¿Por qué son esenciales las pruebas de usabilidad en el desarrollo de aplicaciones móviles?
- ¿Cuáles son las etapas y métodos para llevar a cabo pruebas de usabilidad en aplicaciones móviles?
- ¿Cuáles son los beneficios específicos que aportan las pruebas de usabilidad en el desarrollo de software para aplicaciones móviles?
- ¿Cómo se evalúa la usabilidad de una aplicación móvil?
- ¿Cuáles son los pasos prácticos para realizar y analizar pruebas de usabilidad en aplicaciones móviles?

¿Preparado/a? ¡Empecemos!



¿Por qué son importantes las pruebas de usabilidad?

Definición

Las pruebas de usabilidad de *software* consisten en evaluar la medida con la que un producto puede ser usado por determinados usuarios para conseguir objetivos específicos con efectividad, eficiencia y satisfacción en un contexto de uso concreto.

En el caso de las aplicaciones móviles, vemos que en la actualidad son un recurso indispensable al facilitar las actividades cotidianas del ser humano, ya que permite diversas tareas tales como ordenar comida, transporte, libros, ropa o cualquier objeto.

Podría pensarse que la creación de apps es suficiente para cubrir con las demandas de los clientes, pero no es así.

Crear la mejor experiencia de usuario (UX) implica una evolución constante, que depende de otros medios para alcanzar un óptimo posicionamiento en el mercado.



¿Cuándo hacer pruebas de usabilidad?

La respuesta es todo el tiempo. Existen dos fases para las pruebas de usabilidad en el proceso de diseño.

1 Fase formativa:

- En esta fase los *test* que se realizan sirven principalmente para validar la idea cuando ya tenés un boceto o las primeras muestras de lo que será el software.
- Son pruebas sencillas que ayudan a perfilar la usabilidad del mismo.
- Estos casos de prueba se llevan a cabo en cualquier lugar, sin necesidad de mucha preparación previa. Sólo se necesita al **moderador** y un **participante**.
- Lo más importante es ver nuestra idea desde los ojos de otra persona, haciendo esto tendremos una imagen más clara de lo que puede llegar a ser el proyecto

2 Fase sumativa:

- Al final del desarrollo del producto se valida la usabilidad real del producto.
- Cuando está terminado, queremos ver cómo el usuario se enfrenta a él en una situación real, idealmente se hace antes de la salida al mercado para ajustar las últimas correcciones.
- Ahora sí necesitamos organizar una prueba de usabilidad más elaborada que cuente con todos los elementos.
- Se requiere: una ubicación con todos los equipamientos necesarios, participantes que apunten a nuestro público objetivo, las tareas que queremos llevar a cabo, etc.

Ya sabemos cuándo hacer las pruebas, pero **¿cómo hacemos estas pruebas de usabilidad?**



¿Cómo hacer pruebas de usabilidad para aplicaciones móviles?

Definición Una aplicación móvil o una app es un programa informático que se ejecuta en teléfonos inteligentes, tabletas u otros dispositivos móviles. Su principal desafío es brindar experiencias sencillas, de fácil acceso, personalizadas y sin inconvenientes.

Cumplir con estas expectativas no es simple y es muy importante que en el desarrollo de una aplicación se consideren etapas y procesos orientados a garantizar dicha experiencia. En este sentido, las pruebas de aplicaciones móviles representan procesos clave para encontrar fallas de diseño o de funcionalidad de un producto.

En el contexto de una aplicación móvil:

una inconsistencia o error puede costarnos malas calificaciones en una tienda de aplicaciones o simplemente un abandono de la app.

¿Tienen beneficios los test de usabilidad? Te lo contamos a continuación.



Beneficios de los test de usabilidad

- 1 **Detecta problemas oportunamente:** el objetivo es detectar las inconsistencias y corregir fallos en el sistema antes de su lanzamiento al mercado, esto permite asegurar que los usuarios tendrán una buena experiencia en distintos escenarios, sin importar el dispositivo móvil en el que se utilice.
- 2 **Te aproxima a una experiencia real:** los *test* de usabilidad permiten interactuar con el sistema tal y como lo haría el usuario, con la finalidad de identificar los posibles problemas que enfrentaría durante su navegación. La percepción de los/las desarrolladores/as no es la misma que tendrá el/la cliente en su intento por satisfacer una necesidad.

Considerará hacer la prueba con usuarios reales, y observá su comportamiento mientras intentan ejecutar tareas en la app, esta información se puede utilizar para realizar mejoras y determinar si se pueden realizar las tareas con éxito y de forma independiente.

- 3 **Identifica defectos y su gravedad:** las apps móviles están sujetas a constantes actualizaciones, ya sea por la integración de mejoras o la corrección de fallos en el sistema. Las pruebas de usabilidad de software permiten identificar errores comunes que enfrentan los/las clientes con el producto informático para su actualización oportuna.

La descarga puede ser el primer acercamiento del/la cliente con la app móvil, lo cual no asegura su permanencia en ella. El tiempo de permanencia en la aplicación y el tipo de uso son algunos de los aspectos que puedes conocer a través de un *test* de usabilidad.

Existen diferentes tipos de prueba para apps:

- 1 **Pruebas de stress:** Este tipo de prueba de usabilidad somete a los sistemas de software a una carga de uso máxima, con la intención de conocer su comportamiento al ser utilizado por los usuarios. El tiempo de carga y respuesta son algunos de los elementos a medir con este recurso.
- 2 **Pruebas móviles:** Una de las ventajas de los dispositivos móviles es su fácil transportación para la realización de tareas en casi cualquier escenario. Sin embargo, esto puede provocar el fallo de las aplicaciones ante situaciones inesperadas, como una mala conexión WI-FI o débil señal de GPS. En este tipo de casos resultan útiles las pruebas de movilidad.
- 3 **Pruebas de accesibilidad:** Hoy en día las empresas buscan un enfoque incluyente para la mayor cantidad de usuarios. Las herramientas de accesibilidad en las apps permiten que las personas con algún tipo de discapacidad —visual, auditiva, cognitiva, etc.— puedan hacer uso de los productos o servicios desde el dispositivo móvil.

Pero, ¿cómo evaluamos la usabilidad?



Métodos para la evaluación de la usabilidad

La **evaluación de la usabilidad**, la etapa más importante en el proceso de diseño, se puede realizar a través de varios métodos o técnicas y sobre diferentes representaciones de la aplicación (prototipos en papel, prototipos *software*).

Existe una gran diversidad de métodos para evaluación de usabilidad.

¡Comencemos!



Método por inspección

Llamado también **evaluación heurística**.

Definición | Los **métodos de inspección de la usabilidad** son aquellos que se basan en el recorrido y análisis del producto software, identificando errores y problemas de diseño.

Diversos autores han propuesto diferentes **conjuntos de heurísticos o principios de usabilidad** a través de los cuales evaluar la usabilidad. Algunos son:

Visibilidad del estado del sistema:

El sistema o aplicación siempre debe informar al usuario acerca de lo que está sucediendo. Por ejemplo, cuando en una interfaz se adjuntan archivos a un mensaje, el sistema debe informar del hecho mostrando un mensaje de espera.

Lenguaje común entre sistema y usuario:

El sistema debe hablar el lenguaje del usuario, huyendo de tecnicismos incomprensibles o mensajes técnicos.

Libertad y control por parte del usuario:

El usuario debe tener el control del sistema, no se le puede limitar tareas. Se debe ofrecer siempre al usuario una forma de "salida de emergencia", como por ejemplo la representada por la opción para "saltar" animaciones de introducción.

Consistencia y estándares:

La consistencia se refiere a, por ejemplo, no utilizar dos rótulos distintos para referirse a un mismo contenido, o no usar estilos diferentes dentro de un mismo producto. Cuanto más se parezca un diseño y su funcionamiento al resto de aplicaciones conocidas, más familiar y fácil de usar resultará para el usuario.

Prevención de errores:

Mejor que un buen mensaje de error es un diseño que prevenga que ocurra el error.

Es mejor reconocer que recordar:

Este principio hace mención a la visibilidad de las diferentes opciones, enlaces y objetos. El usuario no tiene por qué recordar dónde se encontraba cierta información, o cómo se llegaba a determinado lugar de la app.

Flexibilidad y eficiencia de uso:

La app debe ser fácil de usar para usuarios novatos, pero también proporcionar atajos o aceleradores para usuarios avanzados.

Diseño minimalista:

Cualquier tipo de información que no sea relevante para el usuario y que sobrecargue la interfaz debe ser eliminada.

Permitir al usuario solucionar el error:

Por ejemplo, cuando un usuario introduce una consulta en un buscador y no obtiene ningún resultado, se debe informar al usuario sobre cómo solucionar el problema, por ejemplo con mensajes del tipo "introduzca algún sinónimo" o "quiso Ud. decir...". Además, no se debe borrar el contenido de la caja de búsqueda para que el usuario pueda rehacer la consulta.

Ayuda y documentación:

Siempre es mejor que una app se pueda utilizar sin necesidad de ayuda o documentación, aunque en procesos de interacción complejos (como el rellenado de un formulario), se debe proporcionar información de ayuda al usuario.



Método de *test* con usuarios

Definición

El *test con usuarios* es una prueba de usabilidad que se basa en la observación y análisis de cómo un grupo de usuarios reales utiliza la app, anotando los problemas de uso con los que se encuentran para poder solucionarlos posteriormente.

Como toda evaluación de usabilidad, cuanto más esperamos para su realización, más costoso resultará la reparación de los errores de diseño descubiertos. Esto quiere decir que no sólo **debemos realizar este tipo de pruebas sobre la app una vez implementada**, sino también, **sobre los prototipos de la app**.

Es una prueba complementaria a la evaluación heurística, pero un *test* con usuarios es más costoso, por lo que es recomendable realizarlo siempre después de una evaluación heurística, ya que sería desperdiciar tiempo y dinero utilizarlo para descubrir errores de diseño motivados por el no cumplimiento en el desarrollo de principios generales de usabilidad (heurísticos).

Ventaja que ofrecen los *test* de usuario

Por un lado es una demostración con hechos, por lo que sus resultados son más fiables, y por otro porque posibilitan el descubrimiento de errores de diseño imposibles o difíciles de descubrir mediante la evaluación heurística.



¿Cómo hacer las pruebas de usabilidad?

A continuación podrás realizar un recorrido por las **fases o pasos para hacer las pruebas**.

En este apartado te encontrarás con el paso 1 y 2. ¿Empezamos?

1- Definí los objetivos de la prueba:

Es lo primero que tenemos que pensar y lo que determinará el desarrollo de la prueba. Saber qué es lo que desea nuestro cliente es esencial. Por eso hay que seleccionar cuidadosamente qué objetivos se quieren cumplir.

Para elegir esos objetivos, recorreremos las siguientes etapas:

- ❖ **Identificar preguntas y preocupaciones:** lo mejor es sentarse con el equipo o encargados de diseñar el producto, y preguntarnos lo que queremos conseguir. Establecer cuáles son los errores que estamos buscando o las soluciones que queremos encontrar. Conocer la razón del por qué vamos a realizar la prueba de usabilidad es primordial.
- ❖ **Definir el equipo que te acompaña en la prueba:** una vez hecho el paso anterior, hay que limitar las secciones que se van a evaluar. Por ejemplo en una app, saber en qué dispositivo vamos a probar la interfaz o las distintas configuraciones que soporta el software, puede ser confeccionando un prototipo que presentamos al usuario o explicando la interfaz que se quiere poner a prueba.
- ❖ **Ordenar los objetivos por prioridad:** en este punto es normal tener varios objetivos o preocupaciones sin resolver. Lo ideal es que estos no lleguen a ser más de 3 porque no se enfoca el problema principal, así que lo mejor es ordenar la prioridad de los objetivos, estableciendo qué es lo más importante que se quiere obtener del test de usabilidad para que después se puedan seleccionar objetivos secundarios que también se evalúan en la prueba.

Te mostramos algunos **ejemplos** para ayudarte a redactar tus objetivos:

Te mostramos algunos ejemplos para ayudarte a redactar tus objetivos



Un caso habitual es el rediseño de la interfaz, donde podemos establecer los siguientes objetivos:

- ¿La interfaz es accesible para todos los usuarios?
- ¿La interacción con la interfaz es *responsive* en todos los dispositivos?
- ¿Nuestros antiguos usuarios serán capaces de adaptarse a los nuevos cambios?

Otro posible escenario es el lanzamiento de un nuevo producto, un *e-commerce*, y estos serían algunos ejemplos de objetivos:

- ¿La calidad del rendimiento de la apps es óptima?
- ¿La navegación por los menús es lo suficientemente clara?
- ¿Los usuarios pueden completar el proceso de compra de manera satisfactoria?

2- Tipos de prueba de usabilidad:

Ya hemos establecido las bases de la prueba, por lo que tenemos que decidir cuál es el tipo que vamos a realizar y ahora veremos el método para llevarlas a cabo.



Prueba moderada presencial:

Estas son las pruebas básicas de la usabilidad, una persona hará de moderador y otra de participante. El moderador guía al usuario a lo largo de la prueba para que realice las tareas según los objetivos a cumplir.

Es importante que el moderador sea una persona con habilidades sociales capaz de crear un clima agradable. Así el participante se encuentra relajado y sus respuestas serán más naturales. Se pueden realizar en lugares cerrados, normalmente en salas preparadas con todo el equipo necesario. No es obligatorio contar con equipamientos externos, pero esto dependerá de las tareas a realizar.



Pruebas remotas:

Son muy parecidas a las presenciales, pero aquí no hace falta estar en el mismo lugar.

Vía *streaming* el moderador contacta con el participante y le da acceso al software. Lo único que se necesita es una computadora y una cámara. Se suele utilizar programas de grabación de voz, cara y captura de pantalla para no perder ni un solo detalle de la acción del usuario.

Al no necesitar un espacio físico, la fecha y regularidad de las citas es mucho mayor, sin olvidar los ahorros de tiempo y dinero en todo el proceso si se lo compara a las presenciales.



Pruebas no moderadas:

El papel del moderador deja de existir y toda la responsabilidad cae sobre el participante. Antes de la prueba, se prepara un documento explicativo de la sesión y las tareas que va a completar durante el *test*, una vez hecho, se envía el enlace al usuario para que la haga cuando pueda, dentro de unos límites de tiempo preestablecidos. Lo mejor de estas pruebas es que puedes organizar a muchos usuarios a la vez limitando esfuerzos, y en muy poco tiempo.



Guerrilla testing:

Estos *test* son recomendables para aquellos que quieren ahorrar esfuerzos y necesitan resultados inmediatos, lo recomendable es hacerlo con conocidos o familiares, pero hay más opciones.

Los mejores lugares para un guerrilla *testing* son cafeterías céntricas o centros comerciales donde encontraremos gente con facilidad. Lo primero es pedir permiso porque no a todas las personas les interesa colaborar, además tendrán que firmar los permisos en caso de grabarlos. Con una persona suele ser suficiente, pero es recomendable ir dos, una para explicar el proceso y otra para filmar o apuntar las anotaciones. No prepares algo complicado, las pruebas sencillas o de rápida respuesta es lo mejor para este tipo de *test*.



Pasos 3 y 4 para hacer pruebas de usabilidad

Continuamos recorriendo las **fases o pasos para hacer las pruebas**:

3- Definir las métricas de usabilidad:

Las métricas otorgan objetividad a la hora de debatir los cambios y decisiones en el diseño. De nada sirve medir unos resultados si no tienes métricas fijadas donde poder analizarlos. Está muy bien sacar conclusiones y opiniones subjetivas, pero siempre basadas en unos datos fiables, por ejemplo, nos puede gustar el gris para nuestro botón de compra, pero si nadie hace clic en él tenemos que encontrar la solución.

Tal vez se pueden cuantificar las veces que el usuario hace clic en el botón dependiendo del color o el recorrido que siguió según su tonalidad. Descubrir el porqué de estas decisiones sólo es posible con unos datos contra los que podemos basar nuestra opinión.

Pero, ¿qué es lo que podemos medir?

Dependiendo de la prueba que elijamos necesitaremos ciertos tipos de métricas, pero estas son las principales que se usan en los *test* de usabilidad.

- ❖ **Éxitos en tarea:** básicamente miden si se logra o no completar la tarea. Cuantifica de manera binaria y porcentual cuantas de las tareas fueron exitosas. Las que resultan erróneas son las que queremos enfocarnos para implementar una solución. Se puede ir realizando una planilla colocando el éxito o fracaso en la tarea para, posteriormente, graficar los resultados.
- ❖ **Tiempo en tarea:** cuánto tiempo se tarda en completar la tarea, esta métrica mide la eficiencia y la productividad. Ayuda a identificar si el tiempo en completar una acción está por encima de lo deseado, y qué ajustes se pueden hacer. Al igual que el anterior, se van guardando los tiempos en una planilla tipo y se grafican los resultados.
- ❖ **Errores:** se mide la cantidad de errores que el usuario encuentra en la prueba, aunque la tarea resulte exitosa, el usuario puede encontrar errores que reflejen fallos técnicos de la app o incluso de la misma prueba, conocerlos es esencial para que no se repitan. Recoger la cantidad de errores encontrados en cada tarea es una buena solución para identificar los principales problemas.
- ❖ **Satisfacción subjetiva:** nos ayuda a conocer la opinión de los participantes al completar la prueba, la opinión se puede filtrar a través de diferentes métodos. Para poder cuantificar, se usan niveles de satisfacción numéricos o preguntas concretas sobre su experiencia en la tarea. En el caso de elegir los niveles de satisfacción, realizar una media de los resultados es suficiente para observarlos y sacar conclusiones.

4- Documentos necesarios para la prueba de usabilidad:

Algunos puntos parecen demasiado básicos, pero es importante quedar definida cada parte del proyecto. Esto nos servirá para crear perfil de la prueba que utilizaremos a la hora de reportar los resultados.

Vamos a revisarlo detalladamente:

- **Nombre del producto:** la denominación de tu producto.
- **Objetivos de la prueba:** los objetivos que fijaste en el primer paso estarán presentes en este apartado. A estas alturas deben ser claros y que no sean superiores a 3.
- **Métricas a utilizar:** hay que especificar todas las métricas que se van a utilizar y cuáles son las más importantes.
- **Metodología:** elegimos que tipo de prueba de usabilidad se va a utilizar, ya sea moderada, no

moderada o guerrilla *testing*.

- **Perfil de los participantes:** tenemos que escribir una pequeña descripción de lo que buscamos en un usuario que vaya a participar en la prueba.

- **Texto de bienvenida:** unas pocas líneas que funcionen como presentación del proyecto y de la propia prueba. No tiene que ser demasiado extenso, aunque eso dependerá de la complejidad del *test*.

- **Introducción a los participantes:** a diferencia del texto de bienvenida, ahora se especifican los pasos a seguir para iniciar la prueba, tampoco olvides indicar qué va a ser grabado o que sus datos quedarán registrados en tu base.



Pasos 5 para hacer pruebas de usabilidad

A continuación podrás realizar un recorrido por la **fase cinco o paso cinco**:

5- Tareas de las pruebas de usabilidad:

Después de los objetivos, las tareas son el paso más importante en la prueba de usabilidad, son una extensión de los objetivos, ya que la realización de las tareas nos ayudará a cumplir los objetivos. Hay diferentes tipos de tareas y hay que elegir las que más nos convengan en nuestro proyecto:

Hacé clic en el ícono expandir pantalla  del visor y podrás ampliar la imagen

Tipos de tareas:

❖ **Tareas cerradas:** Son aquellas que tienen bien definida si la tarea será un éxito o fracaso. Los únicos resultados posibles son completar la tarea o no completarla.

Ejemplo | En una aplicación indicar al participante si es capaz de llegar a la sección de nuevos productos.

❖ **Tareas abiertas:**

Se caracterizan por la libertad de opciones a la hora de resolver una tarea, el resultado también será de éxito o fracaso, pero las diferentes formas de cómo se consigue el objetivo es lo importante.

Ejemplo | Indicar al usuario que encuentre X producto. Podrá buscarlo directamente en el buscador de la aplicación o por el contrario navegar por las secciones hasta dar con él.

❖ **Tareas exploratorias:**

No se busca un resultado conclusivo, simplemente recopilar información. Son parecidas a las abiertas, pero en este caso el resultado es más subjetivo.

Ejemplo | En una app, le pedimos al participante que navegue por el menú principal sin un rumbo fijo. Lo interesante es observar las rutas que este toma y cómo se desenvuelve el usuario en la interfaz.

❖ **Tareas específicas:** Prácticamente iguales a las cerradas, pero aquí se especifica claramente lo que la persona tiene que hacer.

Ejemplo | Completar un proceso de compra según un método de pago y unos datos establecidos.

❖ **Búsqueda del tesoro:**

De las más populares. Consiste en dejar pequeñas pistas al participante, para que este logre cumplir el objetivo.

Ejemplo | Volviendo a la app, se indica una búsqueda sobre un contenido más escondido cómo los productos mejor valorados. Y el participante desde 0 deberá encontrar la sección.

❖ **Búsqueda del tesoro inversa:**

Lo mismo que en el anterior tipo, pero al revés, ahora mostramos directamente la solución al problema, ya sea una foto o el producto en sí. Con la diferencia que el usuario ya sabe lo que tiene que encontrar.

Ejemplo | En un *e-commerce*, se muestra a la persona la imagen de un producto, y este navegará por la app hasta dar con él.

❖ **Tareas autogeneradas:** Son retos que los mismos participantes crean, sin necesidad de nuestra intervención.

Ejemplo | Estamos en una app de deportes, y le preguntamos al participante que espera encontrar. Este nos dirá que le gustaría ver los apartados de artículos de opinión o la sección de fútbol, esta es una tarea que no fue necesario haberla creado previamente.

¿Cómo elaboramos las tareas de la prueba de usabilidad?

Pasos esenciales:

a) Definir tarea	b) Establecer la prioridad	c) El escenario
Pensá en el producto que se va a probar, y pensá qué es lo que hace tu producto ya que las tareas van relacionadas con la funcionalidad del software, si es una app de diseño que editen una foto.	Al igual que los objetivos, no hay que realizar un gran número de tareas. Un exceso de información puede cambiar el sentido de los resultados. Lo ideal son 3-5 tareas, que no superen un tiempo de 15 minutos.	Explica la situación donde el usuario realizaría la tarea en un caso real, tomando el ejemplo de la app de deporte, una situación posible sería: “Estás esperando a que llegue el tren para llegar a casa y ver el partido de tu equipo. Pero ya ha empezado y querés saber lo que está pasando, así que entrás en tu app de confianza para comprobar el resultado.”



Pasos 6 y 7 para hacer pruebas de usabilidad

A continuación podrás realizar un recorrido por las **fases o pasos para hacer las pruebas**:

6- Participantes

Los usuarios determinan directamente los resultados de la prueba. Encontrar a la persona que cumpla las características de nuestro público objetivo es sumamente importante.

Lo primero es:

❖ Definir el perfil del participante:

Elegir a un participante significa identificar el comportamiento, habilidades y conocimiento de las personas que utilizan el producto. Al final, los resultados del *test* son válidos si los participantes cumplen las características de este perfil.

Probar el producto con personas que no cumplan este perfil, desembocará en unos resultados erróneos y que no reflejan los datos que queremos analizar. Al igual que hacíamos con el escenario de la tarea, un buen ejercicio es plantear la situación ideal donde se encontraría nuestro participante.

Ejemplo | “Deseamos encontrar personas que reserven entradas para el partido de futbol en nuestra app. El usuario de nuestra app es una persona que concurre a la cancha. A él o ella le encanta disfrutar del partido en directo. También les encanta realizar actividades al aire libre. A nuestro usuario no le importa viajar a otras ciudades para visitar a su equipo favorito”

❖ Elaborar un cuestionario:

Cuando tenemos claro el perfil de nuestro usuario, elaboramos un cuestionario con preguntas específicas, relacionadas con la persona que necesitamos para completar el objetivo de la prueba.

Ejemplo | Siguiendo con el anterior ejemplo pueden ser las siguientes.

- Edad
- Ocupación
- Tendencia de uso de internet
- Qué dispositivo usa para navegar en internet
- Tendencia de ir a la cancha
- Tendencia de viajar a otras ciudades
- Número de participantes

Para pruebas de usabilidad tradicionales, se suele recomendar un máximo de 5 participantes. Según los datos, este número de participantes nos ayuda a conseguir el mejor retorno de la inversión (ROI).

Aunque parezca un número reducido de usuarios, con este número se descubren en torno al 85% de los problemas de usabilidad.

7- Análisis de resultados:

Llegamos al último paso, es hora de estudiar los resultados que hemos obtenido, el análisis de los datos requiere de tablas y documentos que se fueron elaborando a lo largo de la prueba.

Con esta documentación se realiza el informe para presentar los resultados de la prueba al equipo de desarrollo o a quien solicitó el *test*.

Los posibles ítems a colocar en el informe son, entre otros:

- Introducción y perfil de la prueba: los objetivos, métricas, tareas, y todo el trabajo que hicimos anteriormente, se incluye en este punto.
- Resultados de la prueba: lo mejor es optar por capturas de los informes de Excel que reflejen claramente los resultados.
- Recomendaciones y conclusiones: tras estudiar los datos, elabora las diferentes soluciones o mejoras que se pueden aplicar al producto.



Conclusión

Las **pruebas de usabilidad** constituyen un concepto muy amplio que se amoldan a la necesidad del producto. Puede significar una pequeña corrección que optimice el *software* o un gran cambio que modifique totalmente el producto.

E3. Herramientas de IDE Android Studio

Sitio:

[Agencia de Habilidades para el Futuro](#)

Curso:

Desarrollo de Aplicaciones para Dispositivos 2° A

Libro:

E3. Herramientas de IDE Android Studio

Imprimido por:

Johana Micaela Navarro

Día:

miércoles, 29 de abril de 2026, 15:00

Descripción

Tabla de contenidos

1. Introducción al libro

2. TEMA I. IDE de un desarrollo móvil

2.1. ¿Qué es un IDE?

3. Android Studio

3.1. Instalación

3.2. Empty activity

3.3. Creación del proyecto

4. App modelo

4.1. Nuestro diseño

5. TEMA II. Controles de Android Studio

6. Tips para la práctica formativa

7. Los controles y el IDE

8. TextView

8.1. Código

8.2. Entorno XML diseño de la Activity

9. EditText

10. RadioButton (Botones de selección)

10.1. ¿Cómo manipulamos al RadioButton?

11. CheckBox (Casilla de verificación)

12. Uso de varias pantallas

12.1. Pasos para crear la nueva pantalla

12.2. Código para pasar a la nueva pantalla

13. TEMA III. Vínculo con Base de Datos

14. Android y base de datos

15. Estructura de la base de datos

16. SQLite

16.1. Constructor y métodos de la clase SQLiteOpenHelper

16.2. Métodos de la clase SQLiteDatabase

17. Ejemplos con base de datos

17.1. Diseño del ejemplo

17.2. Código de creación de los datos

17.3. Override del onCreate y del onUpgrade

17.4. Código para alta del ABM

17.5. Código para mostrar los datos

17.6. Código para modificación y baja del ABM

18. TEMA IV. Preparar el entorno del proyecto en Android Studio

19. Introducción al entorno del proyecto

20. Cómo visualizar la base de datos en el IDE

21. Creación de la base con varias tablas

22. Comenzando con la app del proyecto "Club Deportivo"

23. Primeros tips para el desarrollo de la app

23.1. Cambio de activity de inicio

23.2. Nuevo IDE nuevos usuarios



Introducción

En este libro se presentan cuatro temas relacionados, que permiten conocer a fondo una herramienta fundamental en el desarrollo de aplicaciones para dispositivos móviles: **Android Studio**

Los cuatro temas que encontrarás son:

- I IDE de un desarrollo móvil
- II Controles de Android Studio
- III Vínculo con base de datos
- IV Preparar el entorno del proyecto con Android Studio

**Te recomendamos leer con atención el contenido y, al mismo tiempo, hacer pruebas sobre la herramienta.
Recordá que se trata de un instrumento ¡necesita práctica!**

¡Comenzamos!



IDE de un desarrollo móvil

¿Alguna vez te preguntaste cómo los desarrolladores solían construir aplicaciones antes de la llegada de los Entornos de Desarrollo Integrados (IDEs)?

La realidad era tediosa: cambiar entre editores de texto, compiladores y herramientas de verificación de errores consumía tiempo y esfuerzo. Pero, ¿qué revolucionó esta dinámica? **¡Los IDEs!**

Definición | Un **IDE**, más que un simple editor, es un conjunto de herramientas que simplifica el desarrollo de *software*. Por ejemplo, Android Studio, el IDE oficial para aplicaciones en Android, ofrece un entorno que reúne todo lo esencial para el desarrollo de la aplicación.

Siguiendo pistas, consejos, y analizando una app modelo, aprenderás a gestionar eventos, crear funciones y tomar decisiones en tu código, llevando tu proyecto a un nuevo nivel.

Preguntas clave de este tema:

- ¿Qué diferencia crucial existía en el proceso de desarrollo antes y después de la introducción de los entornos de desarrollo integrados (IDEs)?
- ¿Cuáles son las funcionalidades básicas que componen un IDE?
- ¿Cuál es el propósito y la importancia de Android Studio en el desarrollo de aplicaciones para Android?
- ¿Qué tipos de módulos están presentes en los proyectos de Android Studio, y cómo se organizan estos proyectos?
- ¿Qué elementos se encuentran dentro de un proyecto creado en Android Studio, y qué representan en el contexto del desarrollo de una aplicación?

¿Listo/a para empezar? ¡Adelante, el mundo del desarrollo te espera!



¿Qué es un IDE?

Antes de la llegada de los IDE, los desarrolladores utilizaban simples editores de texto para codificar, guardar la aplicación en un editor de texto, ejecutar el compilador, comprobar si había errores y volver al editor para comprobar el código. Todo este proceso consumía mucho tiempo y esfuerzo del desarrollador, ya que siempre tenía que cambiar entre varias aplicaciones.

Aquí es donde entra en juego un IDE que reúne todas las herramientas esenciales del desarrollador bajo un mismo marco.

Definición | IDE es un programa de software o una amalgama de herramientas que necesita para escribir y probar su software. En resumen, un IDE es una combinación de herramientas básicas necesarias para el desarrollo de aplicaciones.

El IDE consta al menos de

- ◆ un editor de texto,
- ◆ herramientas de automatización de la construcción y,
- ◆ un depurador.

Además, algunos IDE vienen con la ventaja de poder instalar *plugins* para ampliar sus funcionalidades a otro nivel.



Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial que se usa en el desarrollo de apps para Android.

Cada proyecto de Android Studio incluye uno o más módulos con archivos de código fuente y archivos de recursos. Entre los tipos de módulos, se incluyen los siguientes:

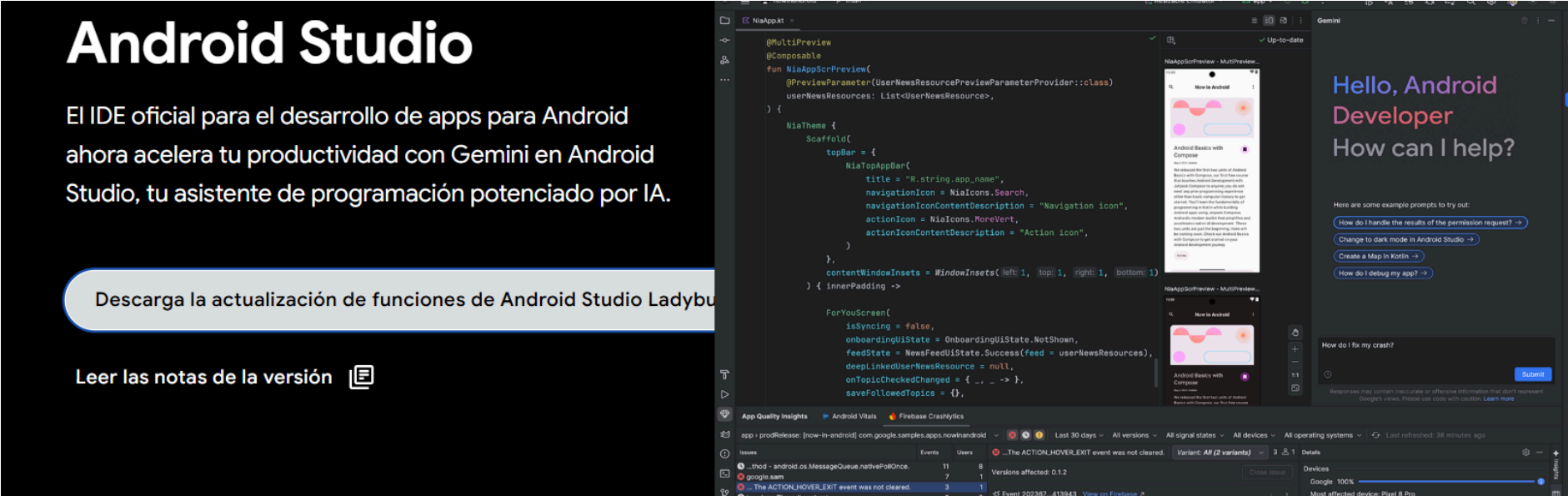
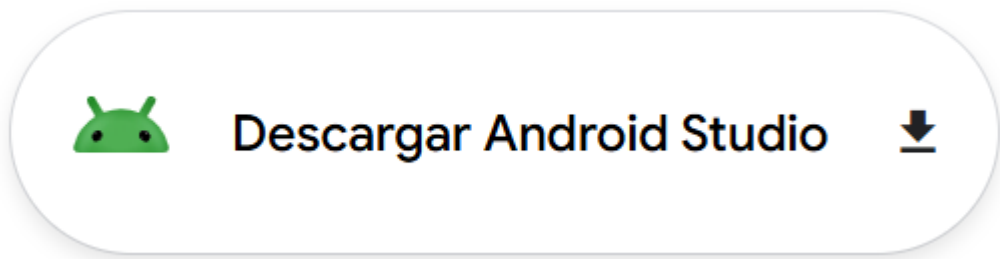
- ❖ Módulos de apps para Android
- ❖ Módulos de biblioteca.
- ❖ Módulos de Google App Engine

Los proyectos de Android Studio

contienen todo lo que define tu espacio de trabajo para una app: desde código fuente y recursos hasta código de prueba y configuraciones de compilación.

Cuando comienzas un proyecto nuevo, Android Studio crea la estructura necesaria para todos los archivos y los hace visibles en la ventana Project de Android Studio.

Vamos a la web oficial: <https://developer.android.com/>



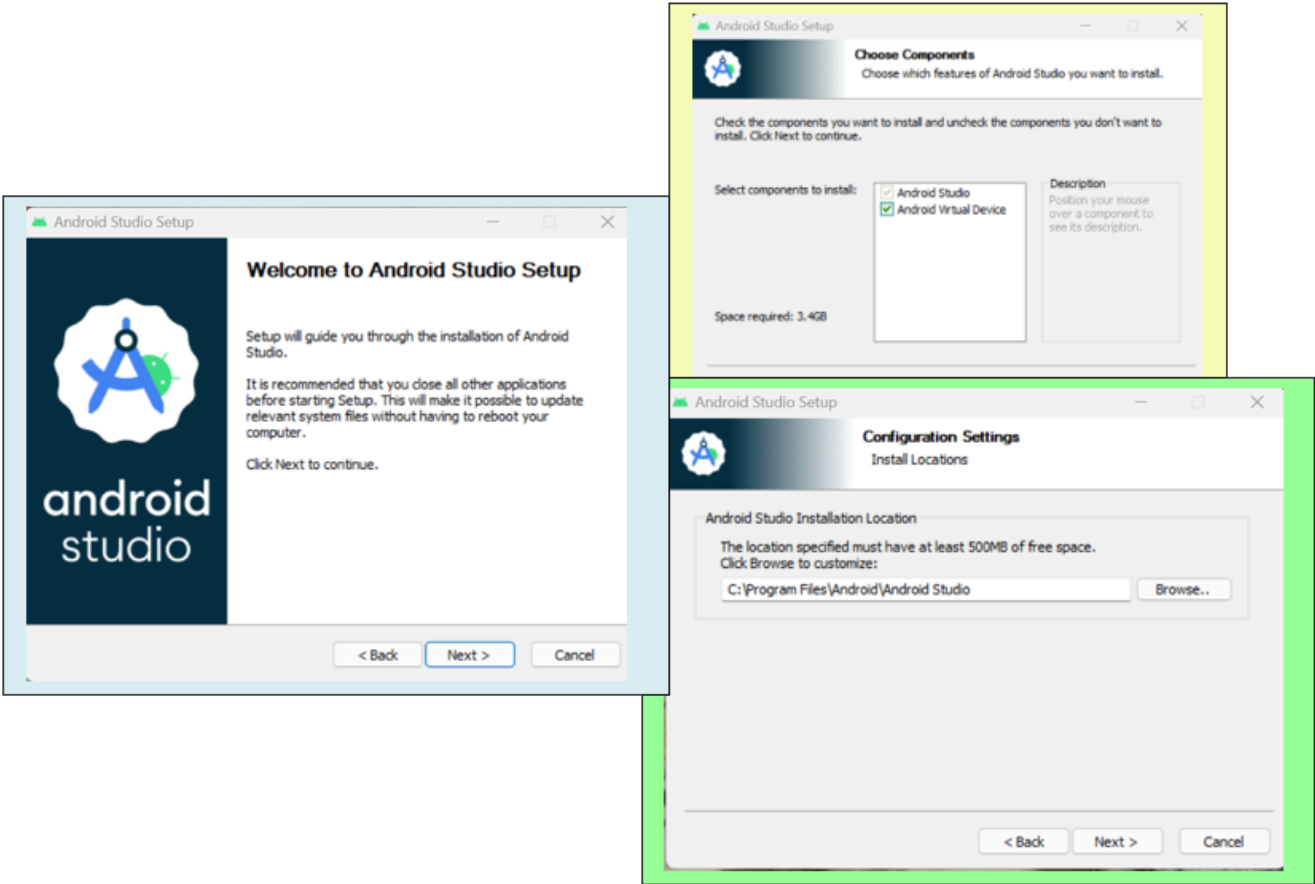
Veamos de que se trata...



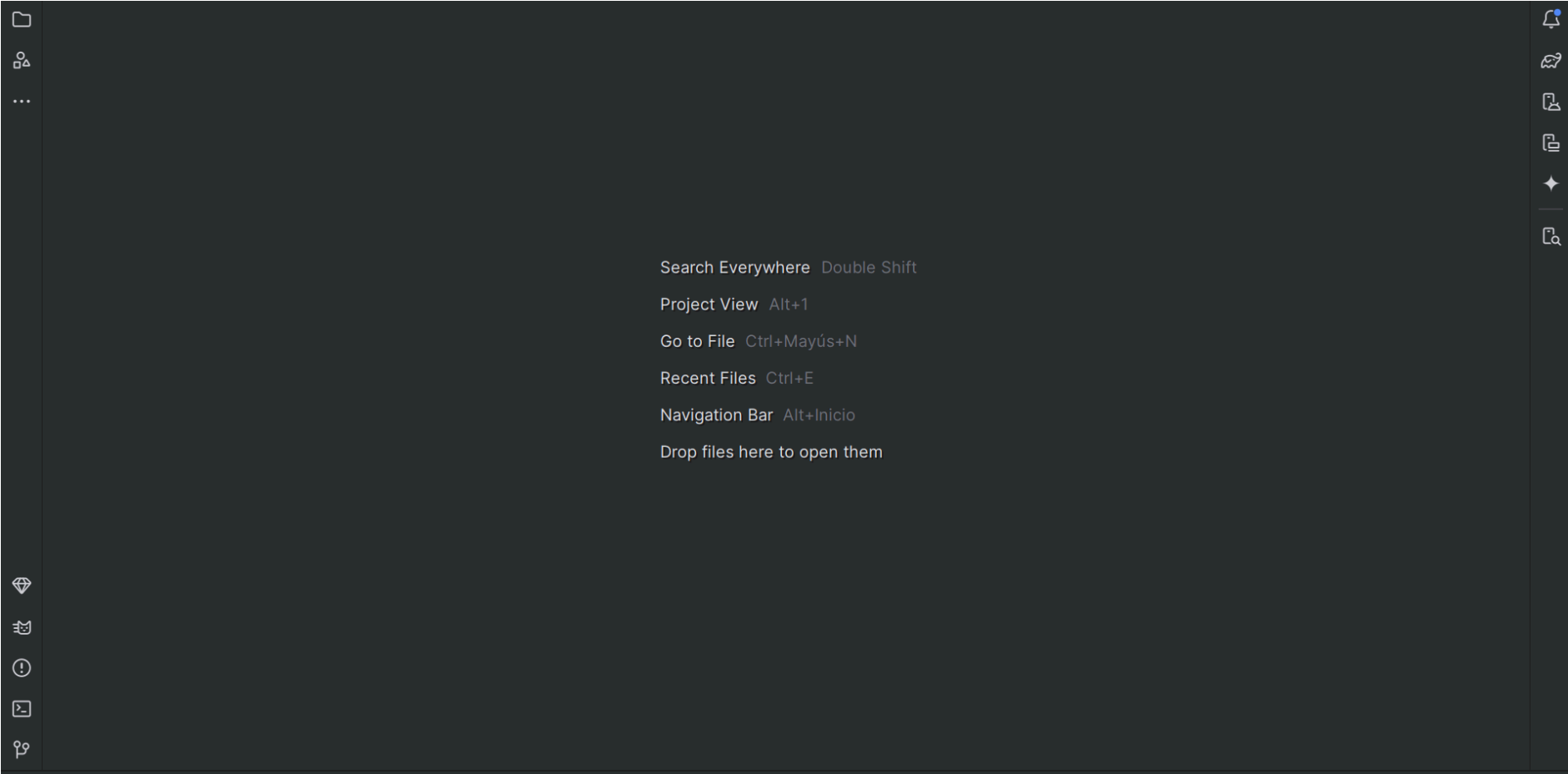
Instalación

Lo primero es **descargar el Android Studio** para proceder a la instalación siguiendo las instrucciones.

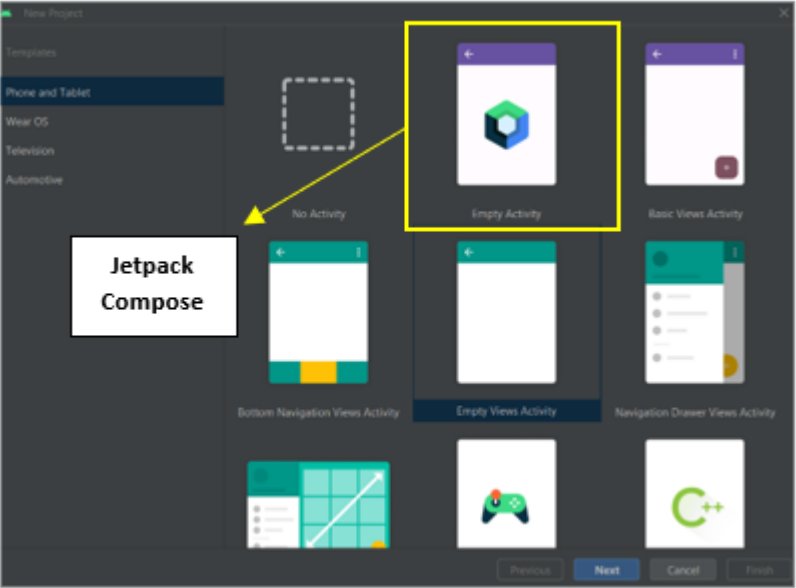
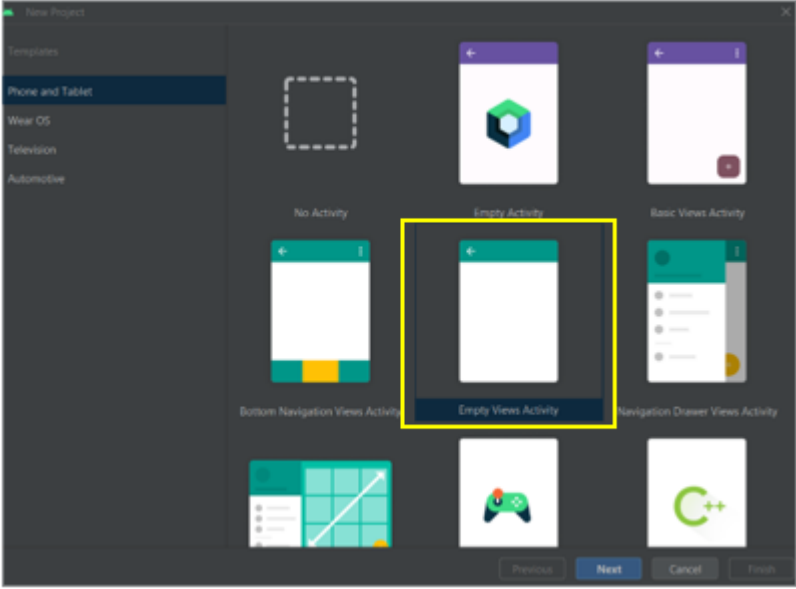
A medida que avances en la instalación verás las siguientes ventanas (pueden cambiar en función de la versión):



Llegarás a:



Al crear un nuevo proyecto nos encontramos con una serie de plantillas:



Te preguntará cuál es la diferencia, Veamos...



Empty Activity vs Empty Views Activity

- **Empty Views Activity:** Crea una actividad vacía con la estructura mínima necesaria para empezar a desarrollar una aplicación. Se usa XML para definir la interfaz y código en Kotlin para manejar la lógica.
- **Empty Activity:** Es similar a la *Empty Activity*, pero ya viene configurada para trabajar con una forma más moderna de interactuar con los elementos de la interfaz, facilitando la manipulación de las vistas dentro del código, trabajando con Jetpack Compose. No se trabaja con XML, se integra vista y lógica dentro de Kotlin.

Diferencia entre Jetpack Compose y XML

Para **diseñar interfaces en Android**, hay dos enfoques principales:

XML (Views tradicionales)

- Usa archivos XML para definir la interfaz de usuario.
- Se organiza con componentes como **TextView**, **Button**, **LinearLayout**, etc.
- Es el método más utilizado en proyectos existentes y ha sido el estándar en Android por mucho tiempo.

Jetpack Compose

- Permite construir interfaces de usuario con código Kotlin puro, sin necesidad de archivos XML.
- Se basa en un enfoque declarativo, más moderno y flexible.
- Requiere aprender nuevas formas de estructurar la UI, diferentes a las vistas tradicionales, algo más complejas.

¿Por qué trabajaremos con XML?

- Es el enfoque más utilizado actualmente en aplicaciones en producción.
- Permite comprender mejor la estructura de la interfaz y la relación entre diseño y lógica.
- Facilita la transición a otros conceptos como Fragments y Activity.

Más adelante, si el tiempo lo permite, exploraremos *Jetpack Compose*.

¿Qué es una *activity*?

Activity o actividad es lo más básico y utilizado en el desarrollo de Android, se considera que una pantalla es un *activity* por lo tanto si nuestra aplicación tiene 3 pantallas podemos decir que tiene 3 *activity*. Cada *activity* está formada por dos partes, una parte lógica y una parte gráfica.

Si seleccionamos empty views activity la parte lógica es un archivo .java o un archivo .kt (si el lenguaje es *kotlin*) que nos muestra la clase que se crea para poder manipular, interactuar y colocar el código de esa actividad. La parte gráfica es un XML que tiene todos los elementos que estamos viendo de una pantalla declarados con etiquetas parecidas a las de HTML.

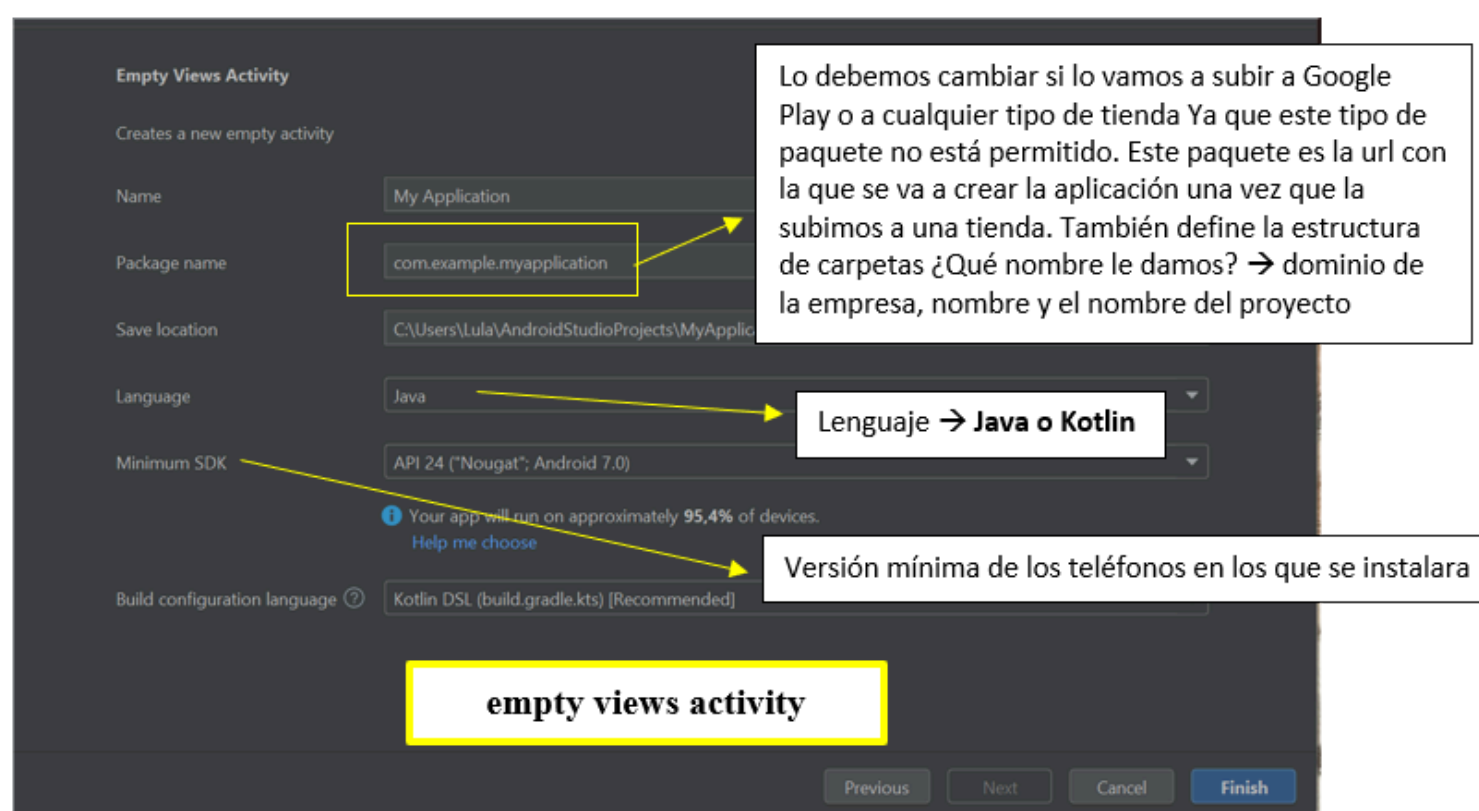
Si seleccionamos empty activity la parte lógica y la parte gráfica están unificadas en un mismo archivo .kt

Una vez elegida la *activity* avanzamos con la **creación del proyecto** .



Creación del proyecto

Vemos la siguiente pantalla para asignarle el nombre y la ubicación:



Una vez que creamos el nuevo proyecto, llegamos al entorno de desarrollo de Android Studio.

Si es la primera vez que se trabaja con Android Studio y ejecutamos, se produce un error porque no tenemos seleccionado ningún dispositivo que sea capaz de ejecutar nuestra app.

Tendremos dos **opciones para proceder a la ejecución**:

- **Ejecutarlo en un emulador**, es decir, una utilidad que simule un dispositivo Android.
- **Ejecutarlo en un dispositivo físico**.

Para ver las partes del IDE vamos a mostrar en una app un botón con la leyenda “**presione aquí**” y, al hacerlo, que muestre un mensaje.

Manos a la obra...



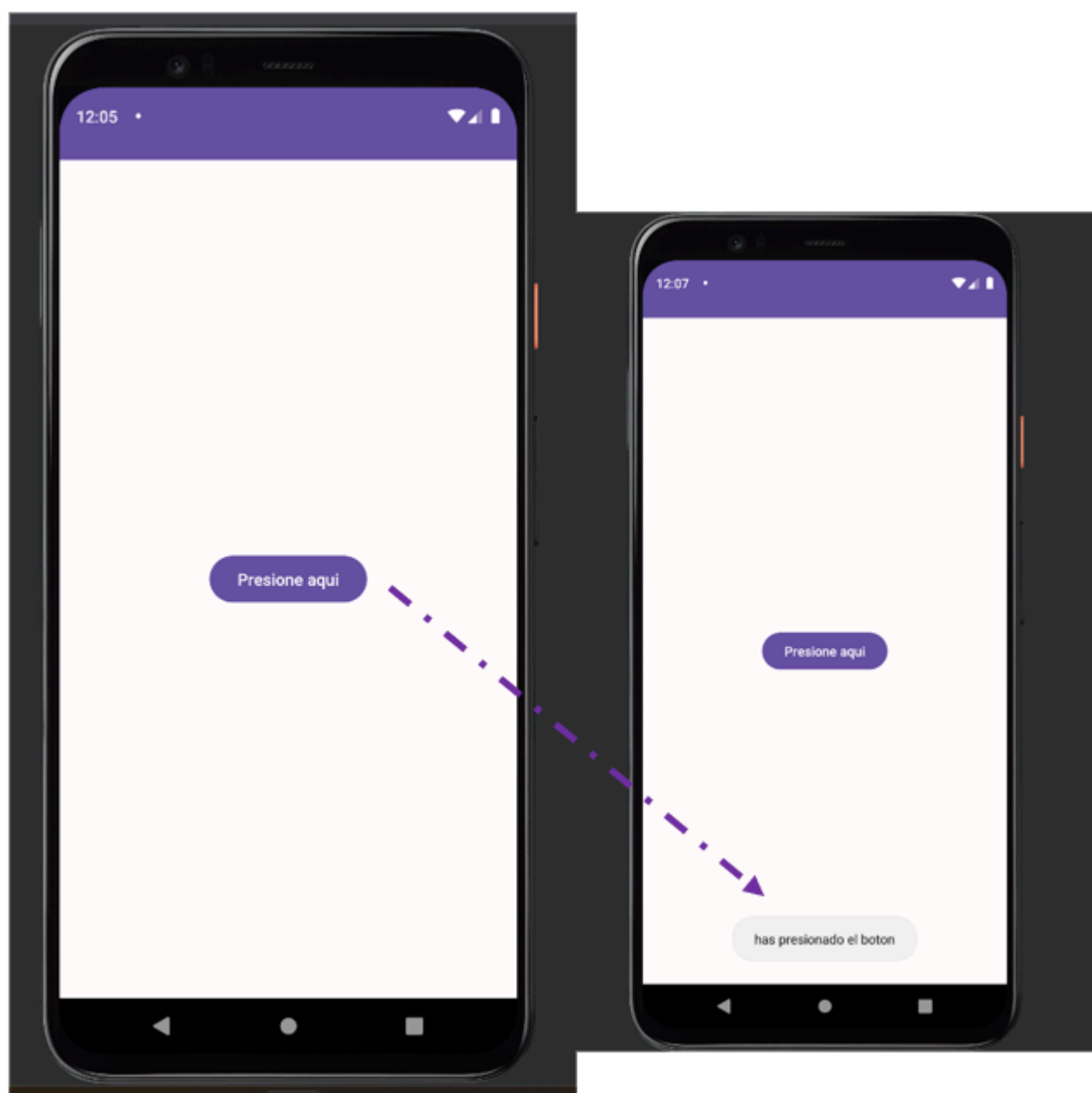
App modelo

Comencemos con el modelo de una App:

1. Creamos una **empty views activity**
2. Almacenamos el proyecto en el directorio bajo un nombre y seleccionamos como lenguaje a **Kotlin**
3. El IDE nos muestra en el extremo de la izquierda un árbol con la carpeta **app** y la carpeta **Gradle Scripts**

App es nuestra aplicación

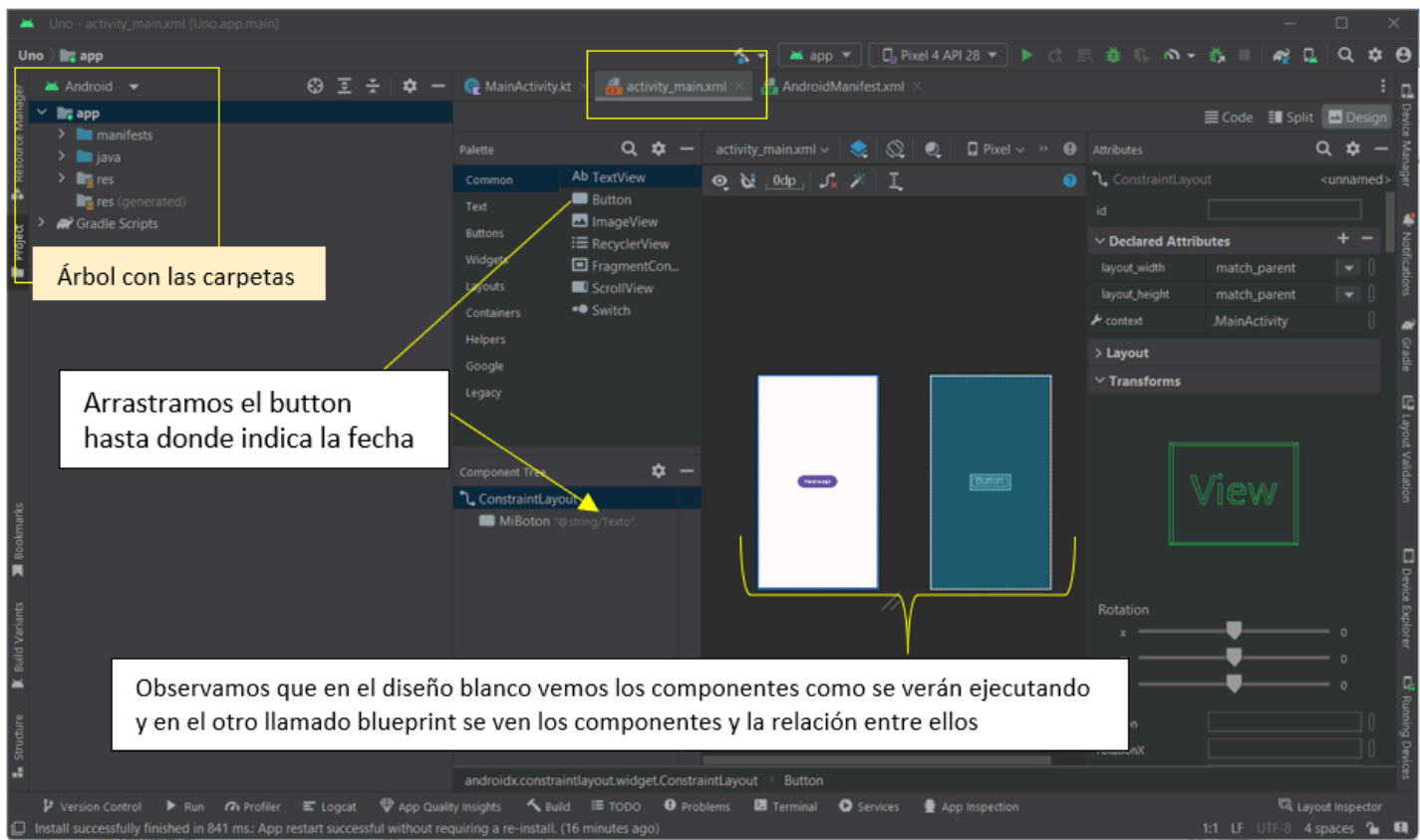
- En Manifests encontramos un archivo llamado AndroidManifest.xml que es el archivo de configuración principal donde se configuran las pantallas. Es una declaración de lo que tiene la aplicación y sus permisos. Acceso a internet. Escritura en el teléfono. Tiene estipulado cuantas pantallas tiene la aplicación, cual es el nombre, el icono, etc.
- Otro archivo es “MainActivity” que contiene la programación de nuestra app.
- En *res* se encuentra *layout* que contiene la parte gráfica.



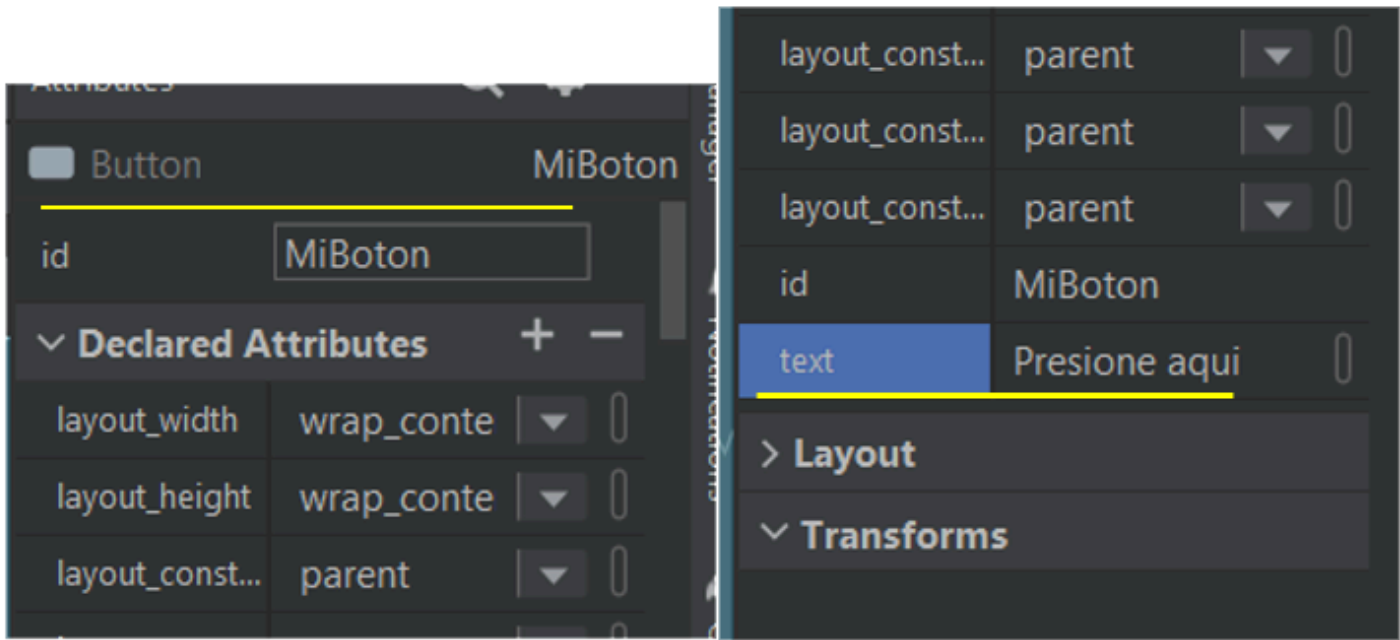
Construyamos el diseño...



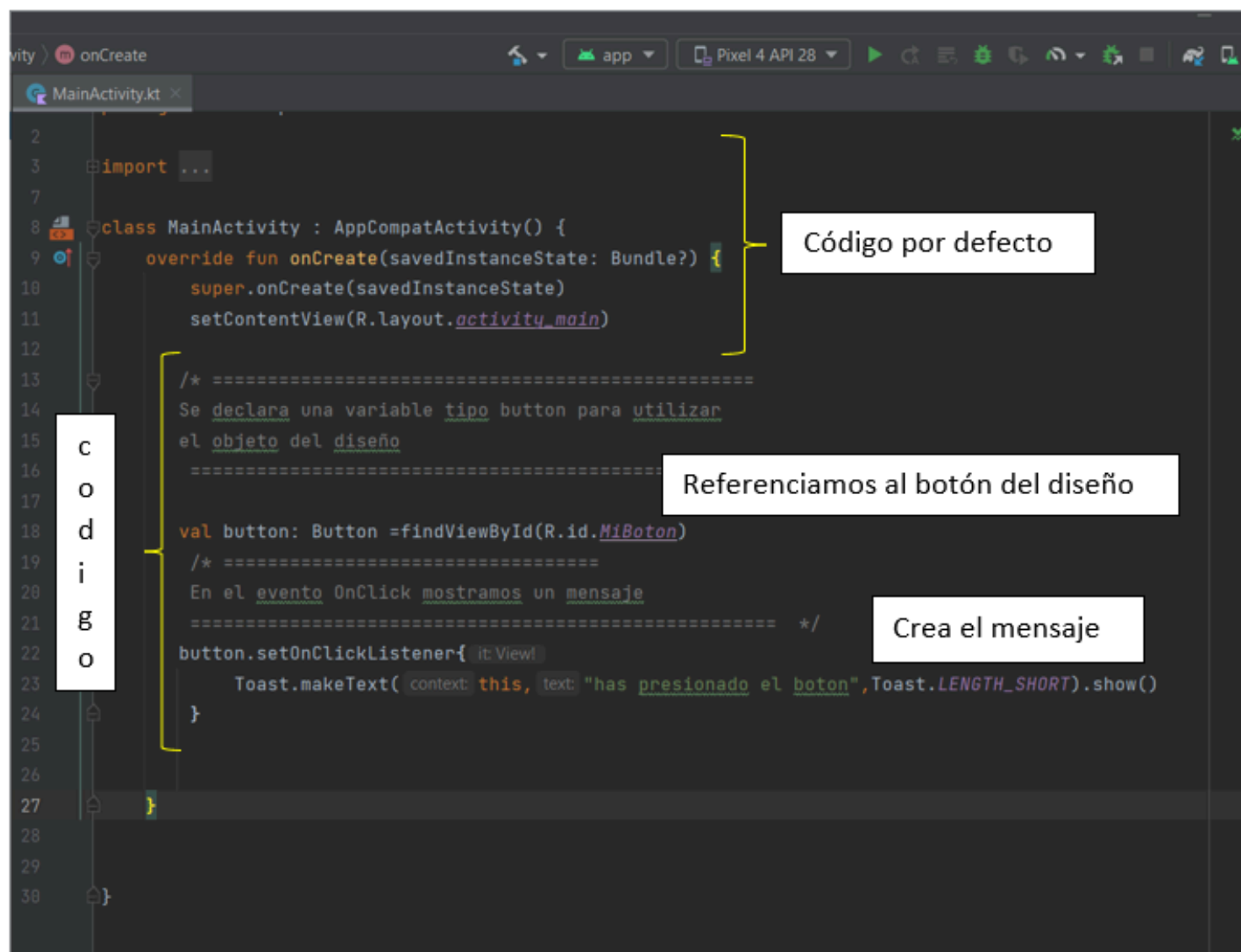
Nuestro diseño



En la barra de los Atributos (ubicado en el lateral derecho) le damos nombre al componente y el texto que contiene. El nombre es el atributo **id** y el texto en el atributo **text**.



Construyamos el código en **MainActivity.kt**.



Accedé al código haciendo clic [aquí](#).



Controles de Android Studio

¡Comenzamos un nuevo paso de nuestro recorrido! En este tema nos sumergiremos en el universo de los **controles para nuestras aplicaciones**. Olvidemos las definiciones aburridas y demos paso a ejemplos cotidianos y prácticos.

¿Alguna vez quisiste crear esos botones "Aceptar" o "Cancelar"? Pues bien, vamos a hacerlo. Y eso no es todo, hablaremos de **TextView, EditText, RadioButton y CheckBox**. Imagina diseñar una app de compras donde podemos elegir nuestra talla de remera, la cantidad y hasta el método de pago.

Además, **aprenderemos a cambiar de una pantalla a otra en una app**. Es como pasar de un enlace a otro en internet. ¡Emocionante, verdad? Así que preparate para una semana práctica y divertida.

Preguntas clave del tema:

- ¿Cómo personalizaríamos un TextView en una aplicación para mostrar un mensaje específico?
- ¿Cuáles son las diferencias clave entre un RadioButton y un CheckBox en Android?
- Expliquen cómo crear un nuevo proyecto en Android Studio y agregar una segunda pantalla (*activity*).
- ¿Qué es un Intent en el contexto de desarrollo de aplicaciones móviles y cuándo lo usaríamos?
- ¿Cómo manipularíamos los atributos de un EditText para que acepte solo un tipo específico de entrada, como un número de teléfono o una contraseña?

¿Listo/a para empezar?



Tips para la práctica

Definición

Un **tip** es un dato o una pista para resolver un problema. Teniendo en cuenta esto, te mostraremos algunas pistas para aplicar en la actividad propuesta.

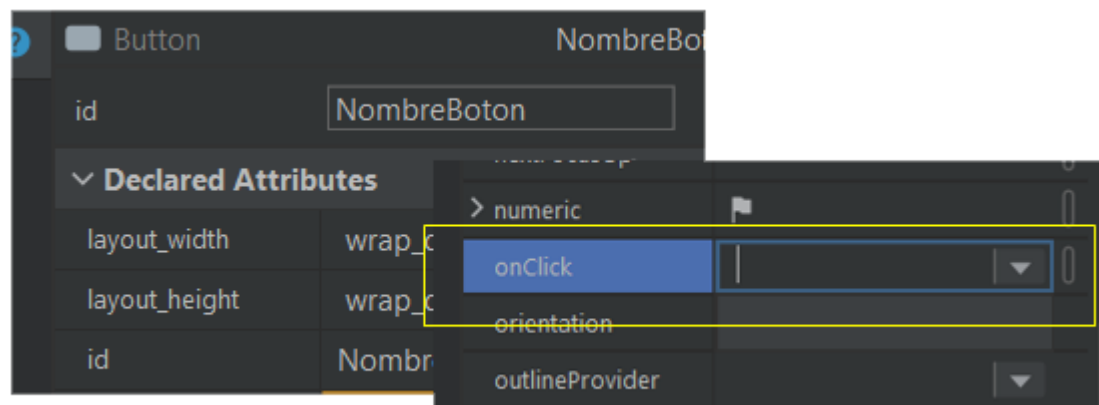
Cuando insertamos en el diseño componentes, estos esperan que desde el exterior se accione algún evento. ¿Te acordás de desarrollo de sistemas orientados a objetos? En los formularios colocamos botones, cajas de texto, opciones, cajas de validación entre otros objetos y programamos en los eventos correspondientes para procesar los datos recibidos.

Veamos entonces:

- Si la App tiene botones el evento por defecto es el **click**, podemos tratarlo desde:
 - Código**

```
NombreBoton.setOnClickListener{
}
```

- Crear una función y agregarla al **OnClick** de los atributos



- ¿Cómo creamos una función?: en este caso el parámetro de la función es **view: View**. ¿Qué es un View? Es un componente que permite controlar la interacción del usuario con la aplicación.

```
fun NombreFuncion(view: View){
}
```

- Podemos usar lo que conocemos como múltiples opciones en la toma de decisiones, es decir un **Case**, en Kotlin se llama **when() { }**. En los paréntesis se coloca la variable o atributo del componente que puede tomar múltiples valores y que en función del dominio es el código que debe ejecutar.
- Es conveniente el uso de las banderas (término utilizado mucho en programación) es decir valores en una variable que tengan un significado propio.
- En este caso para identificar las diferentes operaciones, por ejemplo, 1 para suma, 2 para resta, 3 para multiplicación, 4 para división.



Los controles y el IDE

A esta altura de la cursada ya estás familiarizado/a con los términos **clases**, **instancia** y **objetos**.

Muchos programadores cuando instancian una clase y obtienen un objeto para colocarlo en el diseño de una programación le dan el nombre de **control**. Se comienza a manipular los atributos de esos controles para personalizarlos en función del diseño que queremos lograr.

En nuestro intercambio cotidiano con los dispositivos nos encontramos con un botón “Aceptar” o “Cancelar”, o con mensajes que nos brindan información. Lo interesante es que no debemos preocuparnos por crearlos sino que ya vienen con el IDE.

Nuestra tarea:

se centra en modificar sus propiedades: tamaño, color, etc. para incorporarlos en nuestras aplicaciones y asociarles el código necesario para que se comporten como esperamos al ejecutar la app.

¿Qué esperamos?



TextView

Definición

Es un **componente** que podemos incluir en las vistas de la aplicación en la que estamos trabajando con el propósito de presentarle un texto al usuario.

Se utiliza mucho en las interfaces de usuario, debido a que nos da la posibilidad de presentar títulos, contenido que pretende ser informativo.

¿Cómo lo creamos en el diseño?

Podemos hacerlo desde código o desde la ventana de controles que nos proporciona el entorno XML de diseño de la *activity*.

Analicemos los dos casos el código y la ventana del entorno.



Código

Se utiliza la etiqueta asignada “TextView”. Con ella se le indica al programa lo que queremos incluir y, luego, se especifica los diferentes parámetros para el mismo.

```
<TextView
  1 → android:id="@+id/viewId"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
  3 → android:text="@string/text"
      android:layout_margin="8dp"
      android:textSize="@dimen/text_size"
      android:textColor="@android:color/holo_purple"
  6 { android:background="@android:color/holo_orange_dark"
    />
```

Diagram illustrating the XML code for a TextView widget with numbered annotations:

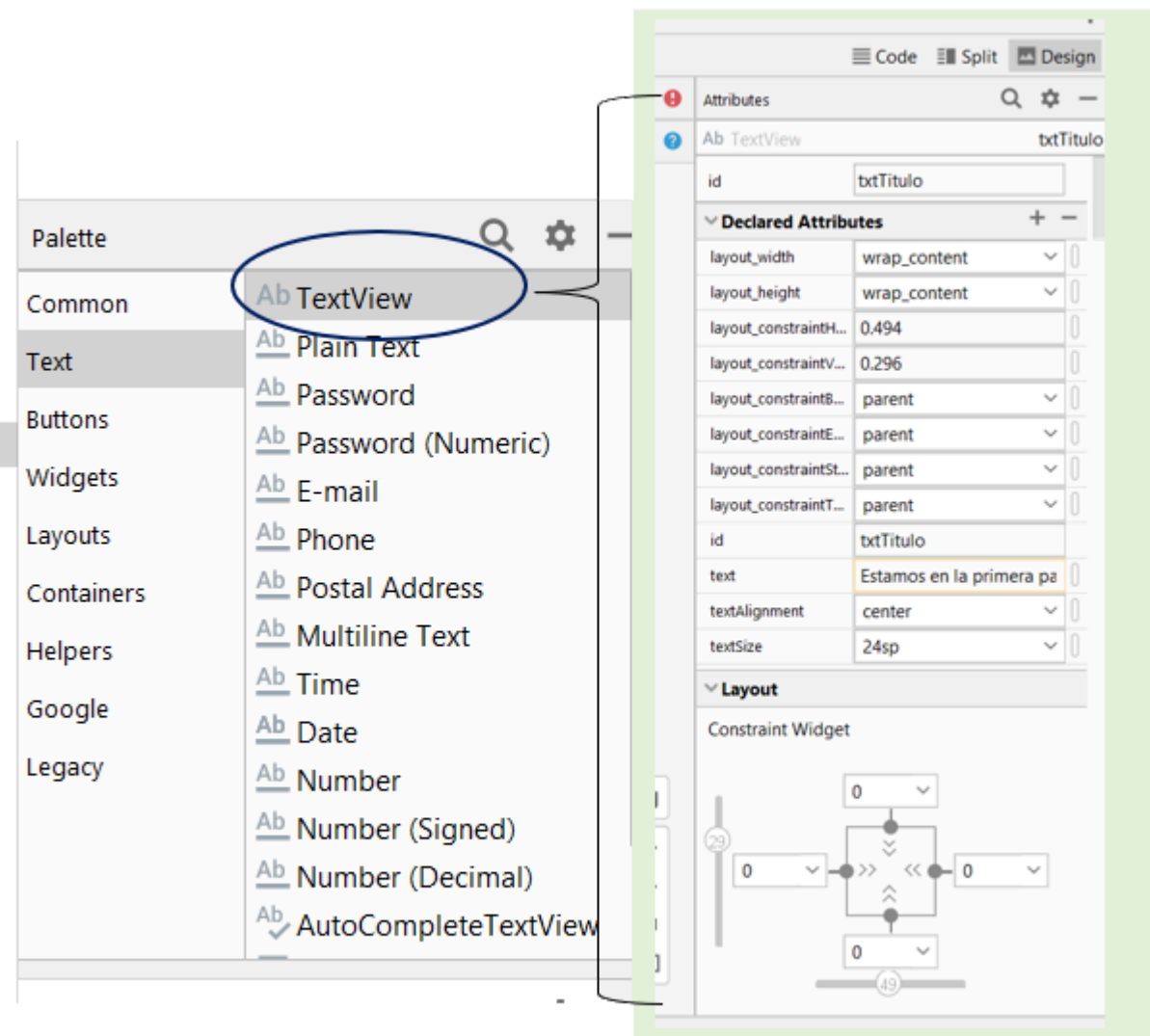
- 1: `android:id="@+id/viewId"` (Identifier)
- 2: `android:layout_width="wrap_content"` and `android:layout_height="wrap_content"` (Layout dimensions)
- 3: `android:text="@string/text"` (Text content)
- 4: `android:layout_margin="8dp"` (Margin)
- 5: `android:textSize="@dimen/text_size"` (Text size)
- 6: `android:textColor="@android:color/holo_purple"` and `android:background="@android:color/holo_orange_dark"` (Colors)

- 1 El identificador no es un *string*, sino que al hacer uso del “@” le estamos indicando al programa que es una referencia a otros componentes. En este caso, estamos referenciando a un “id” llamado *viewId*. A su vez, con el signo +, le estamos diciendo al compilador que, si no llega a encontrar ese nombre (*viewId*) en el listado de **id** que se genera de forma interna, debe crearlo.
- 2 Especifica que el tamaño del componente debe variar según el contenido.
- 3 Debe usar el texto que se encuentra en la referencia string.
- 4 Es el margen o espacio que debe haber entre el TextView y el recuadro de la vista.
- 5 Tamaño del texto.
- 6 Las referencias a los colores que queremos usar.



Entorno XML diseño de la Activity

Como en todo IDE que nos facilita la creación de un diseño lo que debemos hacer es trasladar el control a la plantilla de la *activity* y manipular sus propiedades a través de la ventana de “atributos”, encontramos las que visualizamos en la explicación del punto anterior. ¿Cuáles son?





EditText

Los EditText en Android se consideran un TextView que presenta algunas modificaciones. Puede mostrarle un texto al usuario; no obstante, su función es permitirle o brindarle la opción de introducir texto.

Veamos la etiqueta de creación desde código, recordando que si lo creamos desde el diseñador de XML del *activity* debemos clicar en los atributos que deseamos personalizar.

```
<EditText
    android:id="@+id/txtUsuario"
    android:layout_width="345dp"
    android:layout_height="38dp"
    android:ems="10"
    android:inputType="text"
    app:layout_constraintBottom_toTopOf="@+id/btnUno"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.337"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/txtTitulo"
    app:layout_constraintVertical_bias="0.501" />
```

Para tener en cuenta

- A diferencia del código del textView, los atributos **width y height** tiene valores asignados, la escala es **dp** que es una unidad de píxel virtual que prácticamente equivale a un píxel en una pantalla de densidad media.
- “**ems**” se usa para determinar el tamaño de un carácter según la cantidad de puntos que use la fuente del texto.
- “**inputType = 'text'**” le indica al código que ese EditText va a aceptar todo lo que se considere texto. A su vez, permite que la app le indique al dispositivo del usuario el tipo de caracteres que acepta. **Por ejemplo** si el valor del atributo es *phone* le indicamos que el ingreso es un número de teléfono, pero si es *textPassword* el ingreso es una contraseña y como tal debe estar camuflada.
- El resto de los atributos que nos muestra están relacionados a la disposición del EditText con respecto a los otros controles del diseño.



RadioButton (Botones de selección)

Los **botones de selección** permiten al usuario seleccionar una opción de un conjunto. A continuación veamos algunas de sus características:

Creación de opciones del Botón de Selección:	• Se selecciona un RadioButton en el diseño.
Exclusividad mutua	• Los botones de selección son mutuamente excluyentes. • Se deben agrupar dentro de un RadioGroup para garantizar que solo se pueda seleccionar un botón a la vez.
Eventos de click	• Cuando el usuario selecciona uno de los botones de selección, el objeto RadioButton correspondiente recibe un evento de click.
Controlador de eventos de click	• Se utiliza el atributo <code>android:onClick</code> al elemento <code><RadioButton></code> en el diseño XML. • El valor de este atributo debe ser el nombre del método al que se desea llamar en respuesta a un evento de click, similar a los botones regulares. • La Activity que aloja el diseño debe implementar el método correspondiente.

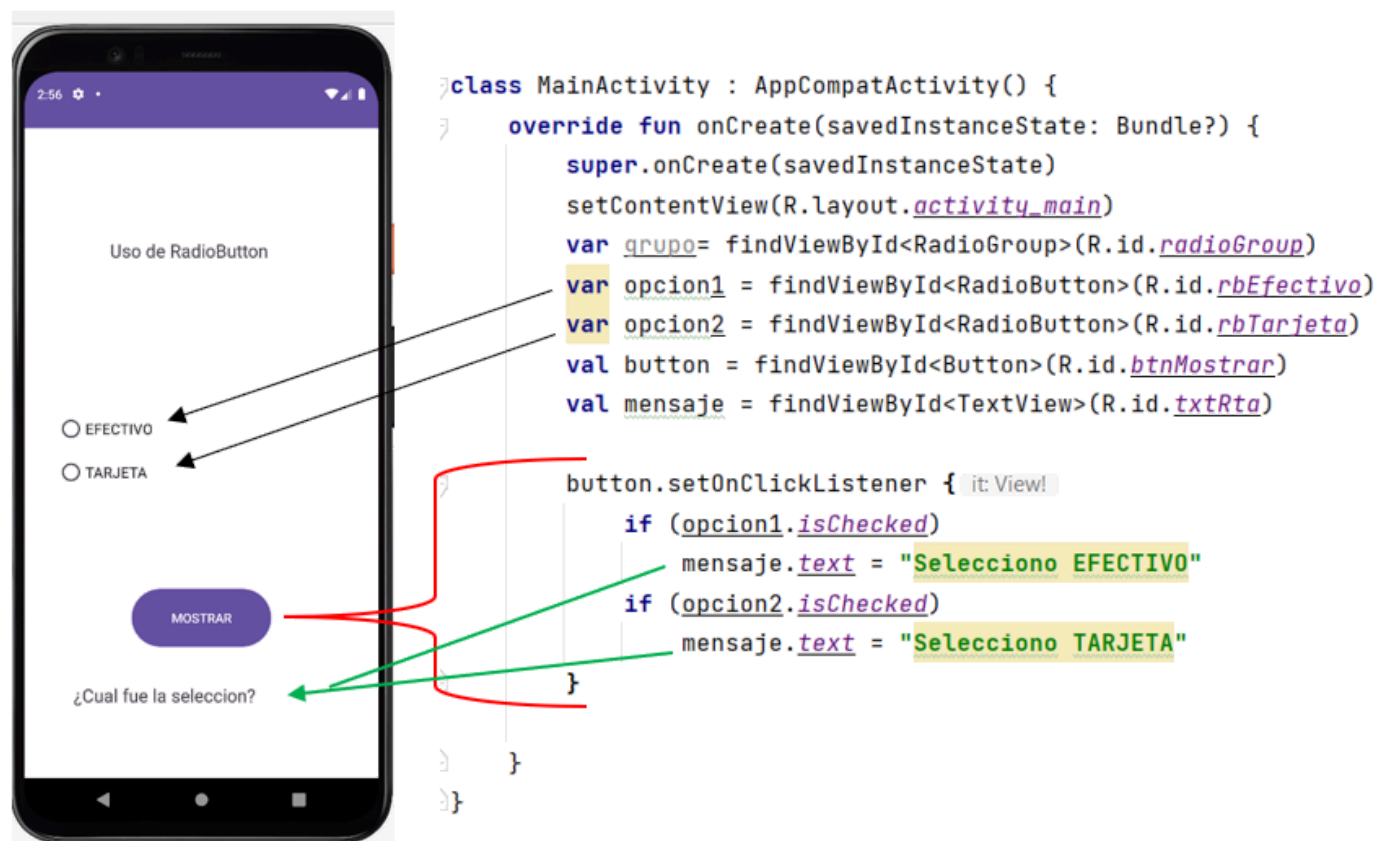
Veamos como los usamos ...



¿Cómo manipulamos al RadioButton?

Lo primero que hacemos es **crear variables asociadas a ellos, personalizándolas** con la asignación del **id** que establecimos. Colocamos en el diseño también un botón para tener control sobre la selección.

Veamos entonces:





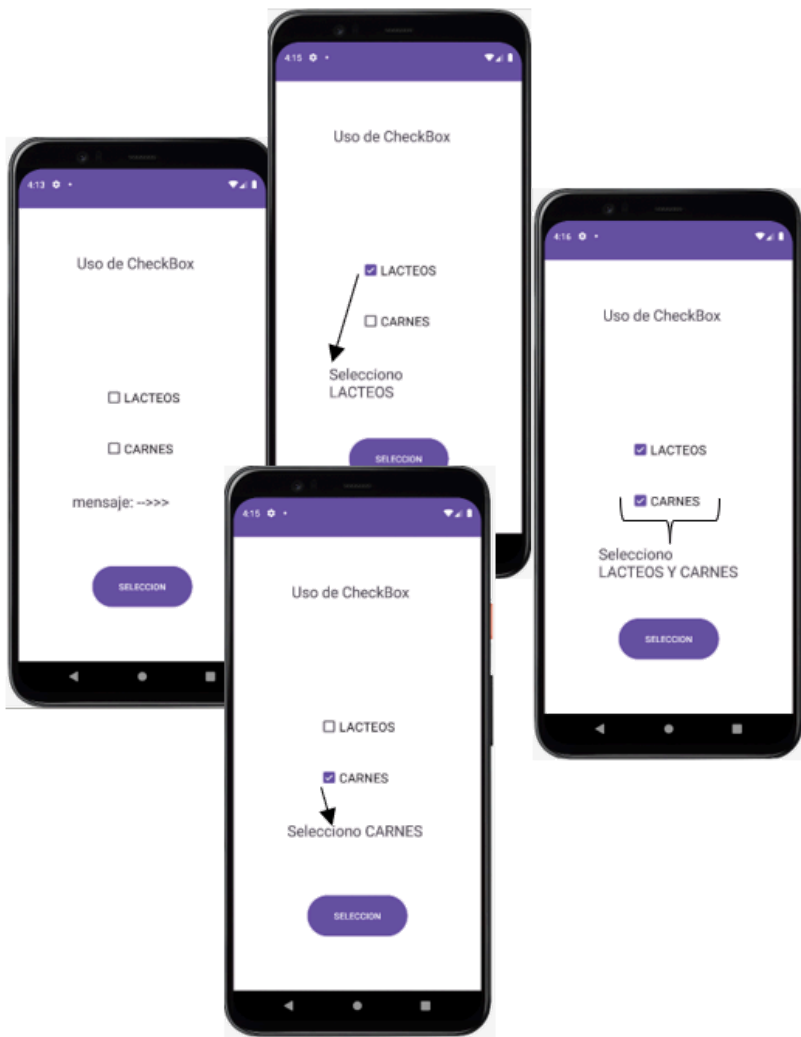
CheckBox (Casilla de verificación)

Las **casillas de verificación** permiten que el usuario seleccione una o más opciones de un conjunto. Por lo general, debés presentar cada opción de casilla de verificación en una lista vertical.

Un Checkbox es un botón de dos estados (marcado, no marcado) que actúa como control de selección.

Para manipular los CheckBox usamos la misma lógica empleada en los RadionButton.

Veamos entonces:



```
class MainActivity2 : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main2)

        val check1 = findViewById<CheckBox>(R.id.checkL)
        val check2 = findViewById<CheckBox>(R.id.checkC)
        val button1 = findViewById<Button>(R.id.btnSele)
        val nota = findViewById<TextView>(R.id.txtNota)

        button1.setOnClickListener { it: View!
            if ((check1.isChecked) && (check2.isChecked))
                nota.text = "Selecciono LACTEOS"

            if ((check2.isChecked) && (check1.isChecked))
                nota.text = "Selecciono CARNES"
            if ((check2.isChecked) && (check1.isChecked))
                nota.text = "Selecciono LACTEOS Y CARNES"
        }
    }
}
```




Uso de varias pantallas

Anteriormente hablamos de las [activity](#) ¿te acordás?

Dijimos que cuando tenemos una pantalla en el desarrollo móvil, podemos decir que tenemos una *activity*, y que generalmente en una aplicación solemos tener más de una pantalla, es decir, más de una *activity*.

Para incorporar varias pantallas debemos hablar de [Intents](#).

Definición Los [Intents](#) son objetos que nos ayudan a iniciar una actividad o proceso a través de la configuración con la cual se haya creado, esto nos permite navegar a nuevas pantallas o acceder a funciones extras que no tiene nuestra app, pero si el resto del sistema Android.

Hay 2 tipos de Intents:

- ❖ **Explícitos:** son los que indican qué se desea lanzar exactamente, es el más habitual y se refiere a otra activity. Este se da cuando tengo una app con una pantalla y quiero lanzar la segunda pantalla.
- ❖ **Implícitos:** son los que se utilizan cuando quiero lanzar tareas abstractas, es decir lanzar un servicio del tipo “Quiero sacar una foto”, en este caso no estoy lanzando otra pantalla sino que aparecerá la aplicación de cámara para el dispositivo móvil siempre que el usuario de permiso o acceso a la cámara.

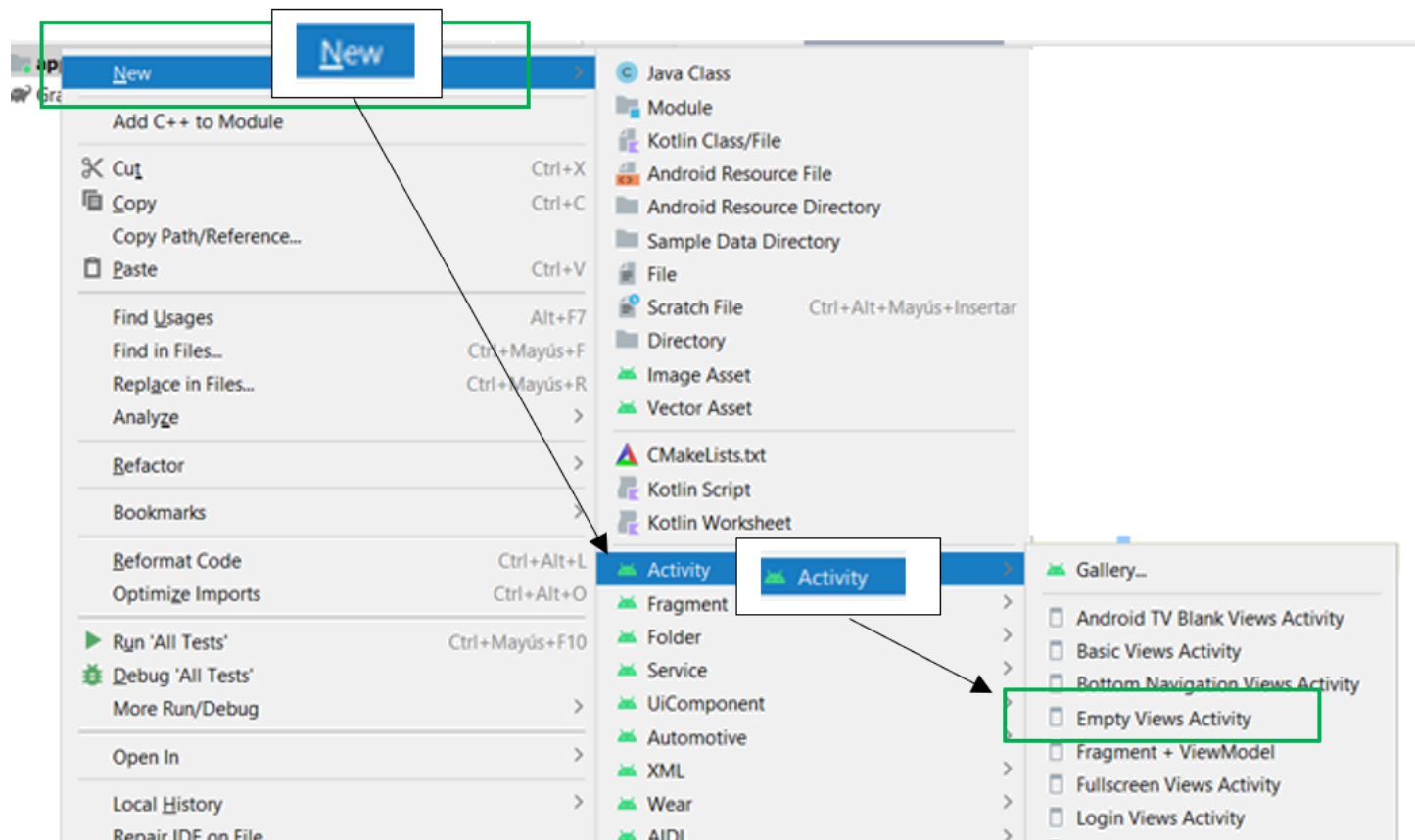
Pero, ¿Cómo hacemos para incorporar más de una *activity* al desarrollo?



Pasos para crear la nueva pantalla

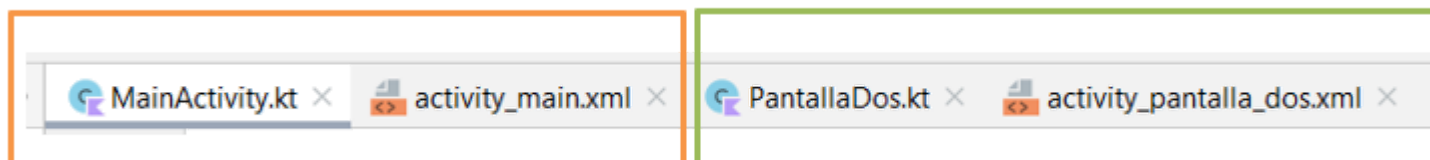
Lo primero es abrir un nuevo proyecto en el IDE, y como estuvimos viendo está compuesto de 2 elementos que son el archivo .kt y el archivo .xml (siempre que trabajemos con la plantilla [empty views activity](#))

Ya estamos listo para agregar la segunda *activity* ...



Seguimos todas las instrucciones que nos pide las ventanas con respecto al nombre, ubicación etc. Para esta explicación llamamos a la segunda *activity* [PantallaDos.kt](#).

El IDE nos muestra las siguientes solapas, las dos primeras corresponden a la *activity* inicial y las dos segundas a la *activity* que agregamos.



¿Cuáles son las líneas de código que nos permite pasar a la nueva *activity*?



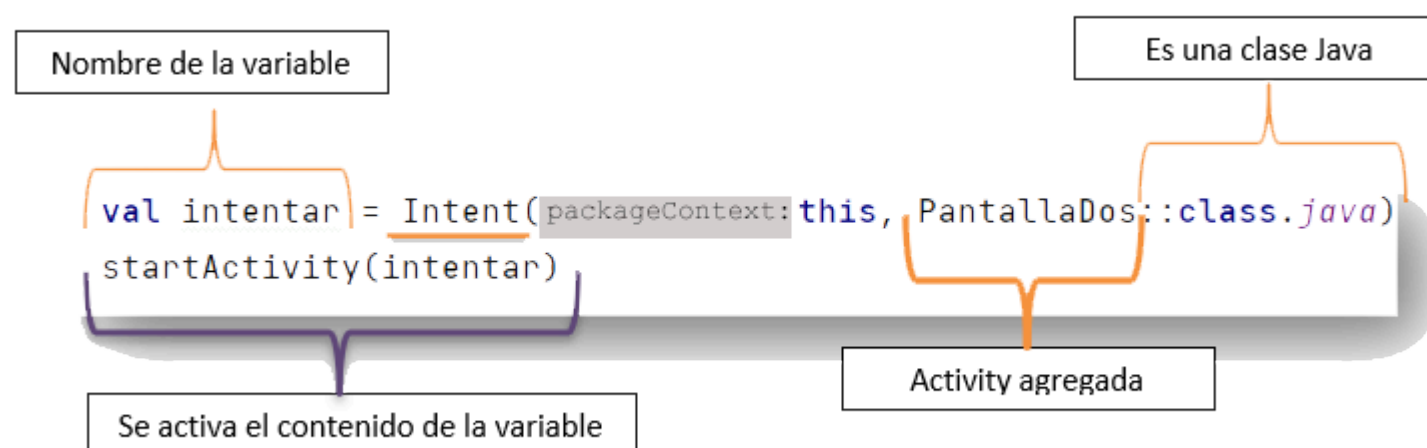
Código para pasar a la nueva pantalla

Aquí es donde usamos el Intent, veamos que tiene que contener la *activity*:

1. El **import** que haga referencia a este objeto (al comienzo).

```
import android.content.Intent
```

2. **Código:** En general estas líneas se ubican en un Button que tiene asignado el evento **setOnClickListener{ }**





Vínculo con base de datos

En esta tema te proponemos un viaje fascinante por el mundo de las [bases de datos con Android Studio](#). Imaginemos que nuestra aplicación es como una casa, y la base de datos es el almacén que sostiene y organiza todos los elementos que conforman nuestra experiencia digital.

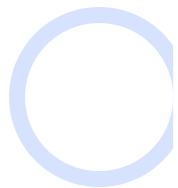
Vamos a explorar [cómo almacenar y gestionar datos de manera eficiente](#), permitiendo que nuestros usuarios, como invitados en esta casa, tengan una experiencia más personalizada y duradera. Pero, ¿cómo creamos un sistema que permita a diferentes usuarios ingresar a nuestra aplicación?

Aquí es donde entra la magia de las bases de datos. No solo [aprenderemos a usar SQLite](#), un sistema integrado en Android Studio, sino que también descubriremos cómo conectarnos a bases de datos externas como [MySQL](#), brindándonos flexibilidad y capacidad de actualización constante.

Preguntas clave de este tema:

- La información es su materia prima.
- ¿Cuál es la importancia de la persistencia de datos en una aplicación Android?
- ¿Qué base de datos utiliza Android Studio de forma predeterminada?
- ¿Cómo se estructura una base de datos relacional?
- ¿Cuál es la función de SQLiteOpenHelper en la creación y gestión de bases de datos SQLite?
- ¿Cómo se realiza la modificación de datos en una base de datos?

¡Empecemos!



Android y base de datos

¿Cómo podemos considerar cualquier usuario con su contraseña?

Para ello necesitamos tener almacenados los datos en una estructura externa que perdure en el tiempo, y que pueda ser actualizada. Esta estructura es la **base de datos** que nos permite crear una app dinámica y totalmente actualizada.

¿Qué base de datos usa Android Studio?

Podemos utilizar un sistema embebido como SQLite o conectar de diversas formas a una base de datos externa, como MySQL.

Veamos de que se trata.



Estructura de la base de datos

La **persistencia de datos** (almacenar o conservar datos en el dispositivo) es una parte importante del desarrollo de Android. Los datos persistentes garantizan que no se pierda el contenido generado por usuarios cuando se cierra la app.

Con el **objetivo** de conservar datos en las apps para Android, el SDK de Android ofrece SQLite. SQLite proporciona una base de datos relacional que te permite representar datos de forma similar a como estructura los datos con clases de Kotlin.

Es un concepto que has visto en asignaturas anteriores. Una base de datos relacional funciona de la siguiente manera:

- ❖ Las tablas definen agrupaciones de datos de alto nivel que querés representar.
- ❖ Las columnas definen los datos que contiene cada fila de la tabla.
- ❖ Las filas contienen los datos en sí que constan de valores para cada columna de la tabla.

La estructura de una base de datos relacional también refleja lo que ya sabés sobre clases y objetos en Kotlin.

```
data class Estudiante(  
    id: Int,  
    nombre: String,  
    apellido: String,  
    nacionalidad: String  
)
```

- ❖ Las clases, como las tablas, modelan los datos que deseás representar en tu app.
- ❖ Como las columnas, las propiedades definen los datos específicos que debe contener cada instancia de la clase.
- ❖ Los objetos, como las filas, son los datos reales. Los objetos contienen valores para cada propiedad definida en la clase, al igual que las filas contienen valores para cada columna definida en la tabla de datos.

Recordá que cada tabla de una base de datos relacional contiene un identificador único para las filas. Este identificador se conoce como la clave primaria.

Cuando una tabla hace referencia a la clave primaria de otra tabla, se conoce como clave externa. La presencia de una clave externa significa que existe una relación entre las tablas.

Nota

- Al igual que con las clases de Kotlin, la convención es usar la forma singular para el nombre de las tablas de la base de datos.



SQLite

Definición

SQLite es una base de datos relacional de código abierto que se utiliza para realizar operaciones de bases de datos en dispositivos Android, como almacenar, manipular o recuperar datos persistentes de la base de datos; hace referencia a una biblioteca C liviana para la administración de bases de datos relacionales con el lenguaje de consulta estructurado SQL

De forma predeterminada, **la base de datos SQLite está integrada en Android**. Por lo tanto, no es necesario realizar ninguna tarea de administración o configuración de la base de datos.

La clase **SQLiteOpenHelper** proporciona la funcionalidad para utilizar la base de datos SQLite .

La clase **android.database.sqlite.SQLiteOpenHelper** se utiliza para la creación de bases de datos y la gestión de versiones. Para realizar cualquier operación de base de datos, debe proporcionar la implementación de los métodos onCreate() y onUpgrade() de la clase SQLiteOpenHelper.



Constructor y métodos de la clase SQLiteOpenHelper

Constructores:

- Si queremos crear un objeto de SQLiteOpenHelper para crear, abrir y administrar la base de datos.

```
SQLiteOpenHelper(context: Context?, name: String?,  
factory: SQLiteDatabase.CursorFactory?, version: Int)
```

Parámetros del constructor de SQLiteOpenHelper

1. context: Context? → Opcional (puede ser nulo)

- Representa el contexto de la aplicación.
- Se usa para acceder a los recursos del sistema y abrir la base de datos.
- **Recomendado:** No debería ser `null`, ya que se necesita para operar con la base de datos.

2. name: String? → Opcional (puede ser nulo)

- Nombre del archivo donde se almacenará la base de datos.
- Si es `null`, la base de datos se creará en memoria (solo existirá mientras la app esté abierta).

3. factory: SQLiteDatabase.CursorFactory? → Opcional (puede ser nulo)

- Permite definir una fábrica de cursores personalizados para consultas en la base de datos.
- Si es `null`, se usa el manejador de cursores predeterminado de SQLite.
- Normalmente se deja en `null` a menos que se necesite personalizar el comportamiento de los cursores.

4. version: Int → Obligatorio (no puede ser nulo)

- Número de versión de la base de datos.
- Se usa para manejar actualizaciones y cambios en la estructura de la base de datos.
- **Debe ser mayor a 0.** Si cambia, se ejecutará `onUpgrade()` para actualizar la estructura.



Métodos de la clase SQLiteDatabase

La clase `SQLiteDatabase` contiene métodos que se realizarán en la base de datos SQLite, como crear, actualizar, eliminar, seleccionar, etc.

- 1 Ejecuta la consulta SQL, no una consulta de selección.
- 2 Inserta un registro en la base de datos. La tabla especifica el nombre de la tabla, `nullColumnHack` no permite valores completamente nulos. Si el segundo argumento es nulo, Android almacenará valores nulos si los valores están vacíos. El tercer argumento especifica los valores que se almacenarán.
- 3 Actualiza una fila.
- 4 Devuelve un cursor sobre el conjunto de resultados.

```
// 1. Ejecutar una consulta SQL (no de selección)
db.execSQL( sql: "DELETE FROM usuarios WHERE id = 1")

// 2. Insertar un registro
val valores = ContentValues().apply {
    put("nombre", "Juan")
    put("edad", 30)
}
db.insert( table: "usuarios", nullColumnHack: null, valores)

// 3. Actualizar una fila
val valoresActualizados = ContentValues().apply {
    put("edad", 31)
}
db.update( table: "usuarios", valoresActualizados, whereClause: "id = ?", arrayOf("1"))

// 4. Seleccionar datos (devuelve un cursor)
val cursor = db.rawQuery( sql: "SELECT * FROM usuarios", selectionArgs: null)
cursor.close() // Cerrar el cursor después de usarlo
```




Ejemplo con base de datos

El ejemplo que proponemos para la explicación crea solamente una tabla de usuario cuyos atributos son el nombre de ese usuario y su clave o contraseña.

Vamos a mostrar cómo crear la tabla y vincularla a la [clase](#) correspondiente (creada por nosotros en el IDE de programación) para tomar los datos desde el teclado del usuario de la App.

También te mostraremos la sintaxis necesaria para el famoso ABM de programación que es, [A](#)lta, [B](#)aja, [M](#)odificación.

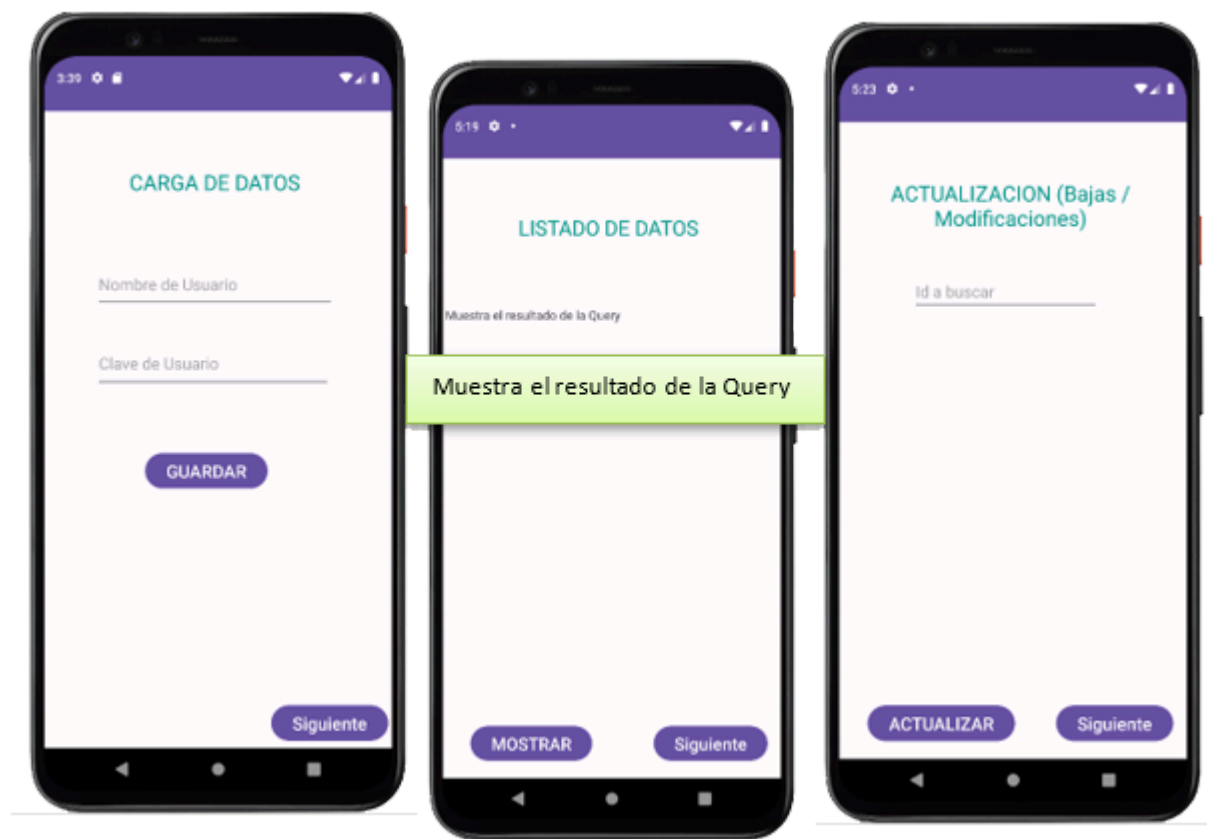
Tenemos que crear la [clase base de datos](#) donde escribiremos el código para manipular los datos.

Luego desde el [MainActivity.kt](#) con los componentes que tenga nuestro diseño usaremos las clases mencionadas.

¡Manos a la obra!



Diseño del ejemplo



La lógica es la siguiente, primero se debe cargar la tabla con el nombre y la contraseña del usuario, ya que consideramos un **id** como *primary key* de tipo *auto_incrementable*.

Una vez cargados los datos podemos ver la carga mostrando el contenido total de la tabla.

En la actualización se debe ingresar el **id** que queremos eliminar o el que corresponda a la fila que queremos modificar.

Nota

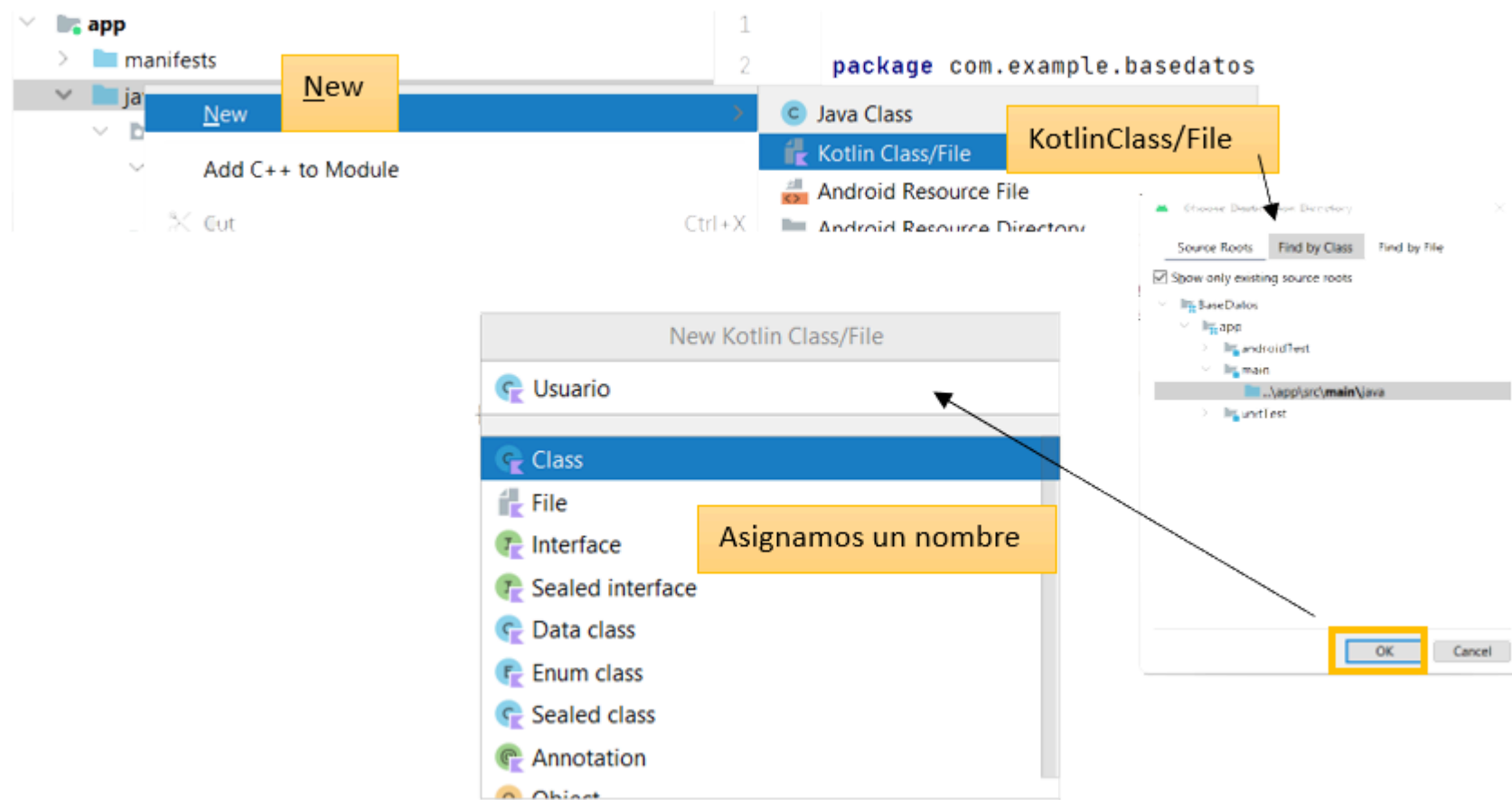
- Recordá que en cualquiera de las tres operaciones de ABM es conveniente mostrar algún mensaje que pregunte si son correctos los datos ingresados.



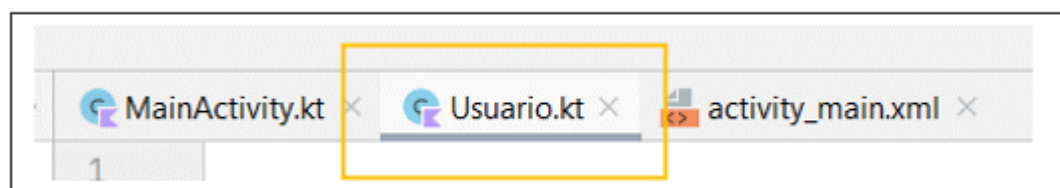
Código de creación de los datos

Como mencionamos al inicio de la explicación del ejemplo.

Después de crear un **nuevo** proyecto, creamos la **clase** que nos permite cargar cada una de las tablas de la base de datos, para ello debemos **agregarla** al proyecto. ¿Cómo lo hacemos?



Como la tabla de nuestra base de datos trata sobre los usuarios, nuestra clase es **Usuario**, nos quedan las pestañas del IDE de la siguiente forma:



Creamos el archivo de Clase para la base de datos (igual que te mostramos en la imagen anterior).

Usamos el **SQLiteOpenHelper**, podemos crear una variable con el nombre de la base para incluirla en la escritura de la sentencia. Completamos todos los parámetros. A continuación debemos implementar los miembros de la clase. ¿Cómo lo hacemos?

Seguramente nos aparece marcada como error la palabra reservada **Class** y el nombre que le dimos y al costado izquierdo la lamparita que automáticamente coloca el IDE, al hacer click en ella debemos tener como opción **implementar miembros**.

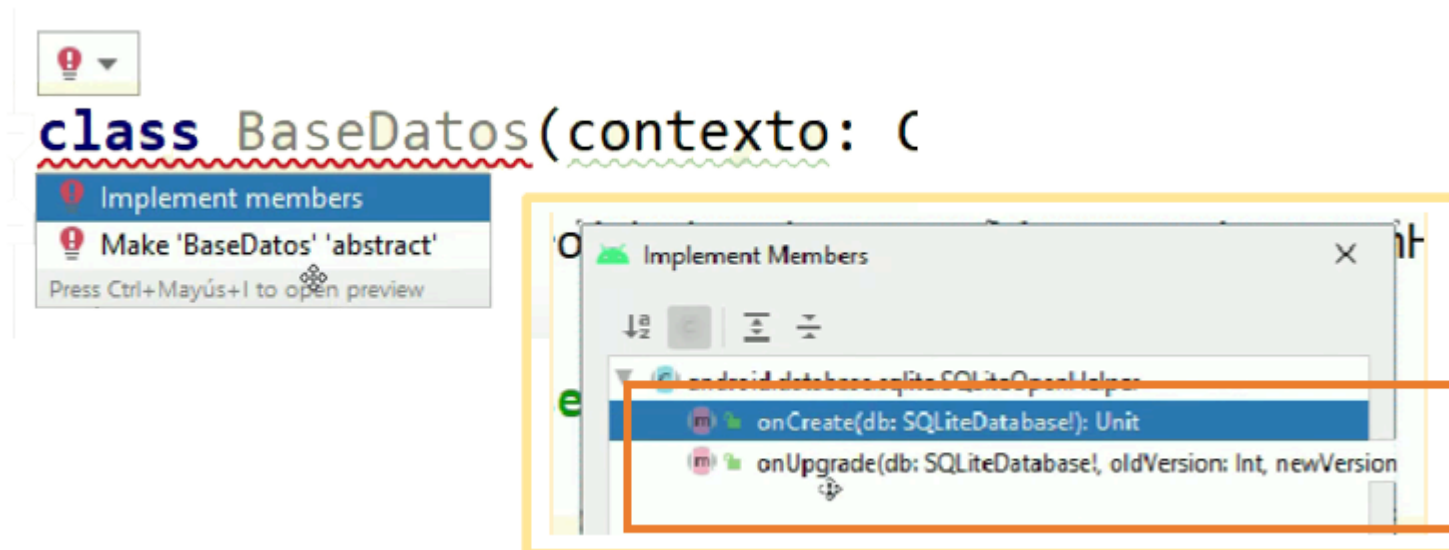
De esta forma creamos el **override** del **onCreate** y del **onUpgrade**. Implementamos los dos.

¡Veamos!



Override del onCreate y del onUpgrade

¿Como los creamos?



Quedando la clase de esta forma, observá que se llamó a la base [BaseDatos](#).

```
var BD = "BaseDatos"

class BDatos(contexto: Context): SQLiteOpenHelper(contexto, BD, factory: null, version: 1) {
    override fun onCreate(p0: SQLiteDatabase?) {
        TODO(reason: "Not yet implemented")
    }

    override fun onUpgrade(p0: SQLiteDatabase?, p1: Int, p2: Int) {
        TODO(reason: "Not yet implemented")
    }
}
```

Ya tenemos establecido en el entorno de nuestro celular que vamos a crear una base de datos con las tablas que la forman.

Usamos OnCreate para crear las tablas, contamos con una variable que almacena nuestra Query y luego usamos el [execSQL](#) para que se ejecute la consulta formulada.

```
class BDatos(contexto: Context): SQLiteOpenHelper(contexto, BD, factory: null, version: 1) {
    override fun onCreate(p0: SQLiteDatabase?) {
        var sql = "create table Usuario" +
            "(id INTEGER primary key autoincrement, nombre VARCHAR(100), clave INTEGER)";
        p0?.execSQL(sql)
    }
}
```



Código para alta del ABM

Creamos una función para insertar los dominios. A esta función le agregamos un retorno tipo texto para que nos indique si el *insert* fue exitoso o no.

```
fun insertarDatos(usuario: Usuario):String{
    val p0 = this.writableDatabase
    var contenedor = ContentValues()

    contenedor.put("nombre", usuario.nombre)
    contenedor.put("clave", usuario.clave)

    var resultado= p0.insert( table: "Usuario ", nullColumnHack: null, contenedor)
    if(resultado == -1.toLong()){
        return "Falla en la carga de datos"
    } else{
        return "Insert exitoso"
    }
}
```

Vamos a analizar el código:

Tomamos la referencia a la base (en este ejemplo `p0`) y especificamos que será de escritura, luego en otra variable creamos un “contenedor” de valores para tomar los ingresos a cada atributo de la tabla. Observá que usamos el constructor de la clase que creamos. Por último, para saber si el *insert* se ejecutó bien la variable `resultado` almacená el resultado de `p0.insert`.

Nota

Cuando estés escribiendo una sentencia como en este caso, la de `p0.insert`, las palabras que ves en gris claro **no debés escribirlas**, se autocompletan solas, quiere decir que el entorno reconoce los miembros de la clase.



Código para mostrar los datos

Cuando queremos mostrar el contenido de la base de datos, escribimos una Query tan simple o compleja según sea el requerimiento de visualización.

Hasta el momento vimos que trabajamos con la variable que tenemos vinculada nuestra base, en toda la explicación en [p0](#) y, como es de suponer, accedemos a ella en modo [lectura](#).

El resultado de la consulta es almacenado en un [cursor](#), ¿recordás el término de base de datos?, lo usaste en *procedures y/o triggers*

Este cursor contendrá las filas resultantes y tendrá la cantidad de columnas según sean los atributos proyectados en el *select* de la Query. Tendremos que posicionarnos en la primera fila y pasar una a una hasta el final para tomar cada dato que nos trae.

Veamos entonces cómo declaramos esas variables, dónde las guardamos para darle salida al exterior.

Creamos una función [traerDatos](#) que nos devuelve una lista tipo MutableList que contiene la clase Usuario.

```
fun traerDatos():MutableList<Usuario> {
    val p0 = this.readableDatabase
    var lista: MutableList<Usuario> = ArrayList()
    val sql = "select * from usuario"
    val resultado = p0.rawQuery(sql, selectionArgs: null)

    if (resultado.moveToFirst()) {
        do {
            var usu = Usuario()
            usu.id = resultado.getString(resultado.getColumnIndex(columnName: "id")).toInt()
            usu.nombre = resultado.getString(resultado.getColumnIndex(columnName: "nombre")).toString()
            usu.clave = resultado.getString(resultado.getColumnIndex(columnName: "clave")).toInt()
            lista.add(usu)
        } while (resultado.moveToNext())
        p0.close()
        resultado.close()
    } // fun traerDatos()
    return(lista)
}
```

A

B

A. Estas variables hacen referencia:

- [p0](#) al modo de uso de la base de datos [lectura](#),
- [lista](#) es la salida de la función de tipo *array*,
- [sql](#) la consulta que vamos a ejecutar para recuperar los datos,
- [resultado](#) representa a nuestro [cursor](#) que está relacionado a la query.

B. Con el if nos aseguramos de estar en la primera fila:

- [usu](#) representa a la clase Usuario,
- en cada atributo de la clase almacenamos el contenido de la columna del cursor (se lo puede referenciar también por la posición recordando que comienza de 0),
- lo cargamos al *array* con el método [add](#),
- con [moveToNext](#) pasamos de fila en fila hasta el final,
- cerramos la base de datos y el cursor.

- Finalmente retornamos la [lista](#).



Código para modificación y baja del ABM

Cuando queremos **actualizar (modificar) algún dato de la base** debemos identificar la tabla y la fila a modificar, salvo que se cambie el contenido de toda la columna de una tabla.

Si modificamos alguna fila en particular no debemos olvidar el valor que tiene la *primary key*.

Como venimos haciendo, construimos una función, la base debe ser accedida de tipo escritura ya que se modifican todos o alguno de los contenidos de la fila/s.

Pasemos a analizar entonces el código.

```
fun actualizaDatos(id:String,nombre:String,clave:Int):String{
    val p0 = this.writableDatabase
    var contenedor = ContentValues()

    contenedor.put("nombre",nombre)
    contenedor.put("clave",clave)

    var resultado= p0.update( table: "usuario",contenedor, whereClause: "id=?", arrayOf(id))
    if(resultado == 0){
        return "No se pudo actualizar la tabla"
    } else {
        return "Actualizacion exitosa"
    }
} // fin actualizarDatos
```

Para borrar una fila de la tabla debemos usar la *primary key*.

Ya estamos familiarizados con las líneas de código ¿verdad?, veamos entonces cómo escribir el borrado.

```
fun borrarDatos(codigo:String){
    if(codigo.length > 0){
        val p0 = this.writableDatabase
        p0.delete( table: "Usuario", whereClause: "id=?", arrayOf(codigo))
        /* si queremos borrar todos
        p0.delete("Usuario",null,null)
        */
        p0.close()
    }
} // fin borrarDatos
```




Preparar el entorno del proyecto en Android Studio

A lo largo de este tema, nos adentraremos en la transformación de proyectos previos, llevando los conocimientos adquiridos con [C#](#) a la práctica con [Kotlin](#).

En esta oportunidad, nos sumergiremos en el análisis documentado, rescatando la esencia de nuestras bases de datos y plantillas de casos de uso.

La misión es clara: [convertir ese valioso diseño en una aplicación vibrante y funcional](#).

Comenzaremos explorando cómo visualizar nuestras tablas en el IDE de Android, desentrañando las herramientas del App Inspection para analizar datos en tiempo real. Nos adentraremos en la creación de bases de datos con múltiples tablas, comprendiendo el poder del SQLiteOpenHelper y las mejores prácticas en la escritura de códigos.

Además, enfrentaremos el reto de cambiar la Activity de inicio y exploraremos cómo el cambio de IDE implica repensar a nuestros usuarios. Con consejos prácticos y la creación de la base de datos para el proyecto "Club Deportivo", esta semana nos sumergiremos en la realidad de programar una aplicación desde cero.

Preguntas clave de este tema:

- ¿Cómo podemos visualizar las tablas creadas en la base de datos SQLite dentro del IDE de Android?
- ¿Cuál es el propósito de usar el App Inspection y qué nos permite hacer?
- ¿Cómo debemos escribir el código cuando nuestra base de datos tiene más de una tabla?
- ¿Qué significa el uso de "companion object" en Kotlin y cómo se relaciona con las constantes en las Querys?
- ¿Cómo podemos cambiar la Activity de inicio en Android Studio?

¿Listo/a para empezar? ¡Adelante, el mundo del desarrollo te espera!



Introducción al entorno del proyecto

Hemos dado hasta el momento los primeros pasos en la programación de App utilizando Kotlin. Seguramente te preguntarás ¿qué haremos a partir de ahora?

Nuestra propuesta es transformar el proyecto de la materia *Desarrollo de sistemas orientado a objetos* en una App

Tendrás los **conceptos y tips** necesarios para codificar en Kotlin lo que codificaste en C#

¿Qué deberemos hacer?

Tendremos que recuperar el *Análisis* documentado, teniendo en cuenta que el diseño de la base de datos y las plantillas de caso de uso se convertirán en las estrellas del resto de las semanas.

Si pensamos en el desarrollo y las actividades que se deben resolver, podemos decir que la primera y fundamental es la comprensión del objetivo.

Teniéndolo claro, podemos dividir nuestras actividades en:

- ❖ **Análisis de los datos**, es decir construcción de la base de datos.
- ❖ **Programación de las acciones** que toman los datos y las convierte en salidas que cumplan con el objetivo.

¡Comencemos!



Cómo visualizar la Base de datos en el IDE

Para visualizar la/s tabla/s creadas en la base de datos en SQLite dentro del IDE de Android debés acceder al [App Inspection](#)



Si no encontrás al [App Inspection](#) en la base del IDE a la izquierda podemos desplegar un menú donde aparece el App Inspection, una vez seleccionado en el resto de los proyectos aparecerá en la barra inferior.

Al hacer click abre una ventana que muestra la base de datos del proyecto y la lista de tablas al seleccionar la que deseamos consultar, accedemos a los dominios ingresados.

Esta herramienta es utilizada para la visualización y análisis de datos en tiempo de ejecución, y no está diseñada para realizar operaciones de actualización directa en la base de datos.



Creación de la Base con varias tablas

Los datos almacenados en la Base de datos están distribuidos en varias tablas vinculados a través de relaciones respetando las reglas de normalización.

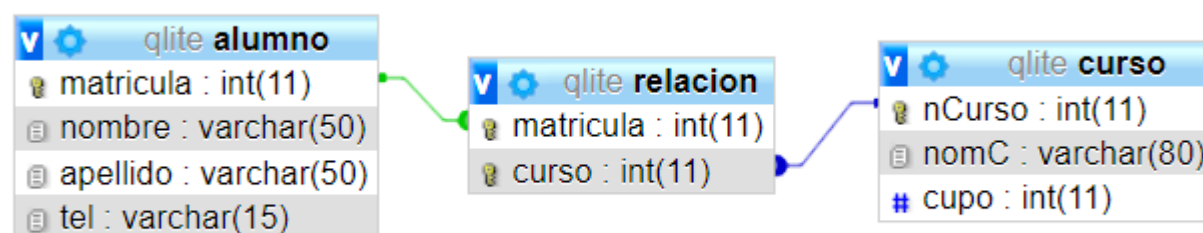
¿Cómo debemos escribir el código?

Sabemos que nuestra app debe tener una clase dedicada a la estructura de la base y además implementamos el onCreate y el onUpgrade de SQLiteOpenHelper.

Como todo lenguaje de programación existen comunidades o reglas a la hora de escribir código que permite que otros/as desarrolladores/as puedan comprender su sintaxis y lógica.

Te mostramos como escribir las Querys cuando la base tiene más de una tabla.

Este ejemplo corresponde al siguiente modelo:



Ya creada la clase destinada a la Base de datos, creamos el onCreate y el onUpgrade.

```

package com.example.nueva_dos

import android.annotation.SuppressLint
import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteOpenHelper
import android.database.sqlite.SQLiteDatabase

class Datos(contexto: Context): SQLiteOpenHelper(contexto, name: "BaseDos.db", factory: null, version: 2) {
    override fun onCreate(db: SQLiteDatabase?) {
        // Creacion de las tablas
        db?.execSQL(CREATE_ALUMNO_TABLE)
        db?.execSQL(CREATE_CURSO_TABLE)
        db?.execSQL(CREATE_RELACION_TABLE)
    }

    override fun onUpgrade(db: SQLiteDatabase?, p1: Int, p2: Int) {
        // se usa para las actualizaciones de las tablas
        db?.execSQL(sql: "DROP TABLE IF EXISTS alumno")
        db?.execSQL(sql: "DROP TABLE IF EXISTS curso")
        db?.execSQL(sql: "DROP TABLE IF EXISTS relacion")
        onCreate(db)
    }
}

```

Nombre del proyecto → nueva_dos

En cada **execSQL** encontramos como argumento una constante

Invoca antes de finalizar a la función **onCreate()**

Si observamos hay un **db?.execSQL** para cada tabla del modelo, y en lugar de colocar la sintaxis de la *query* de creación de la tabla nos encontramos con constantes que pasaremos a explicar.

```
// COMPANION OBJECT define miembros estáticos en una clase.
companion object {

    private const val CREATE_ALUMNO_TABLE = "CREATE TABLE alumno " +
        "(matricula INTEGER PRIMARY KEY AUTOINCREMENT, nombre TEXT, " +
        " apellido TEXT, tel INTEGER)"

    private const val CREATE_CURSO_TABLE = "CREATE TABLE curso " +
        "(nroC INTEGER PRIMARY KEY AUTOINCREMENT, nombreCur TEXT, cupo INTEGER)"

    private const val CREATE_RELACION_TABLE = "CREATE TABLE relacion " +
        "(matricula INTEGER, curso INTEGER, fecha TEXT, " +
        " PRIMARY KEY (matricula, curso)," +
        " foreign key(matricula) references alumno(matricula), " +
        " foreign key(curso) references curso(nroC))"

}
```

Estas constantes están dentro de `companion object { }` pero ¿qué significa?

Companion object en Kotlin

es una característica que te permite definir miembros estáticos en una clase. Los miembros definidos dentro de un *companion object* son similares a los miembros de una clase estática en otros lenguajes de programación.

Companion object también puede implementar interfaces, extender otras clases, y contener propiedades y funciones como cualquier otro objeto. Es especialmente útil cuando necesitas definir miembros que deben ser compartidos por todas las instancias de una clase o cuando necesitas proporcionar un espacio de nombres para ciertos elementos relacionados con la clase.

Observá que el nombre asignado a cada constante es el argumento de cada `execSQL`.

```
package com.example.nueva_dos

import android.annotation.SuppressLint
import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteOpenHelper
import android.database.sqlite.SQLiteDatabase

class Datos(contexto: Context): SQLiteOpenHelper(contexto, "BaseDos.db", null, 2) {
    override fun onCreate(db: SQLiteDatabase?) {
        // Creacion de las tablas
        db?.execSQL(CREATE_ALUMNO_TABLE)
        db?.execSQL(CREATE_CURSO_TABLE)
        db?.execSQL(CREATE_RELACION_TABLE)
    }

    override fun onUpgrade(db: SQLiteDatabase?, p1: Int, p2: Int) {
        // se usa para las actualizaciones de las tablas
        db?.execSQL("DROP TABLE IF EXISTS alumno")
        db?.execSQL("DROP TABLE IF EXISTS curso")
        db?.execSQL("DROP TABLE IF EXISTS relacion")
        onCreate(db)
    }

    // COMPANION OBJECT define miembros estáticos en una clase.
    companion object {

        private const val CREATE_ALUMNO_TABLE = "CREATE TABLE alumno " +
            "(matricula INTEGER PRIMARY KEY AUTOINCREMENT, nombre TEXT, " +
            " apellido TEXT, tel INTEGER)"

        private const val CREATE_CURSO_TABLE = "CREATE TABLE curso " +
            "(nroC INTEGER PRIMARY KEY AUTOINCREMENT, nombreCur TEXT, cupo INTEGER)"

        private const val CREATE_RELACION_TABLE = "CREATE TABLE relacion " +
            "(matricula INTEGER, curso INTEGER, fecha TEXT, " +
            " PRIMARY KEY (matricula, curso)," +
            " foreign key(matricula) references alumno(matricula), " +
            " foreign key(curso) references curso(nroC))"

    }
}
```

¿Cómo procedemos luego de estas líneas de código?

Igual que lo que estudiamos en la semana 7, es decir, cada tabla tiene su clase correspondiente y en la clase de la base de datos codificamos las funciones que nos permitan actualizar (*insert*, *update*, *delete*) los dominios de las tablas. Luego serán invocadas dichas funciones en el MainActivity que corresponda.



Comenzando con la App del proyecto “Club deportivo”

La mejor manera de aprender un nuevo lenguaje de programación junto a su entorno es tener un objetivo que cumplir, y si ese objetivo ya fue desarrollado nos permite enfocarnos en la sintaxis propiamente dicha despreocupándonos del razonamiento lógico.

Vamos a transformar el proyecto de la materia *Desarrollo de sistemas orientado a objetos* en una *App*.

¿Migro la sintaxis de cada línea codificada en C# a Kotlin?, ¿Mantengo el mismo diseño de las pantallas?,
¿El acceso a la aplicación es igual?

Estas preguntas pueden surgir de manera espontánea, todas sus respuestas pueden ser afirmativas para un/a estudiante y negativas para otro/a, esto sucede porque [están íntimamente ligadas a la lógica de razonamiento del/la diseñador/a y programador/a](#).

- 1 **Lo primero que debemos hacer** es armar un croquis de la cantidad de pantallas (Activity's) que tendrá nuestra App, y cuales son las acciones que nos permiten interactuar entre ellas.
- 2 **Lo segundo es** determinar el usuario que identificamos en el desarrollo en C#, si es el mismo en la App de Kotlin. En este caso trabajaremos únicamente en la app para el personal administrativo.
- 3 **Y, lo tercero es** crear las clases necesarias para tener la base de datos a punto en SQLite para comenzar con la codificación.



Primeros tips para el desarrollo de la App

Tips

En programación generalmente se refiere a **consejos, sugerencias o trucos** que los programadores comparten para mejorar la eficiencia, la calidad del código, o para evitar errores comunes.

Estos tips **pueden abordar aspectos específicos del lenguaje de programación**, las herramientas de desarrollo, las prácticas de codificación o la gestión de proyectos.

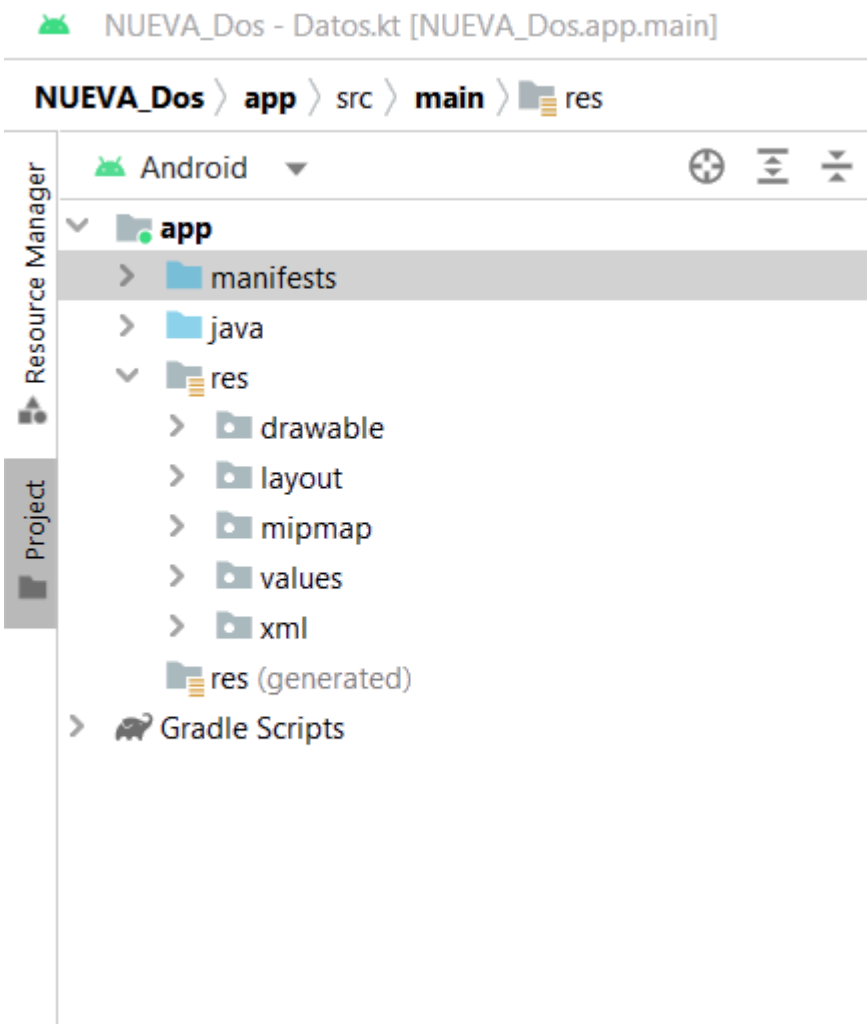


Cambio de Activity de inicio

Cuando construimos un desarrollo que involucra diferentes pantallas, comenzamos muchas veces con la codificación de algún módulo y luego a medida que avanzamos y encontramos la funcionalidad y el mejor acceso para el usuario cambiamos la *pantalla inicial*.

¿Cómo lo hacemos en este IDE?

Debemos identificar en el árbol del proyecto el archivo *Manifests*.



Dentro del contenido de este archivo *AndroidManifest.xml* nos muestra la cantidad de Activity que contiene el proyecto. Siguiendo el ejemplo mostrado en el modelo de datos de esta semana tenemos las siguientes Activity para cargar datos y mostrar a los *Alumnos, Cursos y su relación*.

```
<activity
    android:name=".MainMostrar"
    android:exported="false" />
<activity
    android:name=".MainRelacion"
    android:exported="false" />
<activity
    android:name=".MainCurso"
    android:exported="false" />
<activity
    android:name=".MainAlumno"
    android:exported="false" />
<activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
</application>
</manifest>
```

Vemos que la llamada MainActivity tiene el bloque *<intent-filter>* este bloque identifica cual es la actividad de inicio. Simplemente cambia el nombre de la activity que tiene el bloque *<intent-filter>* por la que deseas iniciar.

Después de hacer estos cambios, debes guardar el archivo AndroidManifest.xml y vuelve a compilar y ejecutar la aplicación. La nueva actividad que has configurado debería ser ahora la actividad de inicio.



Nuevo IDE nuevos usuarios

Tal vez te preguntes [por qué decimos nuevo IDE nuevos usuarios](#).

Cuando desarrollaste el sistema de Club Deportivo en C# el usuario fue el [empleado del gimnasio](#) que registraba las actividades del socio cumpliendo los requerimientos del objetivo de ese sistema.

Entre las acciones codificadas existe un módulo que generó la impresión del carnet del socio, ¿verdad? Ahora todo este desarrollo se convierte en una App y si manejas un modulo de usuario y contraseña puedes internamente dividir la funcionalidad de los procesos para dos usuarios uno de ellos sigue siendo el empleado del gimnasio y el otro el [socio](#).

[Incorporar al socio permite que su carnet lo pueda visualizar de forma directa sin necesidad de tener la impresión como en el desarrollo anterior.](#)

Observa que si incorporas este [tip](#) deberías agregar a la tabla socio una contraseña para el ingreso a la App utilizando como usuario el dato que es único como su documento de identificación o si es de tu agrado agregarle un alias, pero atención a la hora de ingresar ese alias ningún otro socio lo debe tener.

A medida que avancemos en el desarrollo, iremos agregando otros tips.

E4: Gestión integral en la app del club deportivo

Sitio: Agencia de Habilidades para el Futuro

Curso: Desarrollo de Aplicaciones para Dispositivos 2° A

Libro: E4: Gestión integral en la app del club deportivo

Imprimido por: Johana Micaela Navarro

Día: miércoles, 6 de mayo de 2026, 11:10

Descripción

Tabla de contenidos

- 1. Introducción al libro
- 2. TEMA I. Registro de socios
- 3. Introducción
- 4. Tips para el registro de socio
 - 4.1. Alias único
 - 4.2. Código de ejemplo
 - 4.3. Uso de fechas
- 5. Tips para el diseño y movilidad entre las Activity's
 - 5.1. Sugerencia de movilidad
- 6. TEMA II. Cobro de cuota
- 7. Introducción.
- 8. Tips para el Cobro de cuota
 - 8.1. Lógica para el cobro
 - 8.2. Cobro en el registro de socio
 - 8.3. Cobro de la cuota mensual
 - 8.4. Documento de pago
 - 8.5. Código del documento
 - 8.6. Código del button
- 9. Tips para el diseño del carnet de socio
- 10. TEMA III. Listado de socios
- 11. Introducción
- 12. Tips para el listado diario
 - 12.1. Lógica para obtener el listado
 - 12.2. Diseño del listado
 - 12.3. Código para el listado en la clase de la base de datos
 - 12.4. Código para el listado en la activity que tiene el GridLayout
- 13. Etapa 5:Despliegue y distribución de la aplicación



Introducción

¡Comenzamos con otra etapa de aprendizaje! Llegó el momento de comenzar a aplicar todo lo aprendido al desarrollo de la app para nuestro cliente, **el Club Deportivo**.

Para ayudarte con el desarrollo, presentaremos tres temas que te orientarán en la tarea:

- I Registro de socio
- II Cobro de cuota
- III Listado diario

Como siempre, es importante prestar atención al contenido, pero también ir aplicando estos temas a tu propio desarrollo.

¡Empezamos!



Registro de socios

Es momento de empezar a [aplicar nuestros conocimientos en el desarrollo de la aplicación móvil para el Club Deportivo](#).

- [Algo fundamental en toda institución es](#) conocer el número de socios: para eso, nos sumergimos en el "Registro de socio".
- [Optamos por que el empleado gestione el registro](#), capturando datos específicos como alias y contraseña para convertir al socio en un usuario de la app.

[En cuanto a fechas, en Kotlin, utilizamos java.time para jugar con ellas: abono de cuotas, fecha de vencimiento, ¡todo resuelto!](#)

También te dejamos un consejo sobre movilidad y diseño, con dos tipos de usuarios: el empleado con acceso a módulos específicos y el socio solo visualizando su carnet.

En la actividad obligatoria vamos a codificar la pantalla inicial y el [login](#) para el empleado.

[Preguntas clave de esta semana:](#)

- ¿Cómo podemos garantizar la unicidad de los alias en el registro de socios y qué método se sugiere para implementarlo?
- ¿Cuál es la importancia de trabajar con fechas en Kotlin para el desarrollo de la app del Club Deportivo?
- ¿Cómo se sugiere manejar la movilidad entre actividades y qué componente visual se destaca en el diseño de pantallas?
- ¿Cuáles son los pasos propuestos para codificar la pantalla inicial y el *login* del empleado en Android Studio?

¿Listo/a para empezar? ¡Adelante, el mundo del desarrollo te espera!



Introducción

A partir de ahora, [presentaremos ayudas para desarrollar los distintos módulos del Club deportivo](#). ¡Es momento de pasar a la acción!

- ❖ Estos [tips](#) se pueden aplicar en su totalidad en la codificación o simplemente actuar como disparador de lógicas propias de los programadores del equipo.
- ❖ El módulo que trataremos es el [registro de socio](#).
- ❖ Te mostraremos [cómo lo pensamos desde el punto de vista lógico](#) con los correspondientes controles, vos y tu equipo pueden pensarlo diferente pero estamos seguros que estos tips te ayudaran para la resolución.



Tips para el registro de socio

Tips	Descripción
Registro de socios	• Considera que el registro de socios en la aplicación del Club Deportivo se realice internamente en el gimnasio, en lugar de permitir que los propios socios se registren. Esto garantiza la recopilación de datos específicos y precisos requeridos por el club.
Datos mínimos vs datos específicos	• A diferencia de otros sitios web donde los registros solicitan datos mínimos, como correo electrónico y contraseña, en el caso del club deportivo, pueden ser necesarios datos específicos adicionales sobre los socios. • Esto justifica que el registro sea realizado por un empleado del gimnasio.
Migración del desarrollo	• Ten presente que estás migrando el desarrollo del Club Deportivo codificado en C#, lo que implica que el registro de socios se integrará en el sistema existente del gimnasio.
Diseño de pantalla personalizada	• El diseño de la pantalla de registro de socios dependerá de los datos específicos que se consideren almacenar en la base de datos del club deportivo. Asegúrate de diseñar la pantalla de manera intuitiva y eficiente, teniendo en cuenta los campos necesarios para el registro.
Validación de alias único	• Implementa un control para verificar que el alias elegido por un socio durante el registro sea único entre los demás socios. Esto garantiza la identificación única de cada socio en la plataforma.

Entre los datos solicitados debemos tener una contraseña y un alias.

Pasemos a ver cómo controlar que el alias sea único entre los socios.



Alias único

Lógicamente lo que debemos hacer es lo siguiente. Antes de enviar los datos ingresados a la clase *socio* para insertarlos en la tabla se debe verificar que el dato considerado como alias no exista.

Para ello podemos escribir una *query* que tenga en su *where* el dominio del atributo alias, si esa *query* arroja resultado es decir, tiene una fila nos indica que *existe* ese dominio.

En conclusión

- Debemos obtener el número de filas del resultado de la *query*.
- SQL cuenta con la función `count(nombre_atributo)` para contar las filas resultantes.
- Este tipo de consulta devuelve un solo valor, que es el recuento de filas en la tabla especificada.
- Podés asignar el resultado a una variable en tu aplicación o utilizarlo como parte de una condición en una consulta más compleja.
- Si es 0, procedemos a insertar todos los datos ingresados, caso contrario podemos mostrar un aviso indicando que debe elegir otro alias y dejar el foco del cursor en el `editText` de ingreso de ese dato.



Código de ejemplo

Tomamos un ejemplo que vimos anteriormente donde la base de datos tiene una tabla llamada **curso**.

En la clase que hace referencia a la base de datos creamos una función para buscar en este caso el nombre del curso (se llama **buscaCurso**) y en el MainActivity donde programamos lo referido al *insert* de la tabla curso invocamos a la función de búsqueda.

Veamos entonces cómo queda la sintaxis.

- En la clase de la base de datos:

```
fun buscaCurso(nombre: String): Int {
    val bd = this.readableDatabase

    // Consulta para contar las filas con el nombre especificado
    val query = "SELECT COUNT(*) FROM curso WHERE nombreCur = ?"
    val selectionArgs = arrayOf(nombre)

    val cursor: Cursor = bd.rawQuery(query, selectionArgs)

    var cantidad = 0

    if (cursor.moveToFirst()) {
        cantidad = cursor.getInt( columnIndex: 0)
    }

    cursor.close()
    bd.close()

    return cantidad
} // fin buscarCurso
```

Accedemos a la base en modo lectura

En una variable tipo **Cursor** guardamos el resultado de la query

Asignamos a una variable el valor 0, ella tomara en caso de tener filas el resultado el valor arrojado por la query, si es empty queda en 0

Ingresa al if siempre que tenga por lo menos una fila

Retorna el contenido de la variable **cantidad**

ATENCIÓN la clase debe tener

```
import android.database.Cursor
```

- En el MainActivity del ingreso de datos de curso:

```

fun GuardarCurso(view: View){
    var datos = Datos( contexto: this)
    var cur = Curso()

    if(nomCur.text.toString().length > 0 && cant.text.toString().length > 0){

        // agregamos lineas para verificar que el curso no exista
        var filas = datos.buscaCurso(nomCur.text.toString())
        if (filas == 0) {
            cur.nombreCur = nomCur.text.toString()
            cur.cupo = cant.text.toString().toInt()
            mensaje = datos.insertarCurso(cur)
            Toast.makeText( contexto: this, mensaje, Toast.LENGTH_SHORT).show()
        }else{
            Toast.makeText( contexto: this, text: "ATENCION!!! EL NOMBRE DE CURSO EXISTE", Toast.LENGTH_SHORT).show()
        }
    }else{
        Toast.makeText( contexto: this, text: "SE DEBE COMPLETAR TODOS LOS DATOS", Toast.LENGTH_SHORT).show()
    }
    nomCur.text.clear()
    cant.text.clear()
    nomCur.requestFocus()
} // fin GuardarCurso

```

Una vez que verificamos tener todos los datos ingresados **invocamos** a la función de búsqueda y el retorno lo guardamos en **filas**

Si su valor es 0 insertamos los datos

Si su valor es > 0 mostramos un aviso

De esta manera evitamos que aparezca, en la tabla curso, el mismo nombre de curso con número de curso diferente (recordar que el número de curso es auto incrementable).

Como verás, es sencillo la búsqueda de un dominio en particular dentro de un atributo de una tabla.



Uso de fechas

En Kotlin, podés trabajar con fechas utilizando las clases proporcionadas por el paquete [java.time](#), que forma parte de las API de fecha y hora introducidas en Java 8.

Por ejemplo, si queremos obtener la fecha y hora actuales, el código es el siguiente:

```
val buttonFecha = findViewById<Button>(R.id.btnFecha)
buttonFecha.setOnClickListener() { it: View!
    val fechaHoraActual: LocalDateTime = LocalDateTime.now()
    Toast.makeText(context: this, text: "Fecha y hora actuales: $fechaHoraActual", Toast.LENGTH_SHORT).show()
}
```

La *activity* tiene un *button* cuyo id es `btnFecha` y en el evento `setOnClickListener` instanciamos una variable como `LocalDateTime` y tomamos en valor actual con `now()`

Luego con el `Toast` le dimos salida al valor obtenido.

Nota

- No olvides colocar **import java.time.LocalDateTime**
- Este código te lo presentamos en un botón para que lo puedas probar, pero seguramente el uso se lo darás dentro de la lógica de tu App.
- Si lo que necesitás es solamente la fecha la clase a instanciar, es `LocalDate`.

En el proyecto del Club deportivo

la fecha se usa cuando el socio abona la cuota, ya que se debe indicar la fecha de vencimiento del pase que adquiere. También se usa si este tipo de dato corresponde a algún atributo de las tablas del modelo de datos.

Otro tip que te damos con respecto a la fecha, es a sumarle meses o días, ¿para que te sirve? Pues bien, cuando el socio paga, se debe tomar del sistema la fecha actual y a ella sumarle *un mes* para conocer el vencimiento. Te imaginarás que no es un cálculo manual; la App debe calcularlo por si sola.

Para sumarle un mes a una fecha, usamos el método `plusMonths()` de la clase `LocalDate`; se utiliza para devolver una copia de esta fecha con los meses especificados agregados. La clase tiene los métodos para que en lugar de meses sumes días o años.

[Veamos cómo es la sintaxis:](#)

Mostramos un recorte de código En el ActivityMain.xml se usa un textView para dar salida

```
vista = findViewById(R.id.viewMostrar)
val fechaHoy: LocalDate = LocalDate.now()
vista.append("\n sumamos un mes " + fechaHoy.plusMonths( monthsToAdd: 1))
```

Salto de línea

La variable ***fechaHoy*** contiene a la fecha actual

El argumento del método ***plusMonth*** tiene la cantidad de meses a sumar, en este caso **1**



Tips para el diseño y movilidad entre las Activity's

Cuando nos referimos a la movilidad entre actividades en el contexto de desarrollo de aplicaciones móviles, generalmente estamos hablando sobre la transición o navegación entre pantallas o secciones diferentes de la aplicación.

¿Qué vamos a hacer entonces? Tal vez quieras que en el diseño de las pantallas se visualice alguna imagen, veamos cómo hacerlo:

- Se usa el componente *ImageView* en el diseño (*layout*) de la actividad y la imagen elegida se carga en ese *ImageView* o mediante recursos.
- Para cargar la imagen desde recursos, se debe utilizar *setImageResource*, la imagen debe estar en la carpeta *res/drawable*, pero si preferís cargar imágenes desde una URL, se puede utilizar bibliotecas como Picasso o Glide, teniendo en cuenta que al cargar imágenes desde una URL, se deben manejar las operaciones en segundo plano para evitar bloquear el hilo principal.

¿Cómo encontramos la carpeta *res/drawable*?

Localizamos el directorio del proyecto Android, seleccionamos la carpeta *app*, luego *msrc*, *main*, *res*. Dentro de esta está la carpeta *drawable*, ¡atención! si no existe, debés crearla.

Copiamos el archivo en esa carpeta teniendo en cuenta que la extensión debe ser png o jpg, y su nombre comenzar en minúscula y sin espacios.



Sugerencia de movilidad

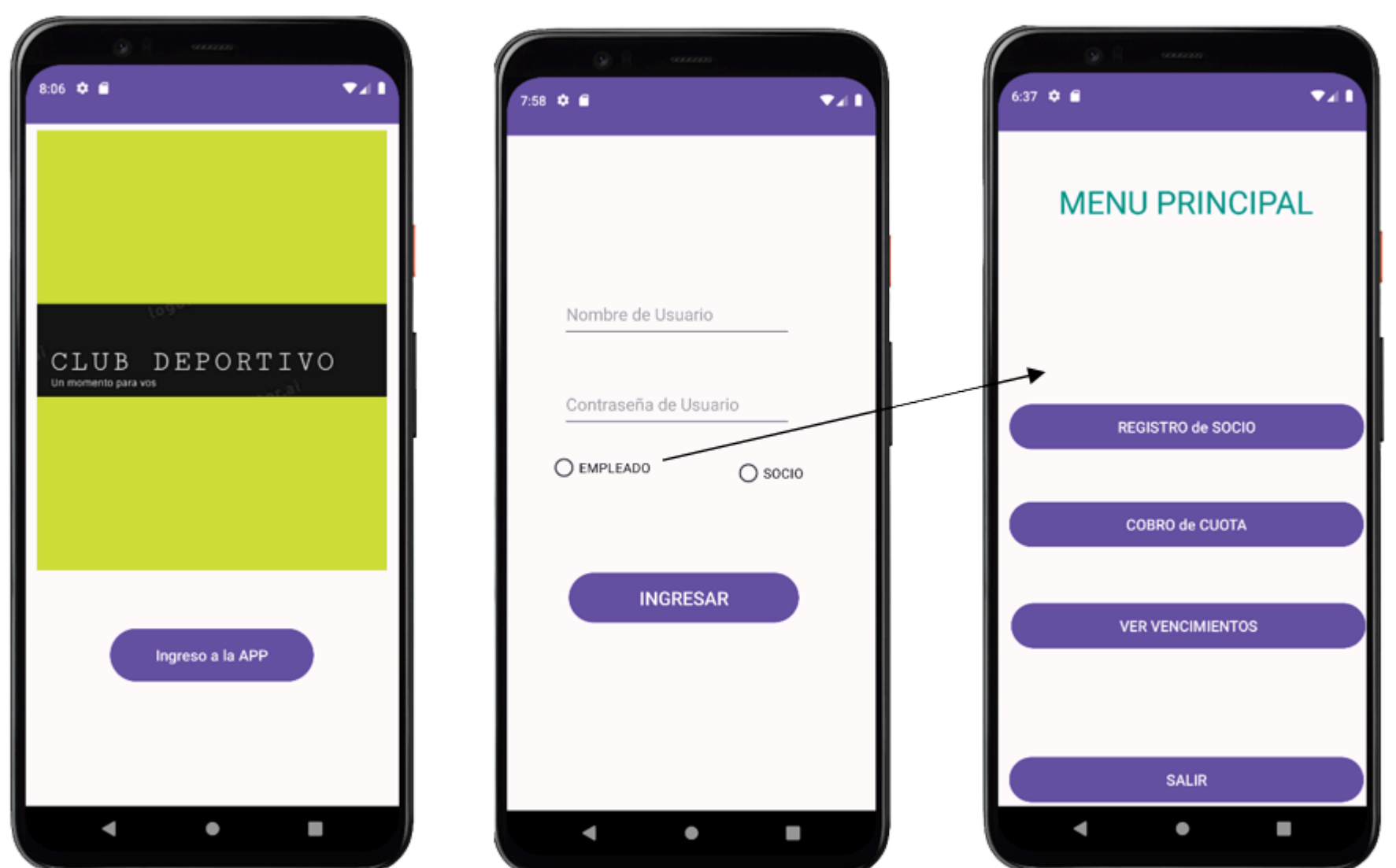
La App del Club deportivo puede tener dos tipos de usuarios, que son el empleado del club y el socio.

Te mostramos nuestra sugerencia de resolución.

- 1 La pantalla de inicio la imaginamos con una imagen / logo / color de fondo (acorde a cada desarrollador) con 1 botón *Ingreso a la App* que nos permite iniciarla.
- 2 A continuación ingresamos el nombre de usuario y contraseña, indicando además el perfil del usuario.
- 3 Suponemos que en **el modelo existe una tabla llamada usuario**, que tiene como usuario modelo, un nombre y contraseña genérico. Este ingreso no pertenece a ningún módulo de esta App, por lo que deberá ser incorporado por fuera o considerar como información confidencial un nombre y contraseña fijas, que estarán integradas en la líneas de código. **Si el perfil es de un socio**, podrá acceder después de que el empleado realice el registro e incluya, entre sus datos, lo sugerido con respecto al alias.

Una vez dentro de la App, la interacción es diferente:

- Si es el **empleado**, tiene acceso a los módulos de Registro de socio, Cobro de cuota, Visualización de los vencimientos (módulos identificados en el análisis realizado en la materia Desarrollo de sistemas orientado a objetos).
- Si es el **socio**, tiene acceso solamente a la visualización de su carnet (que se genera cuando se abona la primera cuota, es decir cuando se registra por primera vez). El diseño queda a criterio de los/las desarrolladores/as.





Cobro de cuota

En esta ocasión, nos sumergimos en los módulos del Club Deportivo, proporcionando una experiencia práctica fundamental para comprender los conceptos de codificación en dispositivos móviles.

Enfocándonos en el módulo de "**Cobro de cuota**", estudiaremos que este no solo integra los conocimientos adquiridos hasta ahora, sino que también presenta particularidades al incluir la emisión del carnet del socio y el documento de cobro de la cuota.

Novedad clave

Al tratarse de una aplicación, los socios tienen acceso exclusivamente para visualizar su carnet, marcando una diferencia significativa respecto al desarrollo anterior en Desarrollo de sistemas orientado a objetos.

Este enfoque estratégico resalta la importancia de la **practicidad** y la **adaptación a las necesidades específicas** de una aplicación para dispositivos móviles.

Preguntas clave de este tema:

- ¿Cuándo se activa el módulo de Cobro de cuota y qué particularidades tiene?
- ¿Por qué es esencial contar con un registro de precios al tratar el cobro de servicios en la aplicación?
- ¿Cómo se maneja el proceso de cobro si se genera inmediatamente después del registro del socio?
- ¿Cuáles son las consideraciones y la lógica detrás del diseño del carnet de socio?



Introducción

En esta oportunidad pasaremos a analizar el módulo de **Cobro de cuota**. Este módulo, además de incorporar lo visto hasta este momento, tendrá sus particularidades ya que contamos con la **emisión del carnet del socio y del documento** que nos avala el cobro de la cuota

La novedad que incorporamos con respecto al desarrollo anterior (en la materia Desarrollo de sistemas orientado a objetos) es que al ser una App el socio puede acceder solamente para visualizar su carnet.



Tips para el Cobro de la cuota

Pasamos a contar lógicamente de que se trata este módulo.

Este módulo es llamado o activado desde dos situaciones:

- 1 **El socio se registra por primera vez en el Club deportivo** y paga su primera cuota dando origen al **carnet de socio**,
- 2 **Se encuentra al día siguiente del vencimiento de su cuota** y para acceder al club deportivo debe pagar el mes entrante.

En ejemplos anteriores se propuso el siguiente diseño:



Como podemos observar

- Cuenta con un **botón** que hace referencia al cobro de la cuota que se usa en cada vencimiento de la misma.
- **Cuando se trata del registro del socio** una vez confirmados los datos e ingresados a la base de datos, la App, debe enviarnos automáticamente a la pantalla de cobro de cuota sin pasar por el click del botón que lo invoca.



Lógica para el cobro

Cuando estamos frente al cobro de un servicio, como es este caso, contamos con un [registro de precios ya sea una escala o un precio único](#).

Tal vez te preguntes ¿por qué y para qué tener un registro de precios? Analicemos juntos la situación.

Ejemplo Si contamos con un registro de precios el código de forma automática calcula el monto final y nos aseguramos que mientras ese registro este actualizado no incurrimos en errores a la hora de cobrar el servicio. Por el contrario si no contamos con el registro de precios el empleado responsable de cobro de la cuota debe ingresarlo manualmente, pudiendo incurrir en errores intencionales o no intencionales.

Recordá que estos son tips para la codificación y tal vez tu modelo de datos no cuenta con una tabla que tenga los precios que cobra el club deportivo.

Tal vez en el relevamiento inicial no se consideró la posibilidad de que el socio quiera concurrir 15 días en lugar de un mes entero y no siempre el valor de los 15 días es la mitad del valor del mes entero.

Como son sugerencias tendrás que amoldarte a tu modelo de datos.



Cobro en el registro de socio

Si el cobro se genera inmediatamente después del registro del socio, ya contamos con el **id** o **dato clave** asignado al nuevo socio que permite identificarlo entre todos ellos.

- ❖ Si consideramos la posibilidad de pagar 1 mes o 15 días, el diseño debe mostrar un **Editas** para ingresar esa información o contar con opciones ya que puede ser un periodo de tiempo o el otro (nunca ambos juntos) o el componente que más te guste que permite tomar información desde el teclado
- ❖ Si contamos con un registro de precios, consultamos con la tabla de la base de datos correspondiente, caso contrario con un **ediText** ingresamos el monto.

El id o dato clave

lo usaremos para almacenar en la tabla que registre la operación de cobro y a su vez nos permite recuperar los datos significativos del socio como ser, su nombre y apellido, de la tabla socio en caso de no traerlos desde la pantalla anterior.

Una vez registrado el cobro de la cuota podemos pasar a mostrar el documento de pago.



Cobro de la cuota mensual

Si el cobro hace referencia a la mensualidad que abona el socio para ingresar al club debemos determinar cuál o cuáles son los datos que se solicitan para proceder con el circuito.

- ❖ **¿Podemos solicitar el nro. o id del socio? ¡Sí!**
- ❖ **Pero, el socio ¿recuerda ese número?** Puede ocurrir que alguno no lo recuerde, entonces, si solicitamos ese dato debe existir otras opciones de filtro para la búsqueda del socio.

La opción que nunca falla es el tipo y número de documentos, no incluimos el nombre y apellido ya que la combinación de ellos puede repetirse en más de un socio.

Ese datos nos permite verificar si esta vencido el plazo de la cuota y quién es el socio para usar los datos significativos en el documento de pago.

Una vez registrado el cobro de la cuota podemos pasar a mostrar el documento de pago.



Documento de pago

El **documento de pago** puede tener el diseño que determine el equipo de desarrollo (ticket, recibo, etc.)

Sus datos son:	• el número del documento, • nombre del documento, • fecha, • datos del emisor del documento, • datos del receptor del documento, • detalle y, • monto.
----------------	---

 Pasemos ahora a analizar este documento desde Kotlin

Si consideramos lo explicado en las páginas anteriores vemos que el documento se visualiza en la pantalla siguiente de la que manipulamos los datos.

Cuando queremos pasar de una *activity* a otra usamos el *intent* para invocar a la activity que queremos pasar, ¿te acuerdas? Vamos a usar dentro del intent al método `putExtra` para llevar los datos que deben aparecer en el diseño del documento.

 Veamos de qué manera usamos las líneas de código:

En la semana 8 cuando tratamos la creación de la base de datos y en el modelo que usamos de ejemplo tenemos las tablas `alumno` , `curso` y la relación entre ellos.

 Vamos a analizar la siguiente lógica:

En una *activity* se pide ingreso de un número de matrícula para mostrar en otra *activity* (pantalla), con un determinado diseño, el nombre apellido del alumno y nombre del curso al que se anotó.



Código del documento

La *query* que debemos armar involucra las tres tablas, la tabla relación para conocer cómo se vincula el alumno con el curso y las otras para traer los datos significativos.

Ya con esos datos veamos lo que en realidad nos convoca en esta explicación que es como pasar datos que por cálculos, resultados de *queries* o ingresados desde el teclado en una *activity* pasan a la *activity* que invocamos.

La *activity* que vamos invocar se llama [MainDocu](#).

Parte del código se escribe en la clase de la base de datos y otra parte en la *activity* origen de la invocación.

Veamos entonces:

- En la clase de la base de datos, que tiene el onCreate, el OnUpgrade y el código que usamos para otros eventos, vamos a crear una función.
- Esta función tiene un parámetro de entrada que para nuestro ejemplo será una matrícula de alumno. La salida de la función es un [cursor](#) que nos traerá el resultado de la *query*. La base de datos será de lectura y al [rawQuery](#) le pasamos la *query* y el parámetro.

```
fun realizarConsulta(parametro : String): Cursor{
    val db = readableDatabase
    val query = "SELECT nombre, apellido, nombreCur from alumno as a inner join relacion as r on a.matricula = r.matricula " +
        "inner join curso on nroC = curso WHERE a.matricula =?"
    return db.rawQuery(query, arrayOf(parametro))
} // fin realizarConsulta
```

WHERE a.matricula =?"

Como verás, [la consulta involucra 3 tablas](#) y el lugar del parámetro colocamos un signo de pregunta.

- Luego en la *activity* origen de la invocación el resto del código. En este ejemplo lo colocamos en un Button en el evento OnClickListener()
- La proyección de la consulta la guardamos en variables para llevarlas a la *activity* que tiene el diseño del documento (MainDocu).

Primero declaramos las variables: el botón, el EditText que recibe el ingreso desde teclado y la que nos permite invocar a la *activity* del diseño del documento.

```
val buttonD = findViewById<Button>(R.id.btnDoc)
val paraDoc = findViewById<EditText>(R.id.ediDato)
val intentar = Intent(packageContext, this, MainDocu::class.java)
```

Veamos el código del *button*.



Código del button

```
buttonD.setOnClickListener(){ it: View!
    // Instanciar el helper de la base de datos
    val dbHelper = Datos( contexto: this)
    // Obtener una instancia de SQLiteDatabase
    val db = dbHelper.readableDatabase
    // Llamar a la función de consulta con INNER JOIN y parámetro en WHERE

    val parametro = paraDoc.text.toString()
    val cursor: Cursor = dbHelper.realizarConsulta(parametro)

    // Procesar los resultados del cursor
    if (cursor.moveToFirst()) {
        do {
            // Obtener valores de las columnas
            val uno = cursor.getString(cursor.getColumnIndex( columnName: "nombre"))
            val dos = cursor.getString(cursor.getColumnIndex( columnName: "apellido"))
            val tres = cursor.getString(cursor.getColumnIndex( columnName: "nombreCur"))
            // Uso del putExtra para llevar los datos a la otra activity
            intentar.putExtra( name: "nombreD", uno)
            intentar.putExtra( name: "apellidoD", dos)
            intentar.putExtra( name: "cursoD", tres)
        } while (cursor.moveToNext())
    }
    // Cerrar el cursor y la base de datos después de usarlos
    cursor.close()
    db.close()

    startActivity(intentar)
}
```

Observa que el código está dentro del evento del botón, instanciamos a la base, ejecutamos la función [realizarConsulta](#) con el parámetro. Posicionamos el cursor en la primera fila. En este ejemplo sabemos que arroja una única fila, no obstante dejamos el ciclo *do ... while* para que veas como realizalo.

Las variables llamadas [uno](#), [dos](#), [tres](#) almacenan los 3 atributos que proyecta la consulta que son [nombre](#), [apellido](#), [nombreCur](#) respectivamente. Estos nombres son exactos a los declarados en la creación de las tablas.

Para tomar los valores usamos el [putExtra\(\)](#), este tiene dos parámetros, el primero que se escribe entre comillas, es el que contiene el dato que usamos en la otra *activity* y el segundo a la variable que guarda el resultado del cursor.

[Veamos cómo queda el código en MainDocu.](#)

```
package com.example.nueva_dos

import android.annotation.SuppressLint
import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainDocu : AppCompatActivity() {
    @SuppressLint("SetTextI18n")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main_docu)

        val nombreD = intent.getStringExtra(name: "nombreD")
        val apellidoD = intent.getStringExtra(name: "apellidoD")
        val cursoD = intent.getStringExtra(name: "cursoD")
        val uno = findViewById<TextView>(R.id.txtUno)
        val dos = findViewById<TextView>(R.id.txtDos)

        uno.text = "El alumno $nombreD $apellidoD"
        dos.text = " Se inscribio en el curso: $cursoD"
    }
}
```

Por último en MainDocu, usamos esas variables en el lugar que seleccionamos. Para que nos muestre su valor debemos anteponer el signo \$ al nombre que le asignamos en el putExtra() y las recuperamos con el método [getStringExtra\(\)](#)



Tips para el diseño del carnet de socio

A continuación te presentamos una serie de tips para que tengas en cuenta al momento de diseñar un carnet de socio:

- ☒ El carnet del socio lo visualiza el empleado y el socio.
- ☒ El diseño del carnet del socio puede tener un formato igual para todos y lo que cambia es el nombre y apellido del socio. Esos son los datos básicos, como dato extra se puede incluir la fecha de vencimiento de la cuota.
- ☒ Todos estos datos están almacenados por lo tanto para obtenerlos debés realizar la consulta que proyecte lo que necesitás en el diseño.
- ☒ Si la visualización la hace el empleado, podés guardar en variables los datos mientras se ejecuta el ingreso del nuevo socio. Para transportar los datos al diseño del carnet usás lo mismo que explicamos en el diseño del documento.
- ☒ Si la visualización la hace el socio tendrás un poco más de trabajo, analicemos por qué.
- ☒ El socio ingresa su alias y contraseña en la App, entonces debés hacer una consulta para ver si existe esa combinación.

¿Cómo?

De la misma forma que vimos en la semana 10 como buscar el curso proyectando el contador `count(*)` que nos indica cuantas filas tiene la consulta.

- ☒ Si es distinta a 0 quiere decir que hay coincidencia, entonces buscá los datos que deben estar en el diseño del carnet del socio.

Usamos entonces lo que vimos en la explicación de esta semana en el diseño del documento.
Crearás muchas funciones, esto permite tener ordenado el código.



Listado de socios

Nos encontramos ante el último módulo del Club Deportivo: [el listado diario de socios a vencer](#). Este componente es fundamental para identificar rápidamente a los socios cuyas cuotas están pendientes, impidiéndoles el acceso a las instalaciones.

Flexibilidad

En el criterio "a vencer" permite ajustarse a la lógica deseada. El listado puede incluir socios cuya fecha de vencimiento es el día en curso o aquellos cuya fecha corresponde al día anterior, considerando que hasta este último día aún pueden disfrutar de las instalaciones y servicios del club.

Siguiendo la tradición de compartir consejos, ofrecemos sugerencias valiosas para la creación del código correspondiente a este módulo.

Este enfoque estratégico no solo resalta la importancia de mantener al día los registros de los socios, sino que también subraya la flexibilidad y adaptabilidad requerida para el desarrollo de la aplicación del Club Deportivo.

Con este último módulo, cerramos un ciclo de aprendizaje enfocado en la aplicación práctica de conceptos para ofrecer una experiencia sólida y eficiente en la gestión de socios.

Preguntas clave de este tema:

- ¿Cuál es la relevancia del listado diario de socios a vencer en el contexto del Club Deportivo?
- ¿Cómo se puede ajustar el criterio "a vencer" según la lógica deseada para obtener información específica en el listado?
- ¿Qué componentes pueden utilizarse para mostrar eficientemente el listado y cómo difieren entre sí, especialmente en el caso de GridLayout y GridView?
- ¿Cuál es la importancia de incluir la cláusula "order by" en la consulta SQL para garantizar un orden adecuado en el listado?
- ¿Cuál es la experiencia del usuario al interactuar con el módulo del listado diario?

¿Listo/a para empezar? ¡Adelante, el mundo del desarrollo te espera!



Introducción

Tenemos frente a nosotros el último módulo del Club Deportivo.

El **listado diario de socios a vencer** es una información muy importante para conocer rápidamente a los socios que no pueden ingresar por no tener su cuota al día.

Como nos dice **a vencer**, el criterio utilizado para la búsqueda puede ser de acuerdo a la lógica deseada.

Para tener en cuenta:

- El listado puede contener los que tiene como fecha de vencimiento los del día propiamente dicho, o aquellos que tengan la fecha del día anterior considerando que hasta ese día pueden hacer uso de las instalaciones y servicios del club.

Como en otras oportunidades, te dejamos algunos tips para la creación del código.



Tips para el Listado diario

Cuando hablamos de listado, la información tiene una determinada estructura, ya que debe estar ordenada por algún criterio.

- ☒ Esta estructura suele tener columnas y filas y entre las columnas los datos se ordenan por jerarquía de importancia que disminuye a medida que nos acercamos al margen derecho, esto se debe al sentido de escritura (izquierda a derecha).
- ☒ Los datos a mostrar son los resultantes de la proyección de la consulta SQL que ejecutamos en la base de datos involucrando las tablas que sean necesarias en función del modelo de datos; además cuenta con la condición de filtro que en este caso es la fecha.
- ☒ No hay que olvidar incluir en la consulta el ***order by*** de esta manera nos aseguramos que las filas resultantes ya estén ordenadas según el criterio adoptado. Este criterio puede ser apellido y nombre de los socios o que estén ordenados por el número de socio asignado entre otros.

Veamos un ejemplo y el componente que podemos usar para mostrar los datos...



Lógica para obtener el listado

Repasemos la experiencia del usuario para este módulo.

- El usuario es el [empleado](#) y accede cuando hace click en el botón que da origen al módulo.
- Entonces se debe pasar a otra *activity*, lo que usamos es el [intent](#).
- Podemos en el evento `OnClickListener()` del botón ejecutar la consulta (como hicimos en la semana 11) que se encuentra almacenada en un [cursor](#).
- En esta instancia de codificación imaginamos que la consulta la colocaste en la clase de la base de datos y que la función tiene como salida al cursor.

La diferencia que tenemos con lo implementado en la semana 11 es que el cursor puede tener [una o varias filas](#).

El componente que podemos usar para visualizar el listado puede ser un [GridLayout](#) o un [GridView](#).

Veamos en que difieren:

- ❖ **El GridLayout** es un contenedor que organiza sus hijos en una cuadrícula rectangular. Se puede especificar el número de columnas y filas, y cada elemento puede ocupar una o más celdas en la cuadrícula. Es más flexible y permite tener más control sobre la disposición de los elementos en la cuadrícula.
- ❖ **El GridView** es un componente específico de Android que hereda de ***AbsListView*** y se utiliza para mostrar datos en una cuadrícula bidimensional. Es más adecuado cuando se trabaja con conjuntos de datos grandes y se necesita una vista de lista que se organice en varias columnas.

En nuestro caso vamos a utilizar un GridLayout.

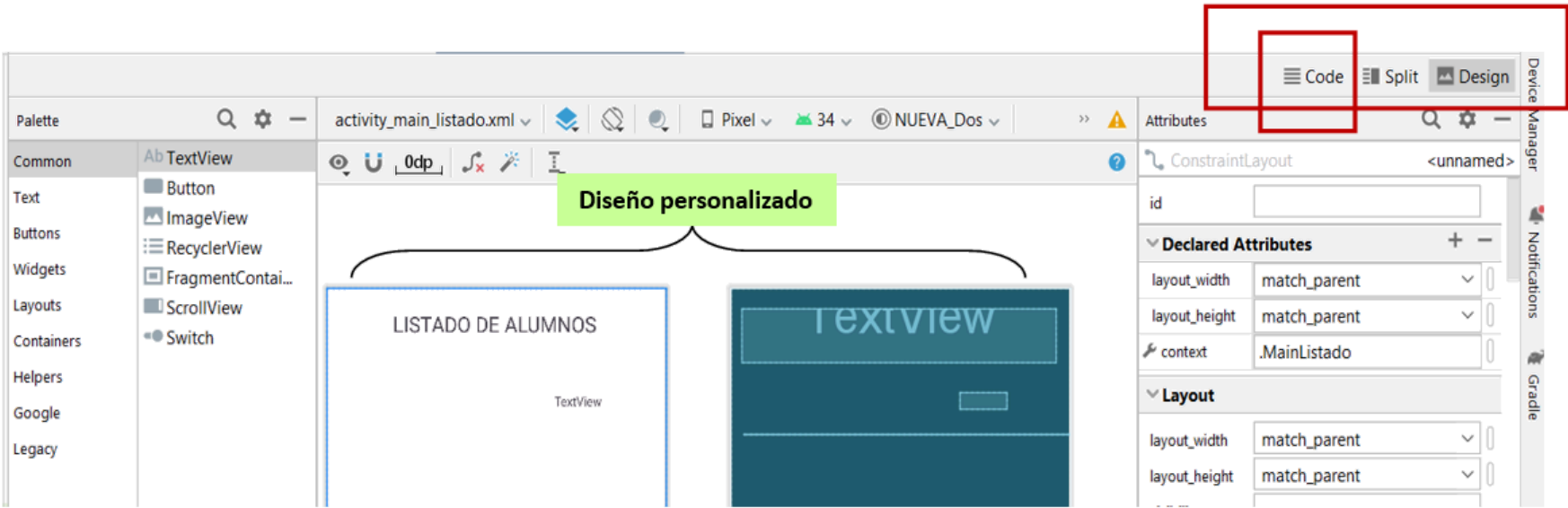


Diseño del listado

Lo primero que debemos hacer es el diseño del contenedor; vamos a la *activity* correspondiente (en este ejemplo se llama *activity_main_listado.xml*) e introducimos el *GridLayout*.

En este caso es conveniente agregar al archivo el código del componente.

Te mostramos como hacerlo:



La imagen te muestra el archivo XML de la *activity* con 2 TextView, para mostrar el título y la fecha actual, para agregar el GridLayout vamos a la solapa de código (resaltada con un cuadro en rojo en la imagen) y agregamos lo referido al componente que está resaltado en la imagen.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainListado">

    <GridLayout
        android:id="@+id/gridLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginStart="16dp"
        android:columnCount="3"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.222"></GridLayout>

    <TextView
```

Asignamos su id y es fundamental determinar la cantidad de columnas, esto lo podemos hacer buscando *columnCount*; en este caso la cantidad es tres porque el ejemplo cuenta con 3 columnas.

Ya con el diseño preparado pasemos al código.



Código para el listado en la clase de la base de datos

Como venimos razonando, lo primero es construir la consulta a la base de datos con la información que deseamos visualizar en el listado, es obvio que esa función debe estar en la clase de la base de datos.

```
package com.example.nueva_dos

import android.annotation.SuppressLint
import android.content.ContentValues
import android.content.Context
import android.database.Cursor
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class Datos(contexto: Context): SQLiteOpenHelper(contexto, name: "BaseDos.db", factory: null, version: 2) {
```

El siguiente código muestra la consulta:

```
@SuppressLint("Range")
fun obtenerDatosComoLista(): List<List<String>> {
    val datos: MutableList<List<String>> = mutableListOf()

    // Realizar la consulta y obtener un Cursor
    val db: SQLiteDatabase = readableDatabase
    val sql : String = "Select apellido, nombre, nombreCur " +
        "from alumno as a inner join relacion as r on a.matricula = r.matricula " +
        "inner join curso on nroC = curso order by apellido"
    val cursor: Cursor = db.rawQuery(sql, selectionArgs: null)

    // Iterar sobre el Cursor y agregar los datos a la lista
    while (cursor.moveToNext()) {
        val fila: MutableList<String> = mutableListOf()
        fila.add(cursor.getString(cursor.getColumnIndex( columnName: "apellido")))
        fila.add(cursor.getString(cursor.getColumnIndex( columnName: "nombre")))
        fila.add(cursor.getString(cursor.getColumnIndex( columnName: "nombreCur")))
        datos.add(fila)
    }

    // Cerrar el Cursor después de usarlo
    cursor.close()

    return datos
} // fin obtenerDatosComoLista
```

Observamos que tenemos variables declaradas de tipo *SQLiteDatabase Cursor MutableList<List<String>>*

También en otra variable declaramos nuestra consulta (observá que en este caso **no** cuenta con condición de filtro en el *where*) que ya tiene el *order by* para que los datos estén ordenados.

El resultado proyectado se encuentra en la variable de tipo Cursor y nos movemos en cada una de las filas con el *While*.

La variable de nombre *fila* arma cada columna de la grilla y coloca el contenido del atributo que mostramos en el *select* de la consulta.

Finalmente retornamos la lista en *return datos*.



Código para el listado en la activity que tiene el GridLayout

Al comenzar el código se debe controlar que figuren los **import** correspondientes a los componentes y clases que manejamos en la *activity*, por lo tanto **debés tener el GridLayout, el de la fecha (si la manipulás), etc.**

Pasemos al código.

```
class MainListado : AppCompatActivity() {
    @RequiresApi(Build.VERSION_CODES.0)
    @SuppressWarnings("MissingInflatedId")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main_listado)

        var hoy: TextView
        hoy = findViewById(R.id.txtHoy)
        val fechaHoy: LocalDate = LocalDate.now()
        hoy.text = fechaHoy.toString()

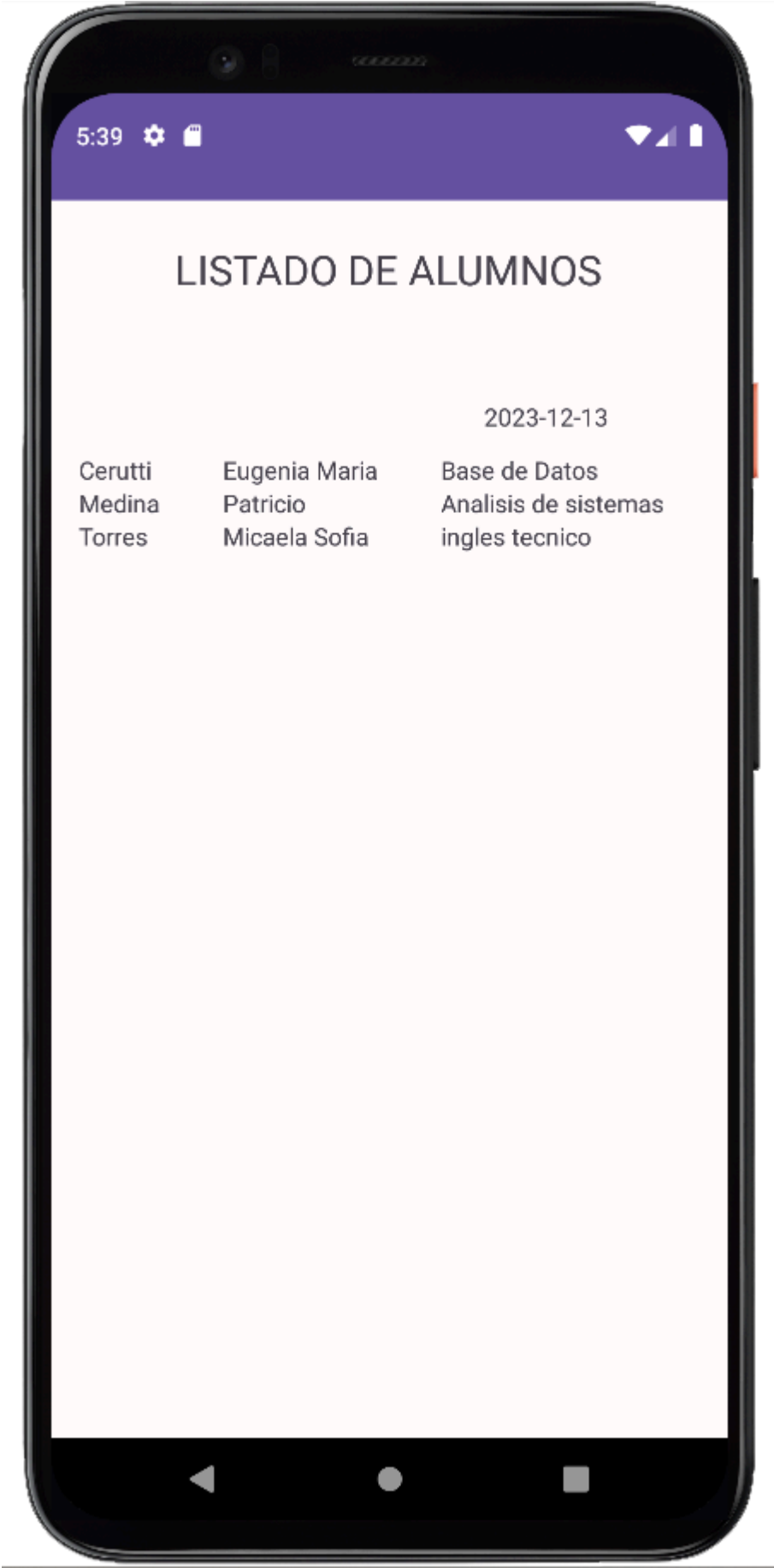
        val GridLayoutResultado: GridLayout = findViewById(gridLayout)
        val base = Datos(contexto: this)
        // Obtener resultados de la consulta
        val resultadosConsulta: List<List<String>> = base.obtenerDatosComoLista()

        // Crear elementos de vista (TextView) y agregarlos a la grilla
        for (fila in resultadosConsulta) {
            for (dato in fila) {
                val textView = TextView(context: this)
                textView.text = dato
                val params = GridLayout.LayoutParams()
                params.columnSpec = GridLayout.spec(GridLayout.UNDEFINED, weight: 1f)
                params.columnSpec = GridLayout.spec(GridLayout.UNDEFINED, weight: 1f)
                textView.layoutParams = params
                GridLayoutResultado.addView(textView)
            }
        }
    } // fin override fun
} // fin class
```

Como mi diseño, muestra la fecha comienzo con la declaración de las variables correspondientes.

- Luego tenemos un bucle **for** que se utiliza para iterar sobre cada fila de esa lista de listas.
- En la variable `resultadosConsulta`, declarada como `List<List<String>>`, se guarda el resultado de la ejecución de la función que arma la lista con los datos de la base de datos, es decir lo que retorna el código explicado en la página anterior de este libro; `resultadosConsulta` representa entonces los resultados de una consulta, donde cada elemento de la lista exterior (fila) es otra lista que contiene los valores de una fila de la consulta.
- Como es una lista bidimensional el bucle externo (**`for (fila in resultadosConsulta)`**) itera sobre cada elemento de la lista exterior, que es otra lista. La variable `fila` ahora representa una fila completa de la consulta.
- El bucle interno (**`for (dato in fila)`**) itera sobre cada elemento dentro de esa fila (que son `String` en este caso), permitiendo acceder a los valores individuales de cada fila.
- Finalmente con el **`addView(textView)`** visualizamos los datos.

Nuestro ejemplo muestra la lista de alumnos ordenada por apellido, mostrando apellido, nombre y nombre de curso.





Etapa 5

Dado el limitado tiempo de cursada, la práctica y revisión de los contenidos de la etapa 5 se ofrece de manera opcional, sin impacto en las evaluaciones. No obstante, reconocemos su valor para comprender integralmente el ciclo de vida de una aplicación y lo ponemos a disposición como recurso adicional.

La etapa despliegue y distribución de la aplicación es crucial tras el desarrollo de una aplicación, ya que se implementa la aplicación en entornos de producción, asegurando su funcionalidad y rendimiento. Se lleva a cabo la distribución a usuarios finales a través de plataformas como tiendas de aplicaciones. Se realizan pruebas exhaustivas para garantizar la estabilidad y se gestionan actualizaciones y correcciones. Además, se establecen estrategias de promoción y marketing para aumentar la visibilidad. Esta etapa asegura una transición eficiente del desarrollo a la disponibilidad pública, maximizando la accesibilidad y la satisfacción del usuario.

Hacé clic [aquí](#) para recorrer los contenidos de la etapa 5.